Client internals and version profiles, part 1 of 4

Client internals and version profiles, part 1 of 4

This part contains 27 notes, from `client-internals/prior-attempts.md` through `client-internals/live-trace-early.md`.

Reading order for the client notes, not directory order. Overview and architecture come first, including how a version id and a version profile are read. A version timeline follows: precursors and early clients, then 1.x, 2.x, and later clients. Topic notes are grouped as movement, combat, the HUD and presentation, then files and the session. Dated decompile-finding batches are last, under Batch notes, each batch kept together and the batches in date order.

Overview

Architecture of the 2.05 client, the build catalogue, and how a version id and a version profile are read.

`client-internals/prior-attempts.md`

Prior attempts to understand the Graal environment

Date: 2026-10-28

Cleaned summary of the analysis passes that preceded the current notes. No raw dumps.

Timeline of understanding

Period	Focus	Outcome
2026-09-30	Entry, forms, formats, config, render, input, anti-debug	Architecture collapsed; GLEVNW01, drop-files, idle/frame identified
2026-10-02	GS1 dispatch, row table, pattern letters, list flush	DispatchGs1 plated; zlib framing confirmed; version token located
2026-10-03	Session VMT, sign decode, tile walk, debugger runs	SessionRecv cases, DecodeSignText, type-0 vs type-1/2 collision
2026-10-04	List ports, world logins, cross-version tokens	14921/14922/14923 behaviour; GNW03081 construction; world rejections
2026-10-14	Seihai drop-ins, offline slice	Formats locked; script shape known; pics1 blit and CmdCall still open

What remained open at decommission

- Full recovery of type-0 allow lists (GZF lacked PE image)
- Fifteen unplated GS1 opcodes that share the default slot
- Raw gserver pcap (described but never archived)
- List-host S-boxes and remaining needles

These gaps are documented in the Drive zip and in the later cleaned notes of this repository. They are not blockers for the offline slice already exercised by Seihai.

Relationship to Seihai

The cleaned notes here (tile-walk, sign-text, gs1-commands, session-opcodes, config, rendering, offline-training) were written so Seihai could continue without the raw decompile corpus. The Drive zip preserves the original passes for anyone who later needs the full trail.

`client-internals/client-architecture-205.md`

Client 2.05 rev 2 architecture

Date: 2026-10-21

Build: `GNW03081`, the "2002ver9" leak build in GServer's `versions.txt`. Delphi (Borland), DelphiX (DirectDraw/DirectSound/DirectInput), zlib 1.1.2. Derived from the class list and behavior of the 2.05 binary. Names only, no code. Tag: **2.05** unless marked.

Networking classes

Class	Parent	Role
`TGraalClient`	TObject	socket, out buffer, zlib flush, u16be frame
`TServerListClient`	TGraalClient	listserver link: base send (no cipher), receive handles ids 0 and 1
`TAdventureClient`	TGraalClient	game link: gen-3 insertion on send, a receive jump table that handles 59 ids, login builder
`TWebClient`	TObject	HTTP (web levels / `openurl`) [inf]
`TAdventureServer`	TObject	local or offline server stand-in [inf]

The game socket keeps the gen-3 iterator, the key, and a "first message sent" flag in the client object. Disconnecting resets the server name to `Offline`. Messages are buffered and flushed at 0x1000 bytes. The client drops the link if 256 KB is queued. A UDP socket handles peer-to-peer traffic (`protocol/udp-p2p.md`).

Game model

Class	Parent	Notes
`TAdventure`	TDXForm	main window and game loop
`TAdventureLevel`, `TBoard`, `TBoardModify`	TObject	level, 64×64 board, pending board changes

Class	Parent	Notes
`TLevelObject`	TDrawableObject	base for drawable level objects
`TAdventureExtra`, `TAdventureLeap`, `TAdventureBomb`, `TAdventureExplosion`, `TAdventureBall`, `TAdventureFlying`, `TAdventureCarry`	TLevelObject	items, leaps, bombs, explosions, fireballs, arrows, carried objects [inf from names]
`TServerPlayer`	TLevelObject	networked character. `TPlayer` (self) and `TEditorNPC` derive from it
`TComputer`	TSurfaceOwner	baddy ("compu")
`TAdventureHorse`	TSurfaceOwner	horse or boat
`TDisplayedNPCImg`, `TTempWeaponImg`	—	`showimg` overlays, weapon icons
`TAdventureWeapon`	TObject	weapon (name, image, GS1 script)
`TTileDef`, `TTempTile`	TObject	`addtiledef` and temporary tile overrides

Class	Parent	Notes
`TGraalMap`	TObject	big-map / minimap overlay [inf]
`TGraalAni`, `TGraalAniStep`, `TGraalAniPart`, `TGraalAniSprite`, `TGraalAniSound`	TObject	GANI model (see `formats/gani.txt`)
`TCachedStream`	TObject	file cache [inf]

Script engine

`TAdventureCommand`, `TAdventureOperation`, and `TAdventureVariable` are the parse-tree nodes, with `TArray`, `TInteger`, `TDouble`, and `TStringPosList` as values. Execution is a recursive tree walk. See `npc-library/gs1-reference-205.md`.

UI forms

`TConnectFrame`, `TReconnectFrame`, `TServerListFrame`, `TLoadLevel`, `TServerMessage`, `TShortMessage`, `TPlayerList`, `TPlayerListOptions`, `TProfileEdit`, `TOptions`, `TMoreOptions`, `TLinkList`, `TLinkOptions`, `TShowCodes`, `TPersonAction` (the NPC "Set Npc's Action" script editor, not a player menu), `TChestEdit`, `TDebuggerFrame`. The editor and the client are **one executable** in 2.05: link, chest, and NPC editing forms are built in. They split into separate programs at 2.3 (see `version-tokens.md`).

Other formats seen in 2.05

- **GWEBL001**: a web-level cache file. It starts with the magic `GWEBL001`, then GSTRING fields and a GSHORT version that is checked against the client. The client also computes a running product hash over a key string, seeded with `0x04A80B38`. **2.05** The field list is not fully traced.
- A Delphi DES cipher class (`UDesCipher`) is linked in. It is not the game wire codec. It protects the saved passwords, the hidden listserver host (`<list-host-classic>`, port 14921, list `newmain`), and the `GWEBL001` web-level cache. **2.05**
- Image decoders: GIF (`TGifAnimation`), MNG (`TMNGAnimation`), JPEG. MNG animation fits with `gif2mng`.

Level magics accepted

`GLEVNW01`, `GR-V1.00`–`GR-V1.03`, `Z3-V1.03`, `Z3-V1.04`. No `GR-V1.04`. A `GRMAP001` reader is also present (with the generated-map and `HEIGHTS` keys), but no static caller was found; see `formats/gmap.md`.

`client-internals/client-assets.md`

Client asset tree

Date: 2026-10-14

Measured from the official client install. No decompile material.

Tile sheets

File	Size	Role
pics1.png	2048×512	Standard tiles, 128×32 of 16 px, 4096 ids
picso.png	1024×512	Offline tileset (type 1)
sprites.png	128×452	Sprite sheet
letters.png	—	Bitmap font

Tile id `t` maps to `col = (t % 16) + (t // 512) * 16`, `row = (t // 16) % 32`, source rect `(col*16, row*16, 16, 16)` on pics1. The sheet is eight side-by-side blocks of 512 ids, each 16 columns by 32 rows, filled row by row. (Corrected 2026-10-25: an earlier version of this note said `col = t % 128`, `row = t // 128`, which is wrong. See `tools/level-editor/222-reference.md` section 5.)

Tiledef

Offline line recovered from the tree:

```
picso.png,offlinetutorial,1,0,0
```

Format is `image,prefix,type[,x,y]`. Type 0 is the old Graal order (pics1). Type 1 is the new order. `addtiledef2` paints a rectangle on top of the base sheet.

levels/ counts (files on disk)

Dir	Files	Contents
ganis/	93	86 .gani plus images
midis/	49	music
heads/	35	33 png, 2 mng
weapons/	33	weapon images, 2 txt
bomys/	29	bomy images and ganis
emoticons/	29	png and mng
webgifs/	26	mostly png
backpals/	23	background palettes
bodies/	20	body images
images/	19	misc images
smilies/	15	
baddies/	10	one sheet per baddy
shields/	7	
swords/	6	
npcs/	5	script text
horses/	4	
hats/	4	

Dir	Files	Contents
tutorial/	4	
letters/	2	blueletters, graal2002letters
snapshot/	2	ss_arrow, ss_bomb
tiledefs/	1	offline tiledef line

Other top-level

- maps/ holds bigmap.txt, minimap material.
- arrays.dat / arrays2.dat and the corresponding .sig files.
- bass.dll, basswma.dll, hvdi.dll.
- readme.txt documents HawkVoice (LGPL), joystick calibration, and classic controls (F1 options, F4 editor, F5 load, arrows/IJKL walk, S/space strike, A act, M map).

`client-internals/build-catalogue.md`

Client build catalogue (1998–2001)

Date: 2026-10-21

This covers every client in the uploaded archive set: 39 top-level archives plus 16 from the second upload, 55 in total. There are 52 Delphi client executables (53 with GraalX, see below), the Graal3D side project, and the 1998 Java precursor. Everything here comes from static reads of the binaries (PE headers, resources, strings, and immediates in the code) and from the release notes shipped beside each exe. Nothing in this file comes from a decompiler.

How each column was read

Column	Source
MD5 / size	The exe as shipped in the archive.
Compiled	The resource-directory timestamp, a DOS date/time that Delphi sets at link time, in local time. The COFF timestamp is useless for these builds because every Delphi build carries the fixed 1992-06-19 value.
FileVersion	The VERSIONINFO resource. It only exists from 1.37 on and is sometimes stale (2.05 still says 2.0.4.0).
Token	The login version string. Up to 1.37 it is a plain string literal. From 1.38 on it is built one character at a time, so `strings` misses it, and it has to be read from the immediates passed to the char-to-string helper.
Gen	The encryption generation, using the GServer-v2 numbering. 1 means no compression. 2 means zlib with no encryption (a `nodecompressionerror` option exists). 3 means zlib plus the one-byte LCG insertion (see `protocol/` codec notes). Gen 3 is detected by the connection-object initialiser storing seed `0x04A80B38` and then the `Random & 0x7F` key. Two other uses of the same seed (web-level cache file encryption, from 1.3 on) are excluded.
Server / list	1.39 and earlier: the default host:port the connect dialog is pre-filled with (no listserver support). 1.4x and later: the listserver host:port and the list name sent after connecting.
Levels	Which level and map magics the loader compares against. `Z3-V` is `.zelda`, `GR-V` is `.graal`, `GLEVNW01` is `.nw`, and `GRMAP001` is `.gmap`. `Graalsav` is the save-game magic.

Catalogue

Sorted by compile date. "Archive" is the archive file name without `.7z`.

Archive	Release name	Compiled	FileVersion	MD5	Size	Token	Gen	Server / list	Levels and saves	Notable
zeldaonline2	Zelda Online (Java applet)	1997–98	–	–	–	none	text lines	host:14801	`Z3-V1.02/1.03`	Java predecessor, see `zelda-online-1998.md`
1stgraal-1	first Delphi beta	1998-09-06 23:54	–	07452f8941fdfcdce878b035156e701c	654336	none	1	the loopback address:14801	Z3-V1.03/1.04 only; Graalsav1	Two-player local game, no `.graal` loader yet, no DirectSound
2ndgraal	beta	1998-09-16 01:33	–	f74b35fc7975a4873087f7fd2aa1d79b	777216	none	1	the loopback address:14801	+ GR-V1.00–1.01; Graalsav1	First `.graal` loader
3rdgraal	beta	1998-09-30 01:47	–	fc78fbc96bdbc6ca22ae56d3ed8635ab	665600	none	1	the loopback address:14801, dialog 14802	GR-V1.00–1.01; Graalsav1	Connect dialog gains a second port field (14802)
4thgraal	beta	1998-10-05 01:33	–	e040af2e19912419ab80aa6dd202b4ea	666112	none	1	as above	GR-V1.00–1.01; Graalsav1	
4thgraalalpha	beta (alpha)	1998-10-12 22:54	–	8cae23a58e1b1b10f3149bdbe53d0431	711680	none	1	the loopback address:14801	GR-V1.00–1.01; Graalsav2	Built after 4thgraal despite the name; DirectSound appears
5thgraal	beta	1998-10-19 02:27	–	0e7c3573f0a916f8b2376060e768fe96	723968	none	1	the loopback address:14801, dialog 14802	GR-V1.00–1.02; Graalsav2	GR-V1.02
5thgraalalpha	beta (alpha)	1998-10-29 02:21	–	e54397e7d4f59d30b77d5cf0475335a1	739840	none	1	the loopback address:14801	GR-V1.00–1.02; Graalsav2	Built after 5thgraal
6thgraal	beta5	1998-11-09 02:08	–	53c3caf3d930534db3f44f346f5d651a	750080	none	1	the loopback address:14801, dialog 14802	GR-V1.00–1.02; Graalsav3	Published by Cyberjoueur; web levels from <cyberjoueur-host>

Archive	Release name	Compiled	FileVersion	MD5	Size	Token	Gen	Server / list	Levels and saves	Notable
7thgraal	beta7	1998-12-04 00:28	–	deb40c31268e499fc4436b8d0687f05d	766976	none	1	as above	GR-V1.00–1.03; Graalsav4	First GR-V1.03 (the last `.graal` revision any build here reads)
8thgraal	beta8	1998-12-08 01:13	–	a49aa1b773e41741f211867fd5ea55d7	775680	none	1	as above	GR-V1.00–1.03; Graalsav4	
9thgraalalpha	beta9 (alpha)	1998-12-16 02:36	–	e226e1b83fd437b0aede968fb395b911	788992	none	1	as above	GR-V1.00–1.03; Graalsav5	Built before 9thgraal
9thgraal	beta9	1998-12-18 01:53	–	f0185ff8525a13f2f4e5997f649bc250	793088	none	1	as above	GR-V1.00–1.03; Graalsav5	
10thgraal	V1.0	1998-12-22 03:24	–	f1f34b4479a8ad7614bd8f7ce56112c6	795648	none	1	as above	GR-V1.00–1.03; Graalsav5	1.0 final: map heads, Alt+1/2 screenshots, Alt+3 minimap
11thgraal	V1.0	1998-12-23 01:00	–	a25521262190 7affc94a4483c6dfc8e3	795648	none	1	as above	GR-V1.00–1.03; Graalsav5	Adds the altern.org RealWorld web level
12thgraal	V1.0	1999-01-05 01:04	–	788ae59e9419806024d14d3e5b611971	794624	none	1	as above	GR-V1.00–1.03; Graalsav5	
13thgraal	V1.1 beta1	1999-01-12 01:15	–	a211a0ecf3b58daa2f46759444e18bd3	797184	none	1	as above	GR-V1.00–1.03; Graalsav6	`this.` vars, canbepushed/pulled, sleeping/sitting, `.gsave`
14thgraal	V1.1	1999-01-15 02:20	–	d0999d44fcd9c226f350fcf204618885	858112	none	1	as above	GR-V1.00–1.03; Graalsav7	

Archive	Release name	Compiled	FileVersion	MD5	Size	Token	Gen	Server / list	Levels and saves	Notable
15thgraal	V1.1	1999-01-25 00:12	–	9a7d66dd5c8c506ad4d3e63ee4a9a1f1	878592	none	1	as above	GR-V1.00–1.03; Graalsav7	Faster script interpreter, IP caching, `puthorse` boats
graal1.1.9	V1.1.9 Gold Edition	1999-02-11 23:50	–	0a6eef88acb4c0fce6e84eee3e76c04a	882176	none	1	as above	GR-V1.00–1.03; Graalsav7/8	Ships `Gserver.exe` (1999-01-22, port 14802, no token); home moves to <graalonline-site>
graal1.1	V1.1 (readme)	1999-02-14 23:03	–	1a9580655cce686c602c01daa251c453	882176	none	1	as above	GR-V1.00–1.03; Graalsav7/8	Same size as 1.1.9 but linked 3 days later
graal121	V1.2.1	1999-05-10 18:29	–	8da2e8284dd8b315a28e2d4fccae79ac	973824	`GSERV007`	2	<list-host-classic>:14804	GR-V1.00–1.03; reads Graalsav9/Graalsv10/Graalsv11, writes Graalsv11	InstallShield 3 package (the exe is inside `_SETUP.1`); "packet compression" (zlib 1.0.4); NPC weapons; GServer and RC on the same token
graal128b2	v1.28 beta2	1999-11-23 01:09	–	8e3c1a7233864da994a0db26f1811212	1196544	`GSERV013`	2	<list-host-classic>:14804	GR-V1.00–1.03; reads Graalsav9/Graalsv10/Graalsv11, writes Graalsv11	"Direct data packages" (positions sent straight to room players); ships GServer (accepts GSERV007–013) and RC 1.28 (GSERV011)
graal13b	v1.3 test	1999-11-29 02:03	–	b62aaa4d606a519e7f4689b3398342 9b	1311744	`GSERV014`	2	<list-host-classic>:14804	GR-V1.00–1.03; reads Graalsav9/Graalsv10/Graalsv11, writes Graalsv11	Alignment system; levels cached on disk (web-level cache seed first appears); ships GServer13 (accepts 007–014)
graal131	V1.31	1999-12-07 02:56	–	bff87a95a34399b2d35b243f281169a7	1318400	`GSERV016`	2	<list-host-classic>:14900	GR-V1.00–1.03; reads Graalsav9/Graalsv10/Graalsv11, writes Graalsv11	Default port moves to 14900; UDP options (`noudp`, fixed port 14899)
graal132	V1.32	2000-01-12 01:13	–	42b2596af4b75bcf82db03e0fc274f7d	1328128	`GSERV018`	2	<list-host-classic>:14900	GR-V1.00–1.03; reads Graalsav9/Graalsv10/Graalsv11, writes Graalsv11	Double-click fullscreen
graal3d	Graal3D	2000-02-05 01:57	–	305c985a020cec6ab8d81ac95d447343	605184	none	own	the loopback address:14901 (UDP 14898)	none	Separate 3D client plus its own GServer.exe (14901); zlib linked, no Graal level loaders

Archive	Release name	Compiled	FileVersion	MD5	Size	Token	Gen	Server / list	Levels and saves	Notable
graal133	V1.33	2000-02-21 00:36	–	a3667ce55a5c67cf02588201 4b9540b0	135014 4	`GNW20 020`	2	<list-host-classic>:14900	GR-V1.00–1.03; carries Graalsav9/ Graalsv10/ Graalsv11 (string scan)	First GNW token; PNG support; 4-section player list
graal134	V1.34	2000-03-23 00:38	–	9e403616e73a6079ac37d65d 475c81cd	134963 2	`GNW22 030`	2	as above	GR-V1.00–1.03; carries Graalsav9/ Graalsv10/ Graalsv11 (string scan)	Script debugger (F6); MNG appears
graal135	V1.35	2000-04-13 23:17	–	fe9d99c4bf6e6bc9b1b12782 3f39646f	138956 8	`GNW13 040`	2	as above	GR-V1.00–1.03; carries Graalsav9/ Graalsv10/ Graalsv11 (string scan)	
graal136	V1.36	2000-05-12 17:34	–	018f51662c7c8862cafb9bfb d14dea41	139929 6	`GNW10 050`	2	as above	GR-V1.00–1.03; carries Graalsav9/ Graalsv10/ Graalsv11 (string scan)	Server player cap 220 → 28000; nickname limit
graal137	V1.37	2000-05-23 02:40	1.3.7.0	5e966a9d1a079af6d682b261 8674edb1	140134 4	`GNW21 050`	2	as above	GR-V1.00–1.03; carries Graalsav9/ Graalsv10/ Graalsv11 (string scan)	First VERSIONINFO; new sound-effects system
graal138	V1.38	2000-06-23 22:11	1.3.8.0	4401f7d02966af709c2df662 cebe34a1	142643 2	`GNW22 060`	2	as above	GR-V1.00–1.03; carries Graalsav9/ Graalsv10/ Graalsv11 (string scan)	Token now built char by char; `setnick`, `warpto`
graal139	V1.39	2000-08-07 02:49	1.3.8.0	62513ab884f1802a0bcd8114 abe349b9	144896 0	`GNW07 080`	2	as above	GR-V1.00–1.03; carries Graalsav9/ Graalsv10/ Graalsv11 (string scan)	New head-image handling; profiles. FileVersion not bumped
graal_gr1411	V1.41 rev1 (<graal in-site>-patched copy)	– (resource stamp missing)	1.4.1.1	a97740e711ffe9b698ec03712 d4c06af	150681 6	`GNW13 110`	**3**	list `<list-host-reborn>`:14923	GR-V1.00–1.03, GLEVNW01; carries Graalsav9/ Graalsv10/ Graalsv11 (string scan)	First server-list window (from 1.4) and first gen 3 build. See the note below on the patching
graalnm24	NewMain client (1.42 notes)	2001-01-10 19:55	1.4.1.1	229f05de7a43b98e223d8e1b 58bffc63	178329 6	`GNW08 011`	3	list <list-host-classic>:14921, "newmain" (plain)	GR-V1.00–1.03, GLEVNW01	First `.gani` animation system; ships GraalShop and RC-NewMain (GSERV021)

Archive	Release name	Compiled	FileVersion	MD5	Size	Token	Gen	Server / list	Levels and saves	Notable
graalnm26	NewMain client (1.42 notes)	2001-01-12 21:19	1.4.1.1	22a08ed154f67948e01b969b68cdb05e	1784320	`GNW08011`	3	as above	GR-V1.00–1.03, GLEVNW01	Adds GraalLevelGen 1.0
Graal-v2.0	v2.00	2001-02-01 14:54	2.0.0.0	c93e659e2cda36379aaea4706db20b38	1812480	`GNW24011`	3	list <list-host-classic>:14921, "newmain" (host DES-hidden)	GR-V1.00–1.03 (no `.nw`)	No 2/3/4-player mode; tutorial levels; server-side bush rupees. First DES-hidden list host
Graal-v2.0-rev1	v2.00 rev1	2001-02-02 19:19	2.0.0.1	30bd230e578d3a347866a87060efec20	1812992	`GNW24011`	3	as above	as above	
Graal-v2.0-rev2	v2.00 rev2	2001-02-05 02:32	2.0.0.2	fc4bb7e3b7ca9cd8ea6693a033635534	1815040	`GNW24011`	3	as above	as above	
Graal-v2.0-rev3	v2.00 rev3	2001-02-07 01:05	2.0.0.3	1ce5c1662b65246ae1c6559af5ff71f0	1815040	`GNW24011`	3	as above	as above	
GraalX (`.rar`, separate upload)	2.01-era side build	2001-03-07 21:40	2.0.0.3	d3ee52ef9d22a0d99b154b47c5f56f60	1821696	`GNW23021`	3	as above	GR-V1.00–1.03, **GLEVNW01**	2.01 plus the `.nw` reader and writer. See the note below
Graal-v2.01	v2.01	2001-03-08 14:59	2.0.1.0	08340a8868f1d11e0285cc70e51200fe	1815552	`GNW23021`	3	as above	as above	Level preloading on the map; showimg fixes
Graal-v2.01-rev1	v2.01 rev1	2001-03-15 21:10	2.0.1.0	010a32afce66729b83825880 9a97c0d9	1815552	`GNW23021`	3	as above	as above	
Graal-v2.02	v2.02	2001-04-15 20:48	2.0.2.0	60018a19ef671c5067438267 28c49718	1808384	`GNW15041`	3	as above	as above	Hats; HTML window navigation

Archive	Release name	Compiled	FileVersion	MD5	Size	Token	Gen	Server / list	Levels and saves	Notable
Graal-v2.02-rev1	v2.02 rev1	2001-04-15 23:16	2.0.2.0	e06ff79aded885a223b933617d456577	1808896	`GNW15041`	3	as above	as above	Speed-cheat protection; client language sent to server
Graal-v2.02-rev2	v2.02 rev2	2001-04-20 18:48	2.0.2.2	993a1829dbb6acaa2b46063f5efe636c	1807872	`GNW20041`	3	as above	as above	
Graal-v2.03	v2.03	2001-05-07 15:31	2.0.3.0	36972c74baad09ceb33dad94b0eb1e33	1836032	`GNW06051`	3	as above	as above	`triggeraction`, `move`; NPC-server support
Graal-v2.04	v2.04	2001-06-25 07:42	2.0.4.0	a40608308d4f890cb5e4efdd45b38ba4	1872384	`GNW24061`	3	as above	as above	`replacegani`, `addtiledef`/`addtiledef2`
Graal-v2.05	v2.05	2001-08-03 19:12	2.0.4.0 (stale)	9abce477945a1307bb2be9ed47d2f170	1891328	`GNW03081`	3	as above	GR-V1.00–1.03, **GLEVNW01, GRMAP001**	`.nw` and `.gmap` return; script-editor button; `setshape2` arrays
Graal-v2.05-rev1	v2.05 rev1	2001-08-04 03:26	2.0.5.0	2780f2f370396c580e1648696698c871	1895936	`GNW03081`	3	as above	as above	
graalnwb1	NewWorld beta 1	2001-08-06 04:13	2.0.5.0	88d1f84c96da1dcc3942b139fbd741c9	1820672	`G2N15041`	3	list <list-host-classic>:14921, "newworld" (DES)	as 2.05	Terrain heights, backtile areas, NW time variables (string arrays are nwb2 only; see `newworld-betas.md`); ships NPCControl, LevelFiller, TerrainGenerator, Greaditor and other tools
Graal-v2.05-rev2	v2.05 rev2	2001-08-16 17:24	2.0.5.2	f3fdf54d943fa91c89acba5bb695db99	1891840	`GNW03081`	3	list <list-host-classic>:14921, "newmain" (DES)	as 2.05	The decompile target; same bytes as `Graal_v2.05_rev2.exe`
graal21test2	v2.1 test 2	2001-09-21 17:54	2.1.0.0	d15d877822ad4145578107857c90fb3c	1748480	`GNW03081`	3	list <list-host-classic>:**14922**, "newmain" (DES)	as 2.05	New tiles, new starting window; the token is still 2.05's

Archive	Release name	Compiled	FileVersion	MD5	Size	Token	Gen	Server / list	Levels and saves	Notable
graal21test3	v2.1 test 3	2001-09-27 21:11	2.1.0.0	2116a7ba03648dcc00403553 4ec94de8	1753600	`GNW03081`	3	as above	as 2.05	
graalnwb2	NewWorld beta 2	2001-10-03 20:46	2.1.0.0	af2d21066659bed6af6c77e52b2cc0e1	1613824	`G2N15041`	3	list <list-host-classic>:14922, "newworld" (DES)	as 2.05	String arrays (`addstring` etc.), terrain heights with per-level `HEIGHTS` grids from the gmap; see `newworld-betas.md`

GraalX

GraalX came as a separate upload, `GraalX.rar`, holding a single `GraalX.exe`. Its VERSIONINFO says 2.0.0.3, the same stale value as 2.0 rev3. It was linked one day before 2.01 and carries 2.01's login token. Its classes and instance sizes, its GS1 command table (144 commands) and every packet and prop id it handles are identical to 2.01. Matched function by function, it shares 6350 identical functions with 2.01 and differs in only 7. The one real difference is `.nw` support. GraalX keeps the GLEVNW01 reader, writer and chest-line parser that NewMain 1.42 had and that 2.0 and 2.01 dropped. Several of those functions are identical to NewMain's. It also adds the GLEVNW01 magic checks and the `.nw` open-dialog filter that reappear in 2.05. So it looks like a 2.01 build compiled with the `.nw` code left in, for example a level-format test. Nothing in the binary says why it was built or who it went to.

Patterns worth knowing

The GNW token is a build date. It is `GNW` followed by DDMMY, with Y being the last digit of the year. Every build from 1.33 to 2.05 fits, and the token date is on or a few days before the link date: GNW20020 is 20 Feb 2000 (1.33, linked 21 Feb), GNW07080 is 7 Aug 2000 (1.39, linked 7 Aug), and GNW03081 is 3 Aug 2001 (2.05, linked 3 Aug). A rev that does not touch the protocol keeps the previous token (2.0 rev1–3, 2.01 rev1, 2.02 rev1, 2.05 rev1–2), and so does the 2.1 test line. So an unmapped token like GNW30123 can be read as 30 Dec 200x, which probably points to a 2.1-era build, since 2.1.0.x is known to use GNW31101.

Before GNW, the token was a protocol revision. `GSERVnnn` went 007 (1.21), 011 (RC 1.28), 013 (1.28b2), 014 (1.3 test), 016 (1.31), 018 (1.32), and 021 (RC-NewMain, Jan 2001). The bundled GServers accept a range rather than one token: GServer 1.28 accepts GSERV007–013 and GServer13 accepts 007–014.

Encryption generations by build.

Gen	Builds	How it was determined
1	1st Graal through 1.1.9	No zlib linked; no decompression-error option
2	1.21 through 1.39	zlib 1.0.4 linked; the 1.2 notes say "packet compression"; `nodecompressionerror` from 1.28b2
3	1.41 rev1, NewMain 1.42, all 2.0x, 2.1 tests, NW betas	Connection initialiser seeds the insertion LCG with `0x04A80B38` and a 7-bit random key. 1.39 lacks it

Server discovery. 1.39 and earlier have no listserver: the connect dialog is pre-filled with a host:port and the player edits it. The defaults were the loopback address on 14801 for the 1998 betas through 1.1.9, <list-host-classic>:14804 for 1.21–1.3 test, and <list-host-classic>:14900 for 1.31–1.39. The server-list window arrives with

1.4. NewMain 1.42 writes host, port and list name as plain char-by-char strings. From 2.0 the host is DES-hidden with the bit-reversed key described in the 2.05 notes. The key and ciphertext are the same in every 2.0x build and decrypt to `<list-host-classic>`. The list name chooses the main list ("newmain") or the NewWorld list ("newworld"). The port moves from 14921 to 14922 with the 2.1 test builds.

The 1.41 rev1 copy is patched. It contains both "<graalonline-site> Server List" and "<graalin-site> Server List", `<www-graalin-site>` register and homepage links, and `<list-host-reborn>:14923`. Its resource-directory timestamp is zeroed, which is what a resource editor leaves behind. This is the only 1.41 copy available, so the listserver host and port it shows may not be the original 1.41 values. The token (GNW13110 = 13 Nov 2000) and the code-level facts look untouched.

`.nw` support is not monotonic. NewMain 1.42 (Jan 2001) and the 1.41 rev1 copy contain the GLEVNW01 loader, but 2.00–2.04 contain no GLEVNW01 or GRMAP001 string, even as char-built strings. Both reappear in 2.05 (Aug 2001) together with gmap support. This suggests 2.0 branched from a pre-NewMain tree and the NewWorld code was merged back for 2.05. That conclusion is an inference from strings only. A live check (loading a `.nw` level in 2.04) would confirm it.

Save magics after 1.1.9. 1.21 to 1.32 write `Graalsv11` (with `sav` shortened to `sv` so the magic stays 9 characters) and read `Graalsav9` and `Graalsv10` too. A search for `Graalsav` misses the last two. 1.33 to 1.39 and the 1.41 rev1 copy carry the same three magic strings in the same arrangement as 1.21 to 1.32: `Graalsv11` is referenced from two places (writer and reader) and the other two from one (reader only). So they write `Graalsv11` too. The writer itself was traced only in 1.21 and 1.32. See `formats/graalsav.md`.

Compiler. Every build is Borland Delphi with linker 2.25, DelphiX for DirectDraw, and an RTL copyright reading "1983,97 Borland". Two compilers are in use, and the class metadata tells them apart. In the older layout the VMT self pointer sits 64 bytes before the VMT and the class name 32 bytes before it, which is the Delphi 2 layout. In the newer layout those offsets are 76 and 44, which is the layout from Delphi 3 on. Only 2nd Graal and 1.28 beta 2 onwards use the newer layout. They also link `TMultiReadExclusiveWriteSynchronizer`, which arrived in SysUtils with Delphi 4, so they are Delphi 4 builds. 1st Graal, 3rd Graal to 1.1.9, and 1.21 use the older layout and lack that class. Their exact compiler is unresolved: the layout says Delphi 2, but the RTL string "1983,97" is a year late for Delphi 2. A Delphi-2-level compiler shipped in 1997, such as C++Builder 1's, would fit both, but that hasn't been checked. Either way the switch to Delphi 4 happened between 1.21 (May 1999) and 1.28 beta 2 (Nov 1999), and 2nd Graal is an outlier built with the newer compiler. (An earlier version of this note said every build from 2nd Graal on has the class. A per-build check shows otherwise.)

Archive naming vs build order. The "Nth graal" archive names are not strictly chronological. 4thgraalalpha and 5thgraalalpha were linked after their non-alpha namesakes, and 9thgraalalpha before 9thgraal. `graal1.1` and `graal1.1.9` have identical sizes, but graal1.1 was linked three days after the Gold Edition.

Other executables in the archives

Archive	File	Compiled	MD5	Token	Port
graal1.1.9	Gserver.exe	1999-01-22	7882dba6ebb2ab1026de260c3043fa47	none	14802
graal121	GServer.exe	1999-05-09	bfcdaae0217d953124a9b21ff7858967	GSERV007	14804
graal121	remotecontrol.exe	1999-05-10	5932d6baa08289a65515342dfc4b9f5a	GSERV007	14804
graal128b2	GServerv.exe	1999-11-22	8d6bff85bda6e6c9558503afa6b7259d	accepts GSERV007–013	14804
graal128b2	RemoteControl1.28.exe	1999-10-20	d7150d1385ea11b812d10d30212f9757	GSERV011	14804

Archive	File	Compiled	MD5	Token	Port
graal13b	GServer13.exe	1999-11-29	d1d0470e53a60b8f000b9ea5da9d37b6	accepts GSERV007–014	14804
graal13b	RemoteControl13.exe	1999-11-29	1f2c19ab385c8b8af6236197f98ad735	GSERV011	14804
graal3d	GServer.exe	2000-02-05	aab1a34a1f196d4f4fddf12112ccbf52	none	14901
graalnm24/26	RemoteControl-NewMain.exe	2001-01-08	bc2b8a2e80acc09a6bb418970cff5a64	GSERV021	14804
graalnwb1	RemoteControl-NW.exe	2001-05-22	6fe24bdd4e135900fc40d7fb70bdcdd2	–	14804
graalnwb2	RemoteControl-NW.exe	2001-08-27	ce454437af8880f61ac2173b0bd11047	–	14804

Also present: gif2mng (2000-06-12, identical in every archive that has it), GraalShop (gani editor, 2000-12 and 2001-04), GraalLevelGen and GraalLevelGenNW, NPCControl (2001-07-13), LevelFiller with levelviewer, TerrainGenerator, PNGView, YoshiChess, and Greaditor (UPX-packed).

See also `drive-clients-inventory.md` (the Drive clients set: which of these builds exist as exes, file versions, and Wine start-up notes) and `../version-profiles/drive-clients-exes.tsv`.

`client-internals/drive-clients-inventory.md`

Drive clients set: what it contains (2026-10-24)

Derived facts from a shared Drive archive of old client builds (one zip, about 122 MB). It holds 73 exes in 55 folders. Full list with sizes and PE file versions: `version-profiles/drive-clients-exes.tsv`. The PE timestamps in these exes are all the same dummy 1992 date, so they say nothing about build date; only the file-version strings are useful, where present.

Versions present

- Early: 1st, 2nd, 3rd, 4th (+alpha), 5th (+alpha), 6th to 15th Graal ("Nth graal" folders), 1.1, 1.1.9 (+ Gserver), 1.21 (setup only), 1.28b2 (+ GServer, RemoteControl), 1.3b (GServer, RemoteControl), 1.31 to 1.39, 1.411, Graal 3D (+ GServer).
- 2.x: **2.00 (file version 2.0.0.0, 1,812,480 bytes) and rev1 (2.0.0.1), rev2 (2.0.0.2), rev3 (2.0.0.3)**; 2.01 (+ rev1); 2.02 (+ rev1, rev2); 2.03; 2.04; 2.05 (+ rev1, rev2).
- 2.1 test builds `graal21test2` and `graal21test3` (file version 2.1.0.0, about 1.75 MB). They still send the 2.05 token.
- NewMain 24 and 26 (file version 1.4.1.1, token `GNW08011`, with GraalShop, RemoteControl and LevelGen); NewWorld betas 1 and 2 (`Graal-NW.exe`, file versions 2.0.5.0 and 2.1.0.0).
- **Not present:** any 2.1.x release, 2.2x or 2.3x. There is no 2.17, 2.22 or 2.31 exe in this set, and nothing between 2.05 and 2.17 except the two 2.1 test builds. The 2.1 to 2.16 and 2.3 rows therefore stay inferred (`version-profiles/version-coverage-2026-10-24.md`).
- What the set does close: the missing real 2.00 exe, plus three revisions of it.

Running them (Wine)

- Wine 10 starts the 2.00, 2.05 rev2 and graal21test3 exes (the process stays alive and a window is created).

- Each one stops at "Cannot open file ... commands.rtf", because the Drive folders for 2.0x contain only the exe (and sometimes an `.idc`). The exes need their data files beside them: `commands.rtf`, the image folders and the language files.
- With the data files of another build (checked: 2.00 exe plus NewMain 26 data files) the 2.00 start screen draws normally.
- Run settings: new prefix, loopback-only lab network, private display. Nothing contacted a public network.
- Not run: any login to the lab, any 2.0x network flow, and the 1.x builds.

Candidates for the next decompile pass

2.01 to 2.04 and the two 2.1 test builds have no decompile yet. A static pass over them would settle the per-build handler differences in `version-profiles/plo-handled-ids-by-build.tsv`, which were read from dispatch tables only.

`client-internals/version-id-map.md`

Version ID map

Date: 2026-10-14

Protocol token → version

GNW tokens are `GNW` + build date as DDMMY (GNW03081 = 3 Aug 2001). The 2.1 test builds (Sep 2001) still send GNW03081. Per-build hashes, dates and the rest are in `build-catalogue.md`.

Token	Version	Source
`GNW07080`	1.39 (CLVER_1_39)	version-token pack / gs2lib
`GNW13110`	1.41r1	allowedversions.txt
`GNW03081`	2.05 rev 2 / "2002ver9" leak (CLVER_2_052, sorts below CLVER_2_1)	gs2lib 2026; GServer versions.txt; built char-by-char in the 2.05 login
`GNW24011`	2.0 (and rev1–rev3)	version-token pack; confirmed in all four 2.00 exes (2026-10-24), see `version-tokens.md`
`GSERV007`	1.21	build catalogue
`GSERV013`	1.28 beta2	build catalogue
`GSERV014`	1.3 test	build catalogue
`GSERV016`	1.31	build catalogue
`GSERV018`	1.32	build catalogue
`GNW20020`	1.33	build catalogue
`GNW22030`	1.34	build catalogue
`GNW13040`	1.35	build catalogue
`GNW10050`	1.36	build catalogue
`GNW21050`	1.37	build catalogue
`GNW22060`	1.38	build catalogue
`GNW08011`	NewMain client (1.42 notes)	build catalogue
`GNW23021`	2.01, 2.01 rev1	build catalogue
`GNW15041`	2.02, 2.02 rev1	build catalogue
`GNW20041`	2.02 rev2	build catalogue
`GNW06051`	2.03	build catalogue
`GNW24061`	2.04	build catalogue
`G2N15041`	NewWorld betas 1 and 2	build catalogue

Token	Version	Source
`GNW22122`	2.1.7.0–2.1.7.1 (CLVER_2_17)	allowedversions.txt; gs2lib; GServer versions.txt
`GNW03014`	2.2.2.0 (CLVER_2_22)	allowedversions.txt; gs2lib
`GNW28015`	2.3.1.0 (CLVER_2_31)	allowedversions.txt; gs2lib
`GNW30123`	listserver token of the 2.22 and 2.31 clients (sent in listserver id 7 as key + `GNW30123newmain`), not a game build	2.22/2.31 binaries; seen on listserver port 14922; 2.22 confirmed in the lab (`protocol/listserver.md`)
`G3D03014`	Linux 3.04 rev 1 (= 3.041 in the server table below)	3.x client binary (same date stamp as GNW03014)
`G3D28015`	Mac OS X 3.153	3.x client binary (same date stamp as GNW28015)
`G3D30123`	3.x listserver login	3.x client binary
`G3D17026`	Windows 4.04r2	4.x engine DLL (date stamp 17 Feb 2006)
`G3D27076`	Mac OS X 4.20r6	4.x client binary (date stamp 27 Jul 2006)
`G3D20126`	Windows 4.20r8	4.x engine DLL (date stamp 20 Dec 2006)

Filename → build

Binary	Build
`Graal_2171.exe`	2.171
`Graal_222.exe`	2.22
`Graal_231.exe`	2.31

Construction (v2.05)

Token is eight `_LStrFromChar` immediates in `TAdventureClient_Create`, not a string literal. Random prefix `(rand & 0x7F) + 0x20` precedes it.

Update 2026-10-21: the login line is GCHAR client type 0, then GCHAR key (`rand & 0x7F`), then the token. The "random prefix" is that key (see `protocol/login-and-walk.md`).

Confirmed in Graal 139.exe.c

The login/token builder builds the token as char immediates: G N W 0 7 0 8 0 → GNW07080. Connects to port 14900 (PTR_s_14900).

1st Graal.exe.c

No GNW token construction found. Uses .zelda level files (level33.zelda, house19.zelda). Has socket code but no version-token immediates. Predates the GNW scheme.

Dhrake-improved 1.39 export

Same binary, better symbols. ~390 recovered names. Token still built as char immediates (line ~128320). Preferred corpus in shdecompiling/graal-139-dhrake.

Full gs2lib client token table (2026-10)

Token	Build	Token	Build
`GSERV007`	1.21	`GNW20052`	2.151

Token	Build	Token	Build
GSERV009	1.25	GNW12102	2.16
GSERV011	1.27	GNW22122	2.17
GSERV013	1.28	GNW21033	2.18
GSERV014	1.30	GNW15053	2.19 (bz2 + new encryption)
GSERV016	1.31	GNW28063	2.2
GNW07080	1.39	GNW01113	2.21 (compression byte)
GNW12080	1.391	GNW03014	2.22
GNW13110	1.41r1	GNW14015	2.3
GNW03081	2.05 rev 2	GNW28015	2.31
GNW31101	2.1 (2.1.0.0–2.1.0.3, also 2002ver10/13)	G3D16053	3.0
GNW01012	2.12	G3D27063	3.01
GNW23012	2.13	G3D03014	3.041
GNW30042	2.14	G3D28095…	4.x+
GNW19052	2.15		

The older GServer-v2 submodule lists 1.33–1.38 as `GNW20020`, `GNW22030`, `GNW13040`, `GNW10050`, `GNW21050`, and `GNW22060`. Its `versions.txt` writes them without the `G` (`NW20020`, …).

`client-internals/version-tokens.md`

Version tokens

Date: 2026-10-14

Internal version strings recovered from the public version list. No decompile material.

Token	Build notes
GNW07080	1.39
GNW03081	2.05 rev 2 (= 2002ver9 leak build); gen-3 cipher, pre-2.1 code paths
GNW31101	2.1.0.0–2.1.0.3 (version string not updated)
GNW30042	2.1.4.0 (first with server-list tabs)
GNW15053	2.1.9.0 (bz2 + new encryption, new server-list gfx)
GNW01113	2.2.1.1 (plaintext and zlib; extra compression byte)
GNW14015	2.3.0.0
GNW28015	2.3.1.0
G3D03014	Linux 3.04 (Torque rewrite; 2.22 protocol, see `protocol/v3x-clients.md`)
G3D28015	Mac OS X 3.153 (2.31 protocol)
G3D30123	3.x listserver token (the 3.x form of GNW30123)
G3D17026	Windows 4.04r2 (`GraalEngine.dll`, March 2006)
G3D27076	Mac OS X 4.20r6 (Delphi launcher + C++ engine era; see `protocol/v4x-clients.md`)
G3D20126	Windows 4.20r8 (`GraalEngine.dll`)
G3D20126	Windows 4.20r8 (engine DLL; logs into the `Login` game server)
G3D22067	Windows 5.00r7 (engine DLL, Jul 2007; see `protocol/v5x-clients.md`)
G3D14097	Windows 5.12 (lab capture; see `protocol/v5v6-windows-unpacked.md`)
G3D03048	Windows 6.00 beta1 (lab capture)
G3D15010	Windows 6.00 beta2, 5.24r3, 5.24r4 (lab capture)
G3D26090	Windows 5.30r3, 5.31r4, 6.00r2, 6.00r4 (lab capture)
G3D0311C	Windows 6.03r7 (lab capture; Linux 6.03r7 sends G3D0511C)

Token	Build notes
G3D0108A	Linux 6.01r1 (see `protocol/v6x-clients.md`)
G3D0511C	Linux 6.03r7

Filename builds in the client tree: Graal_2171 = 2.171, Graal_222 = 2.22, Graal_231 = 2.31.

From 2.3 the editor and the online client are separate executables. PE FileVersion is authoritative for the editor (2.3.1.1).

The early-client transition (14801 → 14900 → length+zlib) is the protocol-side counterpart of the same era.

Confirmed from the binaries (2026-10-24)

The 2.0x family tokens below were read from the character immediates of each client's login builder, so they are confirmed per build rather than taken from a token list. Every value matches the earlier tables here and in `version-id-map.md`. Machine-readable copy: `version-profiles/version-tokens-by-build.tsv`.

Token	Build	Source
GNW24011	2.00 and rev1, rev2, rev3 (all four exes)	login builder
GNW23021	2.01, 2.01 rev1	login builder
GNW15041	2.02, 2.02 rev1	login builder
GNW20041	2.02 rev2	login builder
GNW06051	2.03	login builder
GNW24061	2.04	login builder
GNW03081	2.05, rev1, rev2; also the 2.1 test builds (`graal21test2`, `graal21test3`), which still send the 2.05 token	login builder
GNW08011	NewMain 24 and 26	login builder
GNW13110	1.411	login builder
GNW07080	1.39	login builder
GNW22060	1.38	login builder
GNW22122	2.17.1	lab capture and exe
GNW03014	2.22 game login	exe
GNW28015	2.31 game login	lab capture
GNW30123 + `newmain`	2.22 and 2.31 listserver hello (not a game build)	exe and lab

A lab GServer-v2 only needs the token in its allowed-versions list to let a client in. For 2.00 (`GNW24011`) that is a config entry, not a code patch (`client-internals/decomp-findings-2026-10-24-b/04-v2.00-client.md`).

`client-internals/version/README.md`

Seihai — Version ID Recovery

How the project recovers version identifiers from Graal classic binaries and their decompiles. Packaged for the Seihai project.

Contents

- `VERSION.md` — consolidated notes on the methods used.

- `docs/` — supporting documentation:
  - `GraalEditor-README.md`, `GraalEditor-architecture.md`
  - `graaleditor.txt`, `graal.txt` (LinuxMan 2 man pages)

Quick reference

Binary	Version signal	Value
GraalEditor.exe	PE version resource (FileVersion)	2.3.1.1
Graal_2171.exe	Filename	2.171
Graal_222.exe	Filename	2.22
Graal_231.exe	Filename	2.31
gif2mng.exe	Banner string	2004-06-03

The PE version resource is the reliable source for FileVersion / CompanyName / ProductName. It is present even in the Thinstall stub of GraalEditor. Client builds are identified by filename encoding.

`client-internals/version/VERSION.md`

Finding the Version ID from the Decompile

The version ID is recovered from the **PE version resource**, not from executable logic.

Primary method: PE version resource

In a Ghidra dump the structure appears as `VS_VERSION_INFO`:

```
struct VS_VERSION_INFO {
    word StructLength;
    word ValueLength;
    word StructType;
    wchar16 Info[16];
    byte Padding[2];
    dword Signature;
    word StructVersion[2];
    word FileVersion[4];      // four-part version
    word ProductVersion[4];
    dword FileFlagsMask[2];
    dword FileFlags;
    dword FileOS;
    dword FileType;
    dword FileSubtype;
    dword FileTimestamp;
};
```

The actual string values live in the `.rsrc` section as UTF-16. For GraalEditor the notes record them directly:

- CompanyName: `Linux Cyberjoueur`
- FileVersion: `2.3.1.1`
- InternalName: `Graal editor`
- ProductName: `GraalEditor`

That resource is present in both the packed Thinstall stub (`graal/GraalEditor.exe`) and the unpacked Delphi image. The packed-stub dump is only the loader (~3 k lines), so the version string is still readable from the stub's resources without unpacking.

Secondary signals

Source	What it gives	Example
Filename	Client build	`Graal_2171.exe` → 2.171, `Graal_222.exe` → 2.22, `Graal_231.exe` → 2.31
PE optional header	Linker version	Editor: Major/MinorLinkerVersion = 2.25
Banner / About strings	Date or product string	gif2mng: `"Graal Image conversion and palette fixing tool 2004-06-03"`; `TAboutDlg` caption "Version"
String search	Hard-coded tokens	`graalversion`, `Version`, product names

Practical steps on a new dump

1. Open the binary (or the Ghidra project) and locate the version resource under `.rsrc` / `VS_VERSION_INFO`.
2. Read the UTF-16 StringFileInfo block for `FileVersion`, `ProductVersion`, `CompanyName`, `InternalName`.
3. Cross-check the PE optional header for linker version if needed.
4. For the game clients, the filename itself is the version token used in the notes.

What the project recorded

- GraalEditor 2.3.1.1 (Linux Cyberjoueur). The file in `graal/` is a Thinstall package; the Delphi image was recovered by running the stub decompressor and the Thinstall `0xD7` block codec. Unpacked image is at `GraalEditor/unpacked/GraalEditor.exe` (Borland linker 2.25).
- Clients: `Graal_2171.exe` (2.171), `Graal_222.exe` (2.22), `Graal_231.exe` (2.31).
- gif2mng banner date: 2004-06-03.

All of the above is measured from the binaries and their resources. The version resource is the authoritative FileVersion; filenames supply the client build numbers used in the notes.

`client-internals/version/docs/GraalEditor-README.md`

GraalEditor.exe — reverse-engineering notes

GraalEditor 2.3.1.1 is the Graal Online classic level editor (Linux Cyberjoueur). The file in `graal/GraalEditor.exe` is not a Delphi image. It is a Thinstall (pre-VMware ThinApp) package. That is why IDR rejected it, and why the Ghidra C dump in `Export/GraalEditor.exe.c` is only 3,136 lines of loader.

The Delphi editor was recovered by running the stub decompressor and then the Thinstall `0xD7` block codec. The unpacked image is `unpacked/GraalEditor.exe` (1,729,536 bytes, Borland linker 2.25). Names below come from Delphi RTTI and binary DFM resources (`TPF0`), not from original source.

Why the dump looks empty

`Export/GraalEditor.exe.c` decompiles the outer stub only (entry into the stub startup, then the package-open helper). The optional-header `LoaderFlags` field is reused as a file offset (`0xFC94E`) of a 0x38-byte package header. Magic is `0x82D0B188`. The first block is the Thinstall runtime (`X:\thinstall\stub\ts_stub.cc`), not the editor. Ghidra never saw the Delphi code, so IDR had nothing with a Delphi VMT to reconstruct.

What the program is

A single-window level editor (`TAdventure`, caption `Graal`) with modal tools for links, signs, NPCs, baddies, and chests.

- Levels open and save as `*.graal` and `*.nw`. Filter string: `Graal levels (*.graal, *.nw)|*.graal; *.nw`.
- A level is a text file with `BOARD`, `LINK`, `SIGN`, `CHEST`, `NPC`, `BADDY` / `BADDYEND`, and `LEVELNAMES` / `LEVELNAMESEND` sections. `BOARD 0` is the tile layer.
- World view is a gmap (`*.gmap`, `loadgmaps.txt`, `bigmap.txt`, `minimap.txt`, `graalmap.png`).
- Preview is DelphiX DirectDraw. Both DirectDraw 5 and DirectDraw 7 init failures are stringed. Sound is BASS (`bass.dll` is imported by the stub's import table and by the editor).
- Spritesheets are PNG, GIF, MNG. Failures are `Gif loading failed!`, `MNG loading failed!`, `Image loading failed!`.
- The same GANI classes as GraalShop are linked in: `TGraalAni`, `TGraalAniSprite`, `TGraalAniStep`, `TGraalActor`, `TGraalActorMod`, plus `TGraalAniPart`, `TGraalAniSound`, `TGraalSpriteAttachment`.

Read this first

Document	Contents
packer.md	What the Ghidra dump actually is
architecture.md	Process, modules, class map
data-model.md	Level, link, sign, NPC, baddy, chest
level-format.md	On-disk sections the editor reads and writes
flows.md	Load, save, paint, mode switch
pseudocode.md	Reconstructed procedures
ui.md	Forms recovered from DFM
diagrams/	Packer, architecture, classes, load/save

Package contents

Thinstall virtual path `%InstallPath%`:

File	Role
`GraalEditor.exe`	Delphi editor, version resource 2.3.1.1
`hvdi.dll`	Bundled helper, 133,632 bytes. No Graal strings; not the editor core
`arrays.dat`	11,500-byte sorted dword table (count word `0x03A6`)
`arrays2.dat`	Server list: `<list-host-3x>:14922` and two fallbacks
`arrays2.sig`, `graaleditor.sig`	0x80-byte signatures. Stub string: `Not ready to verify signature, must have valid public key`
`thinstall.log`	Virtual log path

`Setup_Graal` is the package group name. `TS_EXECUTE_EXTERNAL` is set in the package directory.

client-internals/version/docs/GraalEditor-architecture.md

Architecture

GraalEditor is a Delphi VCL application. The live document is a `TAdventureLevel` owned by the main form `TAdventure`. Preview goes through DelphiX (`TDXForm`, DirectDraw 5 and 7). Sound goes through BASS. Picture codecs are the same family GraalShop uses (GIF, MNG, PNG/DIB).

```
GraalEditor.exe  (unpacked image, base 0x400000)
├── Delphi RTL          AnsiString, exceptions, file IO
├── VCL                 forms, listviews, richedit, menus, dialogs
├── Graphics            TGIF, TMNG, TJPEGImage, TBitmap, TGraalImage
├── DelphiX / DirectDraw  TDXForm, DirectDraw 5 and DirectDraw 7
├── BASS                music, samples, microphone device names
├── Domain              TAdventureLevel, TEditorLevelCache, TGraalMap,
│                       TEditorLink, TEditorSign, TEditorNPC, TEditorChest,
│                       TEditorObject, TLevelObject, TLevelHeights
├── Animation           TGraalAni and the GraalShop actor/step/sprite set
└── UI                  TAdventure + modal frames (see ui.md)
```

See diagrams/architecture.svg and diagrams/classes.svg.

Layers

Runtime

Borland RTL banner: `SOFTWARE\Borland\Delphi\RTL`, `Portions Copyright (c) 1983,97 Borland`. Linker version 2.25. Sections are the classic Delphi layout: `CODE`, `DATA`, `BSS`, `.idata`, `.tls`, `.reloc`, `.rsrc`. Unit name recovered from RTTI: `UAdventureEditor`.

VCL shell

One main form, twenty modal or tool forms stored as `TPF0` DFMs in `.rsrc`. The main form is `TAdventure` / `Adventure`, caption `Graal`, `poScreenCenter`, `KeyPreview` set. It owns the tile canvas, a mode radio group, and the menu that opens the other frames.

There is no plugin ABI. Registry use that shows up is the Delphi locale key and the shell-folders key the Thinstall stub virtualizes, not a Graal account store. Account files the editor knows by name: `buddyaccounts.txt`, `ignoredaccounts.txt`, `mutedaccounts.txt`, `graalconfig.txt`, `headsconfig.txt`, `FILENAMECACHE.txt`.

Picture, map, sound

- GIF: `TGifAnimation`, `TGifAnimationStep`, failure `Gif loading failed!`
- MNG: `TMNGAnimation`, `TMNGAnimationStep`, failure `MNG loading failed!`
- PNG and ordinary bitmaps via `TGraalImage` / `TBitmap`. Image dialog: `CImage files (*.png, *.mng, *.gif)`
- DirectDraw init can fail for 5 or 7: `Error when loading and initializing DirectDraw5` / `DirectDraw7`. More Options has `Don't use Direct3D (lights)` and `Draw in compatibility mode (very slow)`, plus a vertical-blank wait checkbox.
- BASS loads music and samples. More Options has Audio and Microphone pages.

- World map assets: `graalmap.png`, `map.png`, `map2.png`, `minimap.txt`, `bigmap.txt`, `loadgmaps.txt`. Status strings: `Loaded gmap`, `Loaded minimap`, `Loaded minimap from`.

Domain

`TAdventureLevel` is the open level. `TEditorLevelCache` keeps recently loaded levels (the link walker and the gmap need neighbours). `TGraalMap` is the stitched world. `TLevelHeights` is a height overlay distinct from the tile board.

Placed entities are separate types, not one tagged union: `TEditorLink`, `TEditorSign`, `TEditorNPC`, `TEditorDefNPC`, `TEditorChest`, `TEditorObject`, plus baddy state edited by `TEditBaddy`. `TEditorUndoInfo` is the undo record. `TGaniObject` / `TImageAnimation` attach a GANI to a placed object.

Animation subset

The GANI object model is the GraalShop one, compiled into the editor so tile-object scripts and NPC animations can be previewed without launching GraalShop:

`TGraalAni`, `TGraalAniSprite`, `TGraalSpriteAttachment`, `TGraalAniPart`, `TGraalAniSound`, `TGraalAniStep`, `TGraalActor`, `TGraalActorMod`.

`TTilesArrayFrame` is the tile-index / script editor (Courier New edit plus a rich edit). Its sample script is the offline tutorial: `addtiledef picso.png,offlinetutorial,1` and `setplayerprop` / `setcharprop` lines.

Control flow, one screen

```
WinMain-equivalent (Delphi RTL)
        │
        ▼
TAdventure  caption "Graal"
        │
        ├── F5 Load Level ── open *.graal / *.nw ── TAdventureLevel
        ├── F4 Edit Level ── tile / object / baddy / npc / chest radios
        ├── links ── TLinkList ── TLinkOptions
        ├── signs ── TEditSigns
        ├── NPC ── TEditNPC (image + script rich edit)
        ├── baddy ── TEditBaddy (type + three verses)
        ├── chest ── TChestEdit (item + sign index)
        ├── F6 Debugger, F7 Player List, F8 Server List
        └── save ── write BOARD / LINK / SIGN / CHEST / NPC / BADDY
```

Binary facts

- Unpacked image base `0x400000`, `SizeOfImage 0x1B1000`, entry RVA `0x155D4C`.
- Version resource (also present, UTF-16, in the packed stub): CompanyName `Linux Cyberjoueur`, FileVersion `2.3.1.1`, InternalName `Graal editor`, ProductName `GraalEditor`.
- On-disk packed file is the Thinstall stub. Do not point IDR or a Delphi decompiler at `graal/GraalEditor.exe`. Point them at `unpacked/GraalEditor.exe`.

`client-internals/version-profiles/README.md`

Version profiles, 1.41 to 2.17

Date: 2026-10-23

`141-217.toml` describes each client release from 1.41 rev1 to 2.17 well enough for a reimplementation to *be* that version. That covers what it sends and understands on the wire, and also what the player sees and can do. `resolve.py` flattens the inheritance (`base`, `_add`, `_remove`) and prints JSON or one markdown block per version, with a source tag on every value:

```
python3 resolve.py 141-217.toml          # JSON
python3 resolve.py 141-217.toml --md     # markdown
```

Field names follow the Seihai profile schema: `token`, `listserver`, `client_type`, `generation`, `login_key_byte`, `compression`, `cipher`, `pli_sent`, `plo_known`, `props_max`, `prop_diffs`, `login_props`, `level_formats`, `file`, `tampercheck`, `udp`, and `features`.

Sources

Tag	Meaning
bin	Read from that build: switch tables, send sites, DFM forms, strings, shipped images. 24 executables were scanned (1.41 rev1, NewMain 24/26, 2.0 r0–r3, 2.01 r0–r1, 2.02 r0–r2, 2.03, 2.04, 2.05 r0–r2, 2.1 test 2/3, NewWorld betas 1/2, 2.1.7.1)
doc	The release notes shipped with the clients (`newfeatures141/142/2001/2002.txt`)
srv	GServer-v2 source: what a server does differently per version
forms	The DFM resources: start window, options, controls, server list, player list
lab	Seen on the wire from the real client, run under Wine against a lab list and game server (2026-10-30: 1.41 rev1, NewMain 26, 2.00 rev0, 2.02 rev1, 2.02 rev2, 2.05 rev2, 2.1 test 3, NewWorld betas 1/2, 2.1.7.1). A lab tag covers only the build that was run; `resolve.py` turns it into `inf` in the profiles that inherit it
inf	Not checked in that build. Either the value is carried from the nearest scanned build, or it is placed at the release that introduced the feature

No binaries of 2.1 release, 2.12, 2.13, 2.14, 2.15 or 2.16 are in the archive. Their entries are release notes plus server source, and every value they inherit from a scanned build is tagged `inf`.

How the packet sets were read

- **Server → client:** the receive dispatcher is a `case` on `id` with a jump table. The ids with a non-default arm are the ids the build handles. The results match `cross-version-timeline.md` except for one correction (below).
- **Client → server:** every packet goes out through one virtual send method that takes the id as a byte argument. Scanning each build for constant ids passed to that call reproduces the known 2.05 set exactly (`packet-ids.md`), so the same scan was used for the other builds. Id 0 is sent through a computed path and is present in all of them.
- **Props:** each prop parser and the prop writer are `case` tables too. The writer's table is the list of props the client can send. In 2.05 it matches the "sent by the client" list in `player-props.md`.

Wire boundaries

Change	First build	Tag
gen 3 (key byte + LCG insertion), server list window; the listserver speaks first and the client sends nothing	1.41	bin, lab
Client asks the listserver for a list by name (id 0 `newmain`), no account	1.42	lab
`.gani`, light effects, PLO 42, 44, 45, 150, 151, 153; **UPDATEFILE (34) sent**	1.42	bin
`.nw` loader dropped (until 2.05; GraalX keeps it), listserver host DES-hidden, PLO 154, 155	2.00	bin
ADJACENTLEVEL (35), HITOBJECTS (36) sent; PLO 46, 156	2.01	bin
PLO 47; player props 37–41 (`#P1`–`#P5`) parsed and sent; NPC props to 40	**2.02**	bin
LANGUAGE (37) sent after login	2.02 rev2	bin, lab
TRIGGERACTION (38) sent; PLO 48, 165	2.03	bin
PLO 166, PLO 26 dropped; prop 42	2.04	bin
`.nw` + `.gmap`; PLO 168	2.05	bin
Account and password sent to the listserver (id 1 after the id 0 list name); NewWorld beta 1, on the 2.05 code, sends only the list name	2.1 test 3	lab
Bow slot → GANI string, `.png` defaults, PLO_FILE modtime, FILEUPTODATE, `sprite % 4` for NPCs, prop caps lifted	2.1 (GNW31101)	srv
Large files (LARGEFILESTART/END)	2.14	srv, doc
TAMPERCHECK (39), SHOOT (40), SERVERWARP (41), MUTEPLAYER (43), PROCESSLIST (44/45); signed `arrays2.dat` listserver	2.17 (checked); some may be earlier	bin
Login props add PLANGUAGE (52); 39 and a PLI 44 window-title list are sent at login	2.17 (checked)	lab

Corrections to earlier notes:

- **PLO 47 and props 37–41 are already in 2.02**, not first in 2.02 rev2 as `cross-version-timeline.md` says. That note skipped the 2.02 base build.
- **1.41 has no UPDATEFILE sender.** It never revalidates a cached file. 1.42 adds it, which matches the 2.00 note "files are now automatically downloaded again from the server when they have been updated".
- The 2.02 rev1 notes say the client language is sent to the server. No LANGUAGE send site exists in rev1, and the lab run confirms it sends none. The first one is in rev2.
- The 2.1 test builds still send 2.05's token (GNW03081), so a server cannot tell them apart from 2.05 and uses the pre-2.1 paths for them.

What the player sees

These are the `features` keys, which matter most for making the client look and behave like the chosen version.

- **1.41.** The start window lists game modes (One Player, One Player Online, Two/Three/Four Players) beside character heads. Local 2–4 player split screen and save games (F2/F3) still exist. There are no ganis, so animation runs on sprite numbers only. The HUD gets its AP bar under the ASD box this version, plus a "sparring" label and the R key for ratings. Right-clicking a player opens their profile. The options window

has a Save games page. The HUD sheet has two chat-bubble colours and no gralat bag, clock or day/night icons.

- **1.42 / 2.00.** The game becomes online-only. The start window shows a character with Training (built-in tutorial levels) and Start buttons. The 2.00 start window is titled "Graal 2001". Keys are rebindable in the Controls window. Ganis and light effects arrive, with a "Don't use DirectX7" option. The HUD sheet adds admin, guild and mass chat-bubble colours, a gralat bag, a clock and day/night icons. The player list gets a "Staff" section (from 2.00).
- **2.01–2.04.** Levels preload on the map, and a map-reading animation plays. 2.02 adds hats (`#P1`–`#P5`), images in signs and HTML-window navigation. 2.04 adds `replacegani`, tile replacement, attaching the player to an object and a weapon-select toggle.
- **2.05.** The server name appears in the title bar. `.nw` and `.gmap` levels load. The shipped HUD sheet adds the "PK off" sword icon (identical to the 2.1 test sheet), although the notes date that feature to 2.1.
- **2.1.** A new start window and server list show account info. The options window gets tabs (General, Video, Audio, Controls) and a Direct3D toggle. Ghost mode, auto-mapping, map and minimap set by the gserver, the Alt+0 log, smilies in PMs and gani scripts all arrive, and default images switch from GIF to PNG/MNG.
- **2.12–2.16.** The editor gains multi-level editing (2.12). Gani attributes grow to `#P9` (2.13) and then `#P30` (2.16). The server list splits into two tabs (2.14), which also bring voice chat, OGG/MP3/MOD music, volume sliders, player status and `serverwarp`. 2.16 adds transparency modes, `enablefeatures` and `showani`/`showpoly`/`showtext`.
- **2.17.** The start window gains payment buttons and a "Private Server List". The options window gains Audio and Microphone tabs and an Automapping toggle. The player list gets status (Online/Away/DND) and Mute. The client handles the RPG window, Kingdoms and New World modes, and level heights.

Lab runs (2026-10-30)

Each client was run under Wine with its own data folder against a stub list and game server. The stub answers WANTFILE with FILESENDFAILED. Per build:

Build	Listserver	Login line	First messages after login	Login props
1.41 rev1	14923; server sends the list first, client sends nothing	type 0, GNW13110, account, password	none	0, 11, 13, 18, 31, 33
NewMain 26 (1.42)	14921; id 0 `newmain`	type 0, GNW08011	file requests	same
2.00 rev0	same as 1.42	type 0, GNW24011	file requests	same
2.02 rev1	same as 1.42	type 0, GNW15041	file requests (no LANGUAGE)	same
2.02 rev2	same as 1.42	type 0, GNW20041	LANGUAGE, file requests	same
2.05 rev2	same as 1.42	type 0, GNW03081	LANGUAGE, file requests	same
2.1 test 3	14922; id 0 `newmain`, then id 1 account + password	type 0, GNW03081	LANGUAGE, file requests	same
2.1.7.1	same as 2.1 test 3	type 0, GNW22122	LANGUAGE, TAMPERCHECK, file requests, PLI 44	adds 52; head as a file name

The ports are the ones built into each exe (the lab routed every list host to itself). Every login line is the same layout as in `defaults`. Head (11) is an index below 100 in every build up to 2.1 test 3.

Notes for a server or a reimplementation:

- 1.41 stalled at "Loading account from server (1/5)" when the login burst carried 2.05-era ids it does not handle. With only ids from its `plo_known` it entered the game.

- 2.00 waits for `idle.gani` before it sends its login props. If the file is missing locally and the server cannot send it, the client sits in the level with no body.
- 2.02 rev1 and rev2 load `zlib.dll` (cdecl). The archive copies ship without it, so a lab needs one built from zlib.
- Under Wine, the 2.x clients sometimes stopped sending after the login burst when the in-game HTML window opened at the same moment. A rerun always went through, so this is put down to the lab, not the clients.

Open

- 2.1–2.16 packet and prop sets are inferred from neighbours. A binary of any of them would pin them down.
- The original 1.41 listserver host and port: the only surviving 1.41 copy is patched for <graalin-site>.
- Login props were not run for 2.01, 2.03 or 2.04. They are inferred from their neighbours, which all send 0, 11, 13, 18, 31, 33.
- What 2.17's periodic PLI 24 (one byte, every 5 s) carries.
- Which 2.05-era server id stalls 1.41 at "Loading account from server (1/5)".
- The 2.17 other-player prop parser and its GS1 command count were not read.

Default list port

`listserver` carries the endpoint a reimplementation should dial first: `host`, `port`, `kind` (`legacy` or `supported`) and `port_status` (`confirmed` or `assumed`). It is not the value the binary carries: that is kept as history in `built_in_host` and `built_in_port`.

- Host is always `<list-host-reborn>`.
- 14921 is dead (the public service resets it) and is nobody's default.
- The default follows the login type the profile sends (the public listserver answers by login type, not by version or token): legacy = classic login (1.41 to 2.17, including 2.05) defaults to 14923; supported = gen-5 login (2.22, 2.3, 2.31) defaults to 14922.
- Every profile is `confirmed` except 2.05: Seihai's own login for it was run on its port in the matrix of 2026-10-25 (a real client was not captured). 2.05 is `assumed`: its classic login was not run.

The per-profile table and the evidence behind each status are in `list-ports.md`. `python3 list_ports.py --check` fails when a profile breaks the rule. Rule text: Seihai `documentation/port-rule.md`; public results: `../../protocol/public-listserver-2026-10-25.md` and `../../protocol/listserver-login-type-matrix-2026-10-25.md`.

1998 to 1.32 (added 2026-10-30)

Two more files cover the clients before 1.33. They use the flat layout of `v133-v139.toml`: each `[[version]]` holds every value, with no `base`.

- `v1998-v11.toml`: 20 builds from 1st Graal (Sept 1998) to 1.1 (Feb 1999). Generation 1, the text relay in `../../protocol/relay-1998.md`. There is no token, login or account, so `token` and `pli_sent` are replaced by `relay_game` (the game name sent with `N`) and `d_types_sent` (the relay `D` type letters). It also gives the dialog host, the level and save magics, and the config keys.
- `v121-v132.toml`: 1.21, 1.28 beta 2, 1.3 test, 1.31 and 1.32. Generation 2, GSERV tokens, see `../../protocol/gserv-1999.md`. Same keys as the 1.33 file, plus `plprops_sent` (the prop writer's table) and `bundled_servers`.

How the gen-2 sets were read: the same switch-table and send-site method as above, with each table found in 1.32 and then matched in the older builds by function shape. `pli_sent` is a lower bound here. 1.28 beta 2 and 1.3 test pass more ids in a register, which is why they list fewer ids than 1.21.

	1.21	1.28 b2	1.3 test	1.31	1.32
Server ids	0–34 except 30, 100	0–38, 100	0–39, 100	0–40, 100	0–41, 100
Player props parsed	0–24	0–31	0–32	0–32	0–33
Player props sent	0–24	0–24, 26, 31	0–24, 26, 31, 32	same as 1.3	0–24, 26, 31–33
NPC props	0–17	0–32	0–34	0–34	0–34
UDP ids	none	0, 10–22	0, 10–22	0, 10–23	0, 10–23
GS1 commands	71	84	85	85	85

1.32's sets match the 1.33 profile, except that 1.33 adds three GS1 commands (`setbody`, `setfocus`, `resetfocus`).

Correction to `v133-v139.toml`: `save_format` said `Graalsav9`. These builds write `Graalsv11`. The three save magics are referenced the same way as in 1.32, where the writer was traced (see `../../formats/graalsav.md`).

No build in this range was run. Every value comes from the binaries and the shipped notes. The features tagged `doc` come only from the release notes.

`client-internals/version-profiles/list-ports.md`

Default listserver endpoint per profile

Date: 2026-10-25 (updated after the login-type matrix)

This is what a reimplementation should dial first for each profile. It is **not** what the binary carries. The binary's own host and port are kept as history in `built_in_host` and `built_in_port`.

Rule (written in Seihai `documentation/port-rule.md`; evidence in `../../protocol/listserver-login-type-matrix-2026-10-25.md`):

- Host is always `<list-host-reborn>`.
- **14921 is dead.** The public service resets it. No profile defaults to it.
- The public listserver answers by **login type**, not by version or token. A classic login (one plain zlib frame, id 0 then id 1) gets a list on **14923** and nothing on 14922. A gen-5 login (id 7 with `GNW30123`) gets a list on 14922 and on 14923.
- **Legacy** profiles send the classic login and default to **14923**: 1.41 to 2.17, including 2.05 (Seihai's 2.05 profile sends the classic login).
- **Supported** profiles send the gen-5 login and default to **14922**: 2.22, 2.3, 2.31.
- `kind` is `legacy` or `supported`. `port_status` is `confirmed` (a public check of Seihai's login for that profile got a list on that port; a real client of that version was not captured) or `assumed` (not run with that login).

The table is produced by `python3 list_ports.py` (this directory) from the three profile files. `python3 list_ports.py --check` fails if a profile breaks the rule, so a later edit to a TOML cannot quietly bring 14921 back.

version	host	port	kind	port status	built-in host	built-in port
1.33	-	-	no listserver	-	-	-

version	host	port	kind	port status	built-in host	built-in port
1.34	-	-	no listserver	-	-	-
1.35	-	-	no listserver	-	-	-
1.36	-	-	no listserver	-	-	-
1.37	-	-	no listserver	-	-	-
1.38	-	-	no listserver	-	-	-
1.39	-	-	no listserver	-	-	-
1.41	<list-host-reborn>	14923	legacy	confirmed	<list-host-reborn>	14923
1.42	<list-host-reborn>	14923	legacy	confirmed	<list-host-classic>	14921
2.00	<list-host-reborn>	14923	legacy	confirmed	<list-host-classic>	14921
2.01	<list-host-reborn>	14923	legacy	confirmed	<list-host-classic>	14921
2.02	<list-host-reborn>	14923	legacy	confirmed	<list-host-classic>	14921
2.02 rev2	<list-host-reborn>	14923	legacy	confirmed	<list-host-classic>	14921
2.03	<list-host-reborn>	14923	legacy	confirmed	<list-host-classic>	14921
2.04	<list-host-reborn>	14923	legacy	confirmed	<list-host-classic>	14921
2.05	<list-host-reborn>	14923	legacy	assumed	<list-host-classic>	14921
2.1	<list-host-reborn>	14923	legacy	confirmed	<list-host-classic>	14922
2.12	<list-host-reborn>	14923	legacy	confirmed	<list-host-classic>	14922
2.13	<list-host-reborn>	14923	legacy	confirmed	<list-host-classic>	14922
2.14	<list-host-reborn>	14923	legacy	confirmed	<list-host-classic>	14922
2.15	<list-host-reborn>	14923	legacy	confirmed	<list-host-classic>	14922
2.16	<list-host-reborn>	14923	legacy	confirmed	<list-host-classic>	14922
2.17	<list-host-reborn>	14923	legacy	confirmed	-	-
2.22	<list-host-reborn>	14922	supported	confirmed	-	-
2.3	<list-host-reborn>	14922	supported	confirmed	-	-
2.31	<list-host-reborn>	14922	supported	confirmed	-	-
3.04	<list-host-reborn>	14922	supported	assumed	<list-host-3x>	14922
3.153	<list-host-reborn>	14922	supported	assumed	<list-host-3x>	14922

What the status means per profile (matrix of 2026-10-25, one list-only login per profile and port):

status	profiles	evidence
confirmed	1.41, 1.42, 2.00 to 2.04 (with 2.02 rev 2), 2.1, 2.12 to 2.16, 2.17	Seihai's classic login got a legacy list on 14923 (2 or 4 rows) and no reply for about 5.5 s on 14922, in every case. Real clients of these builds were not captured; 2.1 to 2.16 have no binary at all
confirmed	2.22, 2.3, 2.31	the gen-5 login got the modern list on 14922 (8 rows, welcome, site and upgrade lines) and on 14923 (12 rows, 2.31: 17)
assumed	2.05	Seihai's 2.05 profile sends the classic login, which was not run for 2.05. The matrix binary sent the gen-5 login for 2.05; both ports answered it. 14923 answered the classic login of every other classic profile, so it is the port assumed here
assumed	3.04, 3.153	the 3.x clients send the gen-5 login with `G3D30123newmain` (`../../version-profiles/3.04-3.153.toml`). A G3D token was not tried against the public listserver; 14922 is assumed from the 2.22/2.31 result

Open: a classic 2.05 login on 14923; how a real 2.17 client logs in to the listserver.

`version-profiles/matrix/README.md`

Per-version profile matrix, 1998 to 2014

Date: 2026-10-30

This is the one-page view of how each Graal client **thinks** (protocol, scripting), **acts** (timing, movement, rules, sound) and **looks** (window, start screen, HUD, fonts, menus), from the Java Zelda Online of 1998 to the 6.11 Mac client of 2014, plus the Graal3D side project. Each cell is short and points at the note that has the detail. It is meant as the checklist for a client that wants to *be* a chosen version, such as Seihai's version selector.

New in this pass (2026-10-30):

- A static survey of all 57 Delphi client executables (1st Graal to 2.31): forms, main window, frame timer, fonts, sound names, config keys and shipped sheets. Per-build values are in `builds-static-survey.tsv`, and what each build adds or drops is in `builds-deltas.md`.
- Live runs under Wine of 1st Graal, 1.0, 1.1, 1.32, 1.41 and 2.00, with screenshots and a walking-speed measurement (`live-runs-2026-10-30.md`).
- A comparison with Seihai's per-version profiles (`seihai-gaps.md`).

Tags, as elsewhere in this repo: **[bin]** read from the client binary · **[file]** shipped files · **[live]** seen in a running client · **[cap]** packet capture · **[srv]** server source · **[doc]** release notes · **[inf]** inferred.

Eras

The builds fall into eleven eras. Inside an era the clients share their engine and wire format, so a profile can be one era row plus the per-build deltas listed in the linked notes.

Era	Builds	Dates	Engine	Detail
Z	Zelda Online 1 and 2 (Java applets), GServ Standalone	May–Jul 1998	Java	`../../client-internals/zelda-online-1998.md`, `../../client-internals/precursors-and-fan-builds.md`
A	1st Graal to 1.1.9 Gold (20 builds)	Sep 1998 – Feb 1999	Delphi 2-level compiler (2nd Graal: Delphi 4), DelphiX	`../../client-internals/build-catalogue.md`, `../../protocol/relay-1998.md`
B	1.21, 1.28 beta 2, 1.3 test, 1.31, 1.32	May 1999 – Jan 2000	Delphi 4 from 1.28b2, DelphiX	`../../protocol/gserv-1999.md`
3D0	Graal3D	Feb 2000	Delphi, DelphiX Direct3D retained mode	`../../graal3d/README.md`
C	1.33 to 1.39	Feb – Aug 2000	Delphi 4, DelphiX	`../../client-internals/versions-133-139.md`
D	1.41 rev1	Nov 2000	as C	`../../client-internals/version-profiles/README.md`

Era	Builds	Dates	Engine	Detail
E	NewMain 1.42, 2.00 to 2.05 rev2, 2.1 tests, NewWorld betas	Jan – Oct 2001	as C, plus DirectDraw 7	`../../client-internals/cross-version-timeline.md`, `../../client-internals/newworld-betas.md`
F	2.1 to 2.17 (only 2.17.1 survives as a binary)	Oct 2001 – 2002	as E, BASS audio from 2.17 at the latest	`../../client-internals/version-profiles/141-217.toml`
G	2.22, 2.3 (no binary), 2.31	2004 – 2005	as F	`../2.22-2.31.md`
H	3.04 Linux, 3.153 Mac	2004 – 2005	C++ on the Torque Game Engine	`../../protocol/v3x-clients.md`
I	4.04r2, 4.20r6/r8	2006 – 2007	Delphi launcher + C++ `GraalEngine.dll` (+ `Graal3DEngine.dll`)	`../../protocol/v4x-clients.md`
J	5.00r7, 5.12; then 5.24 to 5.31	2007 – 2011	5.00/5.12 as I; from 5.24 one MSVC exe (`quattro_lib`)	`../../protocol/v5x-clients.md`, `../../protocol/v5v6-windows-unpacked.md`
K	6.00 betas to 6.03r7, 6.11 Mac	2010 – 2014	one native C++ program per platform, no Torque layer	`../../protocol/v6x-clients.md`

Thinks: wire and scripting

Era	Login token	Cipher gen	Server discovery	Ids handled / sent	Props parsed	Scripting
Z	none	text lines	`host:14801`	text lobby lines	–	none
A	none	1 (no zlib)	connect dialog, the loopback address on 14801, Cyberjoueur relay 14802	relay lobby `N/P/C/D` lines, `D` payload types per build	relay `D` types	GS1-like, `this.` vars from 1.1 beta1 **[bin][doc]**
B	`GSERV007`–`018` (a protocol revision, not a date)	2 (zlib, no cipher)	connect dialog, `<list-host-classic>:14804`, `:14900` from 1.31	GSERV id ranges (`gserv-1999.md`)	–	GS1, NPC weapons from 1.21
3D0	`GR3DV002`	own (2D codec, no key byte)	the loopback address on 14901, UDP 14898 peer to peer	own renumbered ids	3D floats, 24-bit colours	none
C	`GNW` + date (`GNW20020` … `GNW07080`)	2	connect dialog `<list-host-classic>:14900`, guest/guest	1.33: S→C 0–41, 100; C→S 0–31; grows to 45 / 35 ids by 1.39	own 0–33/34, others 21–23 ids	GS1, 88 → 111 commands
D	`GNW13110`	**3** (key byte + LCG insertion)	first server-list window	+1.41 deltas	props to 36	GS1, 126 commands
E	`GNW08011` … `GNW03081`; NW betas `G2N15041`; 2.1 tests keep `GNW03081`	3	listserver, host DES-hidden from 2.00, `:14921` (2.1 tests `:14922`)	2.05: 59 S→C ids	to 42 (2.05), 50, 51	GS1 134 → 161 commands, ganis from 1.42
F	`GNW31101` (2.1) … `GNW22122` (2.17)	3	2.17: signed `arrays2.dat`, id 0 then id 1	2.17: 79 S→C, 61 C→S; TAMPERCHECK (39) from 2.17	to 74 (gani attrs `#P10`–`#P30` from 2.16)	GS1 + gani scripts (2.1), `this.` client strings (2.19)

Era	Login token	Cipher gen	Server discovery	Ids handled / sent	Props parsed	Scripting
G	`GNW03014`, `GNW14015`, `GNW28015`; list `GNW30123`	**5** (compression byte, partial XOR, bz2)	listserver id 7 `GNW30123newmain`	2.22: 80, 2.31: 81	to 74 (2.22), 80 with X2/ Y2/Z2 (2.31)	GS1; classic client-side NPC mode removed in 2.3
H	`G3D03014`, `G3D28015`; list `G3D30123`	5	RSA-signed `license.graal` → `<list-host-3x>:14922`	79 (3.04), 88 (3.153)	as G	GS1 native + TorqueScript `.dso` for menus; NPC bytecode ids (3.153)
I	`G3D17026`, `G3D27076`, `G3D20126`	5	`license.graal` → `<login-host-4x>:14900`; server named `Login`, no separate list protocol	98–101	–	GS2 bytecode, server-pushed ganis (195), update packages `GRPKG001`
J	`G3D22067`, `G3D14097`, `G3D15010`, `G3D26090`	5	as I; from 5.24 the hosts are compiled in, no license file	95–100	–	GS2, `GRESCR01` script cache; anti-cheat process scan (182/183)
K	`G3D0108A`/`G3D0511C` (Linux), `G3D0311C` (Win 6.03); 6.11 set by script	5; 6.11 can switch to a **script-defined** header with RC4/AES/TLS	compiled-in `loginserver*:14900`; 6.11 HTTPS connector to `con.quattroplay.com`	91–97	–	GS2; in 6.11 even the packet id tables come from the server's script

Login-time extras that a faithful client must send: `PLI_LANGUAGE` (37) from 2.02 rev2 **[bin]**; `PLI_TAMPERCHECK` (39) once (2.17), on gmap crossings and level links (2.22), or every 3.2 s (2.31) **[cap]**; machine-id MD5s after the password from 4.x on **[bin]**.

Acts: timing, movement and rules

What	Value by era	Source
Game tick	**50 ms** in every Delphi build. A through D drive it from a DelphiX `TDXTimer` (interval 50 in the main form); from NewMain 1.42 the form has no DX timer and the frame runs from the idle handler behind a 50 ms gate. Graal3D: 50 ms `TDXTimer`.	**[bin]** survey; `../../client-internals/rendering-observations.md`
Walking	**0.5 tile per tick, 10 tiles per second**, from the first beta on. Measured live in 1st Graal, 1.0, 1.1, 1.32, 1.41; the 2.05 trace and the 2.05/2.17/2.22/2.31 walking routine agree.	**[live]** `live-runs-2026-10-30.md`; `../../client-internals/decomp-findings-2026-10-25-m/01-walking-rule.md`
Wall test	**1st Graal:** six probes, fail closed, push counter wraps above 37 back to 18, hop at 18 (`rules-1st-graal.md`). **1.39 and 2.05–2.31:** six probe points per facing step. 2.05 also has corner nudge 0.1 and flush snap. In 1.1 a position with a 0.025 offset (x = 26.925) showed up after bumping furniture, which fits a similar rule.	**[bin]** 1st Graal, 1.39 (`rules-139.md`), 2.x; **[live]** 1.1

What	Value by era	Source
Death and respawn	**1st Graal:** hearts to 0 set the dead life-state; a key once the shared counter is above 27 restarts `house1.graal` at (24.5, 30) with 3 hearts. The up-count was not found. **1.39:** 30-tick timer, input and STATUS dead bit after counter > 27, online default (30.5, 30), hearts restored to max (no AP bands). **2.05:** same timer/bit; hearts by AP; 2.22 lab-confirmed.	`rules-1st-graal.md`; `rules-139.md`; `../../client-internals/feature-specs/04-death-respawn.md`
Combat	**1st Graal:** hurt 40 ticks, five knockback steps, sword frames 6–10, reach up to 4 tiles ahead and sideways under 2, sword power a fixed half heart. **1.39 and 2.05:** hurt 40 ticks, knockback 1 tile/tick for 5 ticks (counter 39…35), sword reach 0–4 tiles / sideways < 2. 1.39 hurt pose ~10 ticks then blink; 2.05 uses the `hurt` gani.	`rules-1st-graal.md`; `rules-139.md`; `../../client-internals/decomp-findings-2026-10-24-d/`
Ledges, push/pull, carry-hide	2.05: jump on the 18th blocked step; `canbepushed`/`canbepulled` from 1.1 beta1, separated in 1.34; hiding under a carried object (mode 14).	`../../client-internals/ledges-push-pull.md`, `../../client-internals/carry-hide-and-glove.md`
Bush, pot and stone respawn	Client-side before 2.00 **[inf]**; server-side from 2.00 ("bush rupees and pot hearts go server-side"). Exact timers are being measured in a separate thread.	**[doc][inf]**
Local multiplayer	Two players from 1st Graal; Two/Three/Four Players to 1.41; removed in NewMain 1.42 / 2.00.	**[bin][live]** start windows
Sleep on start	1.1 starts the offline game with the player asleep in bed (a key wakes the player).	**[live]**
Sound engine	none in 1st–4th Graal; DelphiX DirectSound from 4th alpha (12 named `.wavs`), 29 by 8th Graal, 24–27 named through 2.31; MCI MIDI, plus Winamp control from 1.41 until BASS replaced it; BASS from 2.17 (no Winamp, adds volumes, radio, voice chat); fmod in 3.x–5.x, SDL_mixer on 6.x Linux. Max 8 effects at once (2.05).	**[bin]** survey; `../../client-internals/sound-and-music.md`
Save games	`Graalsav1` … `Graalsv11`; F2/F3 to 1.41; gone in 2.00.	`../../formats/graalsav.md`

Looks: window, screens, HUD and text

What	Value by era	Source
Main window	Caption "Graal". Outer size 648×521 (1st Graal), 648×516 (2nd Graal to 1.1.9), 648×508–509 (1.21–1.39), 648×514 (1.41), 648×507 (NewMain, 2.00), 648×514 (2.01–2.1 tests), 644×507 (2.17–2.31). Form font MS Sans Serif 14 px throughout. 2.05+ add the server name to the title. 2.12–2.16, 2.3 and 3.x have no outer size on record (see below).	**[bin]**

What	Value by era	Source
Start window	1st Graal: "Graal - The Adventure", radio buttons One Player / Two Player / One Player Online with host the loopback address and port `14801`. 1.0: "Select the game mode" with the grail picture, character pick per player, "Online At Cyberjoueur" box with port 14802. 1.1 to 1.32: the rider-on-horse picture, mode and character drop-downs; 1.32 adds Language and Nick Name and "Managed by: Stéphane Portha". 1.41: Language, Game mode (One Player … Four Players), Character, Nick Name, "Lead programmer: Stefan Knorr". 2.00: dark-green "Graal 2001" window with the GraalOnline logo, Training / Options / Start. 2.1: new start window and server list. 2.22: Start Online / Training / Options; 2.31: Start Online / Editor / Options. 3.x on: Torque GUI or server-scripted screens.	**[live]** screenshots; **[bin]** forms
HUD sheet (`state`)	448×96 GIF (1st, 2nd Graal) → 448×137 (3rd–5th) → 298×96 (6th) → 427×109 (7th, 8th) → 427×125/132 (9th, 1.0–1.1.9) → 224×49 (1.21) → 224×81 (1.28b2–1.37, PNG from 1.33) → 224×97 (1.38–1.39) → **224×145 (1.41 to 2.31)**. The NewWorld betas ship the older 224×97 sheet.	**[file]** survey; `../../client-internals/status-display.md`
HUD layout	Spiral (yin-yang) hearts at the top right in every build seen. 1st Graal shows three rupee digits (`000`), 1.0 drops leading zeros (`0`). The A/S/D item buttons appear by 1.32; 1.41 adds the AP bar under them.	**[live]**
Tileset (`pics1`)	1024×512 (1st–4th Graal) → 1280×512 (5th–1.21) → 1856×512 (1.28b2–1.41) → 2048×512 (NewMain 1.42 on).	**[file]**
Name and chat text font	1.3 test adds `font=/fontsize=`. 1.38–1.41 executables name Helvetica; the 1.41 install has `font=Helvetica`, `fontsize=24`, the 1.32 install `font=Dialog`, `fontsize=28`. **NewMain 1.42 to 2.31 register the bundled `tempsitc.ttf` (Tempus Sans ITC) at start-up**, and the installs set `font=Tempus Sans ITC`, `fontsize=24`. Live: 1.32 and 1.41 draw NPC names large, bold and white with a dark-blue outline; 2.00 draws them smaller in the rounded face. The chat bubble itself is in a separate thread.	**[bin][file][live]**
Other fonts	Courier New from 1.34 (the F6 debugger), Arial Black in 2.17–2.31 forms.	**[bin]**
Options	Options form from 1st Graal; More Options from 5th Graal; Controls from 15th Graal; player list from 1.28b2; profile from 1.39; server list from 1.41; log window, RPG window and tile-array dialog by 2.17 (no 2.1x binary to place them earlier); 2.31 drops the editor forms (29 → 18).	**[bin]** `builds-deltas.md`
Keys	Arrows or IJKL, A/S/D, M map, Q items, P pause, Tab chat; fixed to 1.37, rebindable from 1.38; F1–F7 and Alt+1–9 per `versions-133-139.md`.	**[bin][doc]**

What	Value by era	Source
Heads and characters	Pick list with named characters (Nick, Mandi …) from `heads.gif` in 3rd Graal to 1.1.9; `headsconfig.txt` (Fung Si Yan, Nick, Mandi, Bad Max …) with one image per head from 1.21.	**[file][live]**

Outer window where no form was measured

Date checked: 2026-11-01. Sizes below are only ones already written down. The Decompile folder (client exports and Accessories) has no form-size note and no export of these builds.

Builds	Outer size	Why
2.12, 2.13, 2.14, 2.15, 2.16	not measured	No row in `builds-static-survey.tsv`. `client-internals/drive-clients-inventory.md` records no release between the 2.1 test builds and 2.17.
2.3	not measured	`version-profiles/2.22-2.31.md` says no 2.3 executable was uploaded.
3.04, 3.153	no VCL outer size	The Delphi survey stops at 2.31. These clients are Torque / SDL, not a Delphi form.

The nearest measured Delphi forms sit on either side of 2.12–2.16. 2.1 test 2 and 2.1 test 3 are **648×514**. 2.17.1, 2.22 and 2.31 are **644×507**. **[bin]** `builds-static-survey.tsv`. The change falls between 2.1 test 3 and 2.17.1, so neither size is assigned to 2.12–2.16. 2.22 and 2.31 share 644×507, and that neighbour size is still not assigned to 2.3.

3.04 Linux, in `lab-304-linux-2026-10-30.md`, opens at **800×600** on a 1024×768 display. **[cap]** That sentence records the opening size. It does not split outer size from client area, and it is not a Delphi form size. No window size is written down for 3.153. Torque's minimum screen mode of 640×480 (`torque/engine-and-formats.md`) is a device floor, not an outer window.

What is still unknown

Ranked by how much it changes what a player would notice.

1. **The 1998 to 1.41 rules. 1.39 is done** (`rules-139.md`). **1st Graal is done** (`rules-1st-graal.md`: same five rules, with a fixed half-heart sword, a restart at `house1.graal`, and the death-counter increment left untagged). Still open: 1.32 (Decompile folder re-listed 2026-11-01; still no 1.32 export, so no death, hurt or wall number) and the rest of eras A–B. Walking speed already matches across the measured builds.
2. **2.1 to 2.16 and 2.3.** No binaries. Everything in era F before 2.17 is from release notes and server source. Outer window sizes for 2.12–2.16, 2.3 and 3.x are in the section above: unmeasured, except the 3.04 Linux opening size already in the lab note.
3. **5.x and 6.x look.** The screens come from the server (`Login` server GUI scripts, update packages), so the look is whatever that server sent. `../../protocol/login-session-known.md` records what the existing notes already say that server sends, written with no public capture. The look itself is still unobserved.
4. **Graal3D movement and camera.** Follow distance, look height and field of view are in `../../graal3d/camera.md`. The far plane stays open (library initial back plane 100 **[doc]**; a later DelphiX `SetBack(5000)` was not used; the February 2000 exe was not re-read for `SetBack`).
5. **Default name font before 1.38.** The exes from 1st Graal to 1.37 contain no font family name for the game text; 1.32's install says `Dialog`, which Windows does not have, so a substitute was used.

Files in this folder

File	What
`README.md`	This matrix.
`rules-139.md`	1.39 death, hurt, knockback, sword reach and wall test (matrix unknown #1 for era C).
`rules-1st-graal.md`	1st Graal death, hurt, knockback, sword reach and wall test. 1.32 is recorded as still open.
`builds-static-survey.tsv`	One row per executable: MD5, size, form count, main window, form font, frame timer, sound/config/image name counts, sheet sizes.
`builds-deltas.md`	Per-build additions and removals of config keys, sounds, images, forms and fonts.
`live-runs-2026-10-30.md`	The Wine runs: how, what was measured, screenshots.
`seihai-gaps.md`	Seihai's profiles against this matrix.

The survey script, the form parser, the Wine helpers and the screenshots are in the project folder under `graal-decompile/version-matrix/`, not in this repo.

`version-profiles/matrix/seihai-gaps.md`

Seihai's version profiles against the matrix

Date: 2026-10-30. Compared: Seihai `main` at 72ed21d (`client/src/protocol/profile.rs`, `chain.rs`, `documentation/version-profiles.md`) with `README.md` in this folder.

What Seihai already covers

Seihai has 26 profile rows, 1.33 to 2.31, built as a version chain (1.33 written out, each later build a delta). A row sets the token, client type, cipher generation, listserver endpoint and login shape, the server→client and client→server id sets, prop limits, file and level features, the death and combat rules where traced, and UI switches (server list style, chat recall, HUD sheet layout, minimap, editor, ganis, emoticons, key config, log window, ghost mode, server name in the title, GS1 count). The 50 ms game clock, the half-tile walking step and the 8-voice sound cap are global and match every build measured (`live-runs-2026-10-30.md`), so they need no per-version switch.

Missing, by size

1. Eras with no profile at all

Era	Builds	What a profile would need
Z, A: 1998 (Java and 1st Graal to 1.1.9)	21	The gen-1 relay (lobby `N/P/C/D` lines, game names `Zelda2`, `Graal` … `Graal8`, `D` payload types per build), `.zelda` and GR-V1.00–1.03 levels, `Graalsav1`–`8`, GIF sheets of their own sizes, the 1998 start windows, two-player local mode. `protocol/relay-1998.md`, `formats/graalsav.md`. `EncryptionGen::Gen1` exists in Seihai but no row uses it.
B: 1.21 to 1.32	5	GSERV007–018 login on gen 2 (`protocol/gserv-1999.md`), port 14804 then 14900, the 1.2x id ranges, `Graalsv11` saves, 224×49 and 224×81 HUD sheets.

Era	Builds	What a profile would need
3D0: Graal3D (2000)	1	Its own client and server protocol (`graal3d/protocol.md`); a separate program, so it fits the planned Seihai suite rather than the 2D selector.
H: 3.x	2	Gen 5 as 2.22/2.31, but login tokens `G3D03014`/`G3D28015`, no tamper check, the signed `license.graal` list host, the category prefix that picks a server-list tab, 79/88 handled ids. Close to 2.22/2.31 on the wire.
I, J, K: 4.x to 6.x	about 20	Login id 5 with the machine-id list, the `Login` game server instead of a listserver, update packages, GS2 bytecode and server-built GUI; 6.11's script-defined framing. A different client rather than a profile row.

2. Look switches the rows do not have

Gap	Matrix value	Seihai today
Name and chat text font	Bundled Tempus Sans ITC at 24 from NewMain 1.42 to 2.31; Helvetica-named before (1.38–1.41); `font=`/`fontsize=` from 1.3 test	One 5×7 bitmap font for every version (`render/font.rs`). The chat-bubble thread covers the bubble itself.
Start window	Distinct per era (see the matrix and the screenshots)	One launcher for every version
Main window size	648×508 (1.3x) / 648×514 (1.41, 2.01–2.1) / 648×507 (2.00) / 644×507 (2.17+). 2.12–2.16 and 2.3 were not measured. 3.x has no VCL outer size; 3.04 Linux opens at 800×600 in the lab note, with no outer-versus-client split	Not per version
HUD sheet layout	224×81 (1.33–1.37), 224×97 (1.38–1.39), **224×145 (1.41 to 2.31)**	`HudArt::BlankButtons` covers 1.38 to 2.31 and its comment calls the later builds inferred. The HUD already draws the AP and MP bars only when the sheet has their frames, so the drawing is right; the comment is not. (Fixed in the accompanying Seihai change.)
Options and windows	Per-build form lists (`builds-deltas.md`)	Not modelled
Local multiplayer and save games	Up to 1.41	Not modelled

3. Rules that are only partly per version

- **Death and combat.** 1.41 carries the 2.05 rules from the template, 1.42 to 2.04 carry none, 2.05 and later carry them (`chain.rs` already lists this as unknown). **1.39 is now traced** (`rules-139.md`): same 30-tick death timer, STATUS bit after counter > 27, 40-tick hurt lock-out, five knockback steps, sword box 0–4 / <2, six-probe walk. Differences for a 1.39 row: restore hearts to **max** on respawn (no AP bands; the AP HUD starts in 1.41), and the hurt pose is a short built-in animation (~10 ticks) rather than the later `hurt` gani. **1st Graal is traced** (`rules-1st-graal.md`) and must not copy that row: sword power is a fixed half heart on frames 6–10, hurt is 40 ticks with five knockback steps, and the death key restarts `house1.graal` at (24.5, 30) with hearts and the heart cap both set to 3. Seihai still has no era A profile to hang this on. 1.32 is still open (no decompile).
- **Bush, pot and stone respawn.** Client-side before 2.00, server-side after (release notes). The exact timers are being measured in another thread.
- **Tamper check.** Seihai has the per-build constants and resend rules opt-in; nothing new here.

4. Assets per version

Seihai reads each version's own asset folder (`assets.<label>=`). The matrix lists which sheets and sizes each build shipped (`builds-static-survey.tsv`). The archives for 2.00 to 2.05 hold only the exe, so those versions need the art from a neighbouring install; 1.41 rev1 and NewMain 1.42 (`graalnm26`) are the closest full installs.

Suggested order

1. Name font per version (Tempus Sans ITC for 1.42+), since every screen shows it.
2. A 1.21–1.32 profile (GSERV login on gen 2 is a small step from Seihai's gen-2 1.33 path).
3. The 1998 relay as its own mode.
4. 3.x as two rows next to 2.22/2.31.
5. 4.x and later as a separate client in the planned suite.

`version-profiles/version-coverage-2026-10-24.md`

Version coverage: is Graal_analysis enough for Seihai's multi-version selector?

Date: 2026-10-24 (Asia/Tokyo). Read-only review. No pushes, no PRs, no public connections.

> **Update (same day, after loopback lab runs with real 2.17, 2.22 and 2.31 clients):** the tampercheck gap below is closed. The constants for all three builds are in `tampercheck-values.tsv`, and the payload is 10 raw bytes, not two 32-bit words. The 2.17 listserver flow, the 2.22 gmap edge crossing, the mute payload and the client's short compressed gen-5 frames are now documented in `protocol/listserver.md`, `client-internals/feature-specs/06-gmap-overworld.md`, `.../14-friends-ignore.md` and `protocol/codec-and-framing.md`. 2.17 and 2.31 were accepted by the lab gserver with the merged Seihai. The table and gap list below are left as the original snapshot. Reviewed: `squirelworks/Graal_analysis` main at 8da6800 (PRs #1 to #26 merged), `squirelworks/seihai` main at 3383300, plus the lab handoff notes and lab run directory (including the 2026-10-24 public 2.22 report) and `seihai_notes/`. Only derived facts here: no decompile text, addresses, or internal function/data names.

Short answer

Yes for the protocol skeleton, no for full per-version behaviour. Graal_analysis (PRs #24, #25, #26: `client-internals/version-profiles/*.toml` + `resolve.py`, `version-profiles/2.22-2.31.*`, `protocol/client-versions.md`, `version-tokens.md`, `version-id-map.md`) now has, per version, a token, client type, generation, listserver login, server-to-client id set, client-to-server id set, prop ranges, file-transfer flavour and level formats, each tagged bin / cap / srv / doc / inf. That is enough to *select* 1.33 through 2.31 and log in with the right token, cipher and framing. Seihai already has the abstraction (`ClientVersion`, 23 rows in `profile.rs`, `EncryptionGen` gates login and framing, `plo`/`pli` id sets gate dispatch and sends).

What is missing is mostly **payload layouts of the newer packets, the tampercheck content, and live confirmation** of everything except 2.05 and 2.22.

What Graal_analysis covers

Topic	Where	Coverage
Tokens, build to token	`version-tokens.md`, `version-id-map.md`	Complete 1.21 to 2.31 plus 3.x. 2.10-2.16 from gs2lib only

Topic	Where	Coverage
Game-server login (type, key byte, token, account, password)	`client-versions.md`, `login-and-walk.md`	2.05, 2.17, 2.22, 2.31 from binaries. Older rows inherit
Generations	`codec-and-framing.md`	gen 2, 3 (2.05 verified on 704 messages), 5 (2.22 verified). Gen 4 (2.19-2.21) server-only
Gen-5 client send rules	`client-versions.md`	Size thresholds, XOR limits, queue cap
Listserver	`listserver.md`	2.05 (id 0), 2.17 (id 0 then id 1, host from signed `arrays2.dat`), 2.22/2.31 (id 7 + `GNW30123` + gen 5)
PLO set per version	profile TOMLs	Read from binaries for 1.33-2.05 (24 exes), 2.17, 2.22, 2.31. Inferred for 2.1-2.16, 2.3
PLI set per version	profile TOMLs	Binary for 1.41-2.05, 2.17, 2.22, 2.31; 2.22 has 3 unmatched ids (36, 42, 45). Inferred 2.1-2.16
Packet id tables with payloads	`packet-ids.md`, `packet-layouts.md`	Names for all ids. Payload layouts good for 0-48 and 2.05 senders. Thin for 49 and 168-189
Props (player, NPC, baddy)	`player-props.md`, TOML `props_max`	Good to 2.31 (X2/Y2/Z2 encoding documented). Other-player parser for 2.17 not read
File transfer	`file-transfer.md`	Complete for pre-2.1, 2.1-2.13, 2.14+. Large-file chunking from server source only
Levels	`level-binary*.md`, `glevnw01.txt`, `gmap.md`, `v3x-clients.md`	GR-V1.00-1.04, GLEVNW01, GRMAP001, `.gmap`. Per-version loader lists in TOML
Version differences (player-visible)	`version-profiles/README.md`, `2.22-2.31.md`, `feature-timeline.md`	Rich. HUD, windows, keys, GS1 command counts
UDP	`udp-p2p.md`	2.05 captured. Other versions inherit

Seihai side: how versions are selected and gated

- `protocol/version.rs`: `ClientVersion` enum (1.33-1.39, 1.41, 1.42, 2.00, 2.01, 2.02, 2.05, 2.1, 2.12-2.16, 2.17, 2.22, 2.3, 2.31). `EncryptionGen` = Gen1/2/3/5.
- `protocol/profile.rs`: one `VersionProfile` row per version, built from templates `CLASSIC`, `BASE` (2.05), `NEW_2_1`, `GEN5`. Fields: client type, generation, listserver port and token, `plo`/`pli` IdSets, language and tampercheck flags, prop sets, file flags, level formats, UI flags, `Basis::{Binary,Inferred}`.
- `net/mod.rs`: `login_versioned` and `login_listserver` branch on `EncryptionGen`; sends are refused when the id is not in the row's `pli`; dispatch drops ids not in `plo`.
- The rows were written from the **earlier** notes (cross-version-timeline, client-versions), before the merged TOML profiles. So Seihai and the TOML disagree in places (below).

Per-version table

(TSV copy: `version-coverage-2026-10-24.tsv`.)

Version	Handshake known	Encryption / framing known	Packet table known	Gaps	Verdict
2.00 (`GNW24011`)	Yes, bin: type 0, key byte, token, gen 3. List host `<list-host-classic>: 14921` DES-hidden (dead; redirect needed)	Yes, gen 3 as 2.05	PLO 51+2 ids and PLI from binary (4 revs, no differences). Prop parse 0-36	Token absent from gs2lib: stock GServer-v2 treats it as unknown, so lab needs an allow-list patch. Login props unverified. Never run live. Seihai row reuses the 1.41 PLI set (includes 35/36 which first appear in 2.01)	**Partly**: enough to implement, untested
2.05 rev 2 (`GNW03081`)	Yes, bin + lab	Yes, gen 3, 704 messages	Yes: PLO 59, PLI 0-24/26-30/32-38/80/81	Port and host per listserver (built-in 14921 vs Reborn 14922/14923); lab patch maps it onto 2.1 behaviour; tampercheck absent	**Enough**
2.01-2.04 (`GNW23021`, `GNW15041`, `GNW20041`, `GNW06051`, `GNW24061`)	Token yes (catalogue); login as 2.05	Gen 3	Binary for PLO/PLI deltas per build	Seihai has no 2.02 rev2, 2.03, 2.04 rows (language at 2.02 rev2, trigger at 2.03 not modelled); live untested	**Partly**
2.1-2.16 (`GNW31101` ... `GNW12102`)	Tokens yes (gs2lib). Login inferred as gen 3	Gen 3 inferred (2.17 is gen 3)	**Inferred** from neighbours and release notes	No binary of any 2.1x release in the archive. 2.1 test builds still send 2.05's token	**Needs binary or capture**; low priority, can ride on 2.05 or 2.17
2.17 (`GNW22122`, 2.171)	Yes, bin: type 0, gen 3, list via `arrays2.dat` (id 0 then id 1, host and port from file; port default 14922)	Yes, gen 3	PLO 79, PLI 61 ids from binary; PLO 169-185 present	TAMPERCHECK (39) content untraced (two 32-bit words); other-player prop parser and GS1 count not read; not run live; Seihai `pli` for 2.17 omits 40, 41, 43, 44, 45 and its list port default is 14921	**Partly** (login/framing enough; behaviour thin)
2.22 (`GNW03014`; list `GNW30123`)	Yes, bin + lab + 4 public gservers + public listserver	Yes, bin + lab + public: 27/27 gserver frames and both list frames decode	PLO 80, PLI from binary (36, 42, 45 unmatched)	Payloads for ids 25, 39, 41, 44, 47, 156, 168, 174, 179, 180, 182 (sent by every public server) are names only; tampercheck is a 10-byte per-build constant (lab value known, not sent by Seihai; public servers did not kick within 20 s); client bz2 send path never seen; Seihai `pli` for 2.22 = 2.17 set (allows 36/38/81, refuses 40/41/43/44)	**Enough** to log in and play; **partly** for full parity

Version	Handshake known	Encryption / framing known	Packet table known	Gaps	Verdict
2.3 (`GNW14015`)	Token yes; everything else inferred from 2.31	Inferred (gen 5)	Inferred	No binary exists	**Partly** (treat as 2.31)
2.31 (`GNW28015`)	Yes, bin: same code as 2.22, `GNW30123` for list	Yes, bin (same thresholds, XOR limits, queue cap)	PLO 81, PLI = 2.22 minus 21 plus 46	Never run (lab or public); stock GServer-v2 has the token commented out; `MOVE2` (189), `LARGEFILESIZE` (84), `ENTERLEVEL` (46) layouts not documented; tampercheck value unknown; X2/Y2/Z2 encoding documented	**Partly**: one lab capture away from enough
1.33-1.39 / 1.41 / 1.42	Yes (bin, 1.39 lab-verified as gen 2)	Gen 2 (1.33-1.39), gen 3 (1.41+)	Binary per build	1.41 list host unknown; no listserver before 1.41	**Enough** for a selectable row

Concrete gaps, ranked

1. **Payload layouts of ids 49 and 168-189 (and 25, 39, 41, 44, 47, 156)** are names or one-liners in `packet-ids.md`. Needed for bigmap/minimap, ghost mode, serverwarp, RPG window, status list, process list, level heights, BOARDMODIFY2, MOVE2, LARGEFILESIZE. *Fillable now:* private public-server captures already on the lab box (never committed; all frames decoded) show most of these ids; the 2.17/2.22/2.31 exes + `.idc` name lists cover the handler arms for a cleaned note. 2.00 export is not needed.
2. **PLI_TAMPERCHECK (39) content** for 2.17, 2.22, 2.31. 2.22 has a lab constant in the TOML; 2.17 and 2.31 are unknown. *Fillable from* a static read of the builder in the `.exe`/`.idc` (2.171, 2.222, 2.31), or a lab capture of each real client under Wine. Needed before a stricter server enforces it.
3. **Seihai rows lag the merged TOMLs.** 2.22/2.31 `pli` should follow the binary list (2.22: no 36/38/81, adds 40/41/43/44; 2.31: minus 21 plus 46); 2.17 `pli` should add 34-36, 40, 41, 43-45; 2.00 should not send 35/36; no rows for 2.02 rev2, 2.03, 2.04; 2.17 listserver port. *Fillable now*: no decompile needed. Wire `resolve.py` output into `profile.rs` (or a generated table).
4. **Doc conflict on the gen-5 XOR group rule.** `codec-and-framing.md` and `client-versions.md` still say `floor(n/4)` capped at 4 for types 4 and 6. Lab (610 frames) and public (27 frames) show one step per group present, a partial last group included. The doc needs correcting, with a note of which side (client send vs server) was read. Also the client-originated bz2 frame (>= 0x1000) is unseen. *Fillable now* from existing captures and the 2.22 `.idc`.
5. **2.1-2.16 and 2.3 have no binary.** Rows are inferred. *Needs new binaries*; nothing on the box covers them. Practical fix: mark them as aliases of 2.05 (2.1 line) or 2.31 (2.3) in the UI.
6. **Live acceptance is unknown for everything except 2.22.** Public Reborn servers may reject 2.17, 2.31, 2.05 or 2.00 tokens, and may treat a missing tampercheck differently. Stock GServer-v2 allows 2.22 only by default. *Needs captures*: lab runs for 2.00 (allow-list patch), 2.17, 2.31 with the real Wine clients; public runs need explicit approval.
7. **Listserver per version.** 2.17 id 0 + id 1 against the Reborn listserver on 14922 is untested; 2.05/2.00 depend on a host redirect. Port should be per-profile and overridable. Needs one capture, not a decompile.
8. **Smaller:** 2.22 PLI ids 36, 42, 45 call sites unmatched; 2.17 other-player prop parser unread; 2.22 `#R` GS1 code meaning; 2.1-2.16 GS1 counts.

Which gaps the existing exports can fill

Source on the box	Fills
`donors/Graal_2171.exe` + `.idc` + `.names.tsv`	2.17 tampercheck builder, other-player prop parser, PLI send sites, `arrays2.dat` handling
`donors/Graal_222.exe` + `.idc`	Payload layouts for ids above 48, PLI sites 36/42/45, tampercheck, client bz2 path
`donors/Graal_231.exe` + `.idc`	`MOVE2`, `LARGEFILESIZE`, `ENTERLEVEL`, 2.31 tampercheck
`donors/Graal_v2_00.c` (2.00 export, 193k lines)	2.00 PLI set cross-check, login props, 2.00 level and prop details
Existing public pcaps (box only, never commit)	Payload shapes of 25, 39, 41, 44, 47, 156, 168, 174, 179, 180, 182 and the bigmap/minimap messages
New captures	Anything about 2.31 and 2.17 live behaviour, 2.00 against a patched lab server, other-player parser behaviour, public acceptance of tokens
New binaries	2.1 to 2.16, 2.3

Note: the `.idc` files are name tables. Reading the handler bodies still needs a disassembler pass (Ghidra) over the matching `.exe`.

Recommendation

1. Treat 2.05, 2.22 as the verified baseline; 2.17 and 2.31 as selectable now with "partly" labels.
2. Regenerate the Seihai rows from the TOML (gap 3), then fix the XOR note (gap 4). Both are doc/data work with no new decompile.
3. Run one Ghidra pass each over 2.22 and 2.31 for gaps 1 and 2, write a cleaned note per version, then lab-capture real 2.17 and 2.31 clients to confirm.
4. Ask for approval before any public run for 2.17, 2.31 or 2.05.

Status update, 2026-10-24 (decompile pass of 2.00, 2.17.1, 2.22, 2.31 and the Drive clients set)

- **Gap 1 (payload layouts of ids above 48 and 25, 39, 41, 44, 47, 156):** closed for 2.17.1, 2.22 and 2.31, except one small item (the extra step 2.31 adds in 156; the `PLO_SERVERWARP` field meanings and the `PLO_BOARDMODIFY2` trailer were settled by lab runs). See `protocol/packet-layouts.md` and `plo-layouts-217-231.tsv`.

- **Gap 2 (tamper-check content):** the lab values for 2.17, 2.22 and 2.31 are in `tampercheck-values.tsv`, and the builder is now understood: two chained CRC-32 words over the client's own memory image (`tampercheck-words.tsv`; `client-internals/decomp-findings-2026-10-24-b/01-tamper-check-id39.md`). Whether any server enforces it is still unknown.

- **Gap 6 (2.00):** a real 2.00 exe and three revisions are in the Drive clients set. The "allow-list patch" wording above means a config entry only: the lab server needs `GNW24011` in its allowed-versions list, no code patch (`client-internals/decomp-findings-2026-10-24-b/04-v2.00-client.md`). The 2.00 client still needs its list host redirected to the lab listserver, and its listserver frames have not been exercised.

- **Gap 5 (2.1 to 2.16 and 2.3):** unchanged. The Drive clients set has no 2.1.x to 2.3 builds, only the two 2.1 test builds (`client-internals/drive-clients-inventory.md`).

- **Gap 8:** client to server ids 36, 42 and 45 of 2.22 are still unmatched to call sites.

- Per-build handled server-to-client ids for 2.00 to 2.31 are now tabulated in `plo-handled-ids-by-build.tsv`, and the client-to-server ids per 2.0x build in `pli-sent-ids-by-build-20x.tsv` (2.00 lacks 35 to 38).

- **Update (lab batch b2):** the 2.00 listserver step is partly observed (client request seen; the list reply it accepted was synthetic) and the id 46 and id 186 trailer questions are settled. See `client-internals/decomp-findings-2026-10-24-b/07-lab-findings-46-186-200.md`.
- **Update (lab batch c):** `PLI_ENTERLEVEL` (46) is confirmed on doors, links and server warps in 2.31 and absent in 2.22; `PLI_SHOWIMGPLAYER` (24) is relayed unchanged and 2.22 sends it too with two players; `PLO_SERVERWARP` (178) field order is settled for 2.31. See `client-internals/decomp-findings-2026-10-24-b/08-lab-findings-levelchange-24-178.md`.

`client-internals/reference-reimplementations/README.md`

Reference reimplementations: what to trust

Date: 2026-10-30. Read-only study of two xtjoeytx repositories, weighed against the real-client notes in this repository and Seihai's own documentation. The code is summarized and cited, not copied. Nothing was sent to xtjoeytx (no issue, PR or fork).

repo	commit read	what it really is
`xtjoeytx/2dadventureclient`	`78040b7` (2020-02-29)	GServer-v2 (2020) with an SDL window and one local player bolted on. It hosts a server; it never connects to one. See `2dadventureclient.md`.
`xtjoeytx/LevelScriptEditor`	`2a81976` (2021-02-25)	A C# tool for bulk-editing NPC scripts in a folder of `.nw` levels, with "mark complete" tracking. Not a tile editor. See `levelscripteditor.md`.

Tags used in these notes: **[2dac]** 2dadventureclient source, **[lse]** LevelScriptEditor source, **[srv]** GServer-v2 source, **2.05**/**[2.22]** real-client notes already in this repository, **[cap]** lab captures, **[inf]** inferred.

Verdict in one table

behaviour	reference does	real clients	verdict for Seihai
wall test	one point at (x+2, y+2.75)	six-probe box, corner nudge, flush snap	**invented**, keep Seihai's box
walk speed	4 px per rendered frame (about 15 tiles/s at 60 fps)	0.5 tile per 50 ms (10 tiles/s)	**invented**, keep 50 ms steps
diagonal walking	both axes, one test, last key wins the facing	up, left, down, right as separate tested steps	**invented**
position on the wire	pixel props 78/79 for every client	2.05/2.22 send half-tile props 15/16; pixels from 2.3	**2.3+ only**, keep per-version
`gr.x` / `gr.y` flags	sent by a **server-supplied weapon** `-gr_movement`	—	**correction** to Seihai's `gr-flags.md`, see below
tile id to sheet pixel	512-id blocks of 16 x 32	same	**matches**
chest picture	tiles 1784 (closed) and 829 (open)	same ids in 2.22 `arrays.dat`	**matches**
draw order	all players and NPCs sorted by y	height layers (under, player, over, light)	**invented**
GANI timing	one shared timer per file, `WAIT` and `PLAYSOUND` skipped	per-object counter, `WAIT` holds	**incomplete**, don't copy
level links, gmap crossing, water, lava	not done for the local player	done	**missing**, nothing to learn
`.nw` read/write (LSE)	GLEVNW01 only, BADDY dropped on save	2.05/2.22 write BADDY	**lossy**, don't copy the writer

behaviour	reference does	real clients	verdict for Seihai
NPC metadata headers (LSE)	`//#[KEY:value]` comment lines	not a client feature	**invented**, harmless comments
multi-layer boards (LSE)	keeps each `BOARD` layer apart	2.22 ignores the layer field	**later era**, keep Seihai's merge

The correction: `gr.x` and `gr.y` come from the server

Seihai's `documentation/gr-flags.md` says "real clients keep the player's position in two client flags, `gr.x` and `gr.y`" and models it as a client behaviour per version (`LoginOnly` for 2.17, `Tracking` for 2.22). The source of those flags is the server:

- GServer-v2 ships a weapon `-gr_movement` (`bin/servers/default/weapons/weapon-gr_movement.txt`) and puts `WEAPON -gr_movement` in `defaultaccount.txt`, so every new account on a stock server gets it **[srv]**. The 2dadventureclient fork carries the same files **[2dac]**.
- The weapon is a clientside GS1 script. On `playerenters` or `timeout` it does `setstring gr.x,#v(int((playerx+0.25)*2)/2)` (and the same for `gr.y`), then `timeout=0.05`. Unless the flag `gr.off`, `gr.x.off` or `gr.y.off` is set **[srv]**.
- The server's flag handler swallows `gr.x`/`gr.y`/`gr.z` from clients below 2.3 and turns them into props 15/16/(z) for the other players, but only on ticks where no real movement prop arrived. The readme says this is to "simulate the smooth movement as found in the Graal clients 2.3 and up" **[srv]**. The setting is `flaghack_movement` (default on).
- The lab runs stock GServer-v2 (`tools/lab-setup.md`), and the public GServer-RS run also sent `-gr_movement` in the login burst (`client-internals/decomp-findings-2026-10-25-l/04-public-server-run.md`).

So what the captures show fits a script running in the client:

- the value is the position rounded to half a tile, written without decimals for whole numbers (`#v` formatting) **[cap]**;
- a 50 ms cadence in 2.22 is the `timeout=0.05` loop;
- 2.17 sending once at login fits `playerenters` firing and the timeout loop not running there **[inf]**;
- 2.31 never sending is not explained by the script; the cause was not checked **[inf]**.

For Seihai: a client should not send `gr.x`/`gr.y` on its own. It should send them when, and only when, it runs a weapon script that sets them. On an official 2001-era server there is no such weapon, so a real 2.05 or 2.22 client would send nothing. Seihai's script engine already runs weapon scripts (the `§` line-break fix in Seihai's `public-gserver-run.md`), so the hard-coded `protocol.gr_flags` profile switch is at best a duplicate of that, and at worst a double send on a GServer-v2 server. Worth a lab run with the weapon removed from the lab account to confirm the real 2.22 then sends no `gr` flags.

What Seihai could adopt

Little of the client code, because the real-client notes here are already more precise. The useful parts:

1. **The `-gr_movement` explanation** above (a fix, not an adoption).
2. **GServer's version workarounds as a checklist of server quirks** a Seihai client will meet on GServer-v2: the server deletes `Bomb` and `Bow` right after login because "the client adds them", sends `-gr_zlib_fix` to 2.21 to 2.31, and pushes every big-map file to 2.31 because that client does not request it after `setmap` **[srv]**. These are server behaviours, not client ones, so a nostalgic Seihai should tolerate them, not imitate them.

3. **LevelScriptEditor's find-identical-scripts idea** for Seihai tooling: group NPCs across a level folder by exact script text (optionally the same image), edit once, apply to all. This fits the `npc-library` work. Take the idea, not the writer (see `levelscripteditor.md`).

Small thing found in passing (Seihai)

`client/src/assets/level.rs` still has `tile_src_rect`, which maps a tile id row-major on a 128-wide sheet (`col = id % 128`). That is the mapping `client-assets.md` corrected on 2026-10-25. Nothing calls it (the renderer in `render/tiles.rs` uses the right block mapping), so it is dead code that could mislead.

`client-internals/reference-reimplementations/2dadventureclient.md`

2dadventureclient, behaviour by behaviour

Source: `xtjoeytx/2dadventureclient` at `78040b7` (2020-02-29), read-only. Files: `server/src/TGameWindow.cpp`, `TClient.cpp`, `TAnimation.cpp`, `TImage.cpp`, `TLevel.cpp`, `TPlayer*.cpp`, `bin/data/`. Tags as in `README.md`.

What it is

The tree is GServer-v2 (the 2020 code, with the V8 NPC server) renamed `TServer` to `TClient`, plus an SDL/guichan window. At start it loads the local account `localplayer`, makes a `TPlayer` for it with no socket, marks it local and adds it to the server's own player list **[2dac]**. It still opens the listen socket, so other clients can join it. There is **no client-side network code**: no login builder, no codec, no `PLO_*` handlers. Every "client" behaviour is the server's own object model drawn to a window.

Consequence: it has not been run against real servers as a client, and it cannot tell us anything about the wire from the client's side. What it can show is how GServer-v2 reacts to a client that writes props, which is already covered in `server-notes/`.

Walking and collision

point	reference [2dac]	real clients	verdict
step size	`16 * 0.30` px per axis, truncated to 4 px because the step is an `int`	half a tile (8 px), 1.0 on foot, 1.2 fast (`decomp-findings-2026-10-25-l/01-walking-body-box.md`) **2.05**	invented
cadence	one step per drawn frame, frame capped at 60 fps, so about 240 px/s (15 tiles/s) and slower if the frame rate drops	one step per 50 ms, 10 tiles/s (Seihai `walk-pace.md`) **[cap]**	invented
keys	up, left, down, right all summed into one move; facing is the last held key in that order (right wins)	each key is its own tested step, in the order up, left, down, right **2.05**	invented
wall test	one point, `(pixelX + move + 32) / 16`, `(pixelY + move + 44) / 16`, so tile (x + 2, y + 2.75)	six points on the leading edge of the box x+0.5..2.5, y+1..3; first four must be free; 0.1-tile corner nudge; snap within 0.0625 **2.05**	invented; already recorded in `archival/foot-test-sdl.md`
wall rule	`tiletypes[id] >= 20`, off-board is wall	per-tileset rule; default tileset uses `arrays.dat` lists (`decomp-findings-2026-10-28-b/01-tile-type-tests.md`) **2.05**	GServer's table, not the client's rule

point	reference [2dac]	real clients	verdict
water, lava, chairs, beds	`isOnWater` exists, never called by walking	swim, lava burn, sit **2.05**	missing
chests, NPCs, other blockers	not tested	chest 2 x 2 and blocking sprites block **2.05**	missing
refused step	nothing happens	position put back, "pushing" state starts **2.05**	missing

Nothing here is worth adopting. Seihai's body box is the better reference.

Position on the wire

The local player writes props 78/79 (`PLPROP_X2`/`Y2`, pixels, low bit = negative) and then sprite and gani props **[2dac]**. Those are the 2.3+ precise props (`protocol/player-props.md`). 2.05 and 2.22 send half-tile props 15/16 **[2.05][cap]**. The server then derives the half-tile value as `x2 / 16` for older viewers **[srv]**. Keep Seihai's per-version choice.

`gr.x` / `gr.y`

Not a client feature: see `README.md`. The local account here carries `WEAPON -gr_movement` like the stock server's default account **[2dac][srv]**.

Levels, links and gmaps

point	reference [2dac]	real clients	verdict
level formats	the server's loader (GLEVNW01, GR-V1, Z3-V1, gmap)	same set by version (`build-catalogue.md`)	nothing new; see `formats/`
links	never tested for the local player, so doors do nothing	the client tests links on its own walk **2.05**	missing
gmap	draws every level of the map at its grid offset	crosses levels by walking (Seihai `gmap_cross` lab runs) **[cap]**	drawing only; walking stops at the level edge because off-board is a wall
camera	centred on the player's level-local pixel position, clamped to the map size	follows the player across the map	the camera ignores the gmap offset, so it is wrong outside the first level

Drawing

point	reference [2dac]	real clients	verdict
tile id to sheet	`col = (id / 512) * 16 + id % 16`, `row = (id % 512) / 16`	the same (`client-assets.md`)	**matches**
tileset	hard-coded `zold_tiles_dusty_01.png`	`pics1` or the level's `addtiledef`	invented for their own world
chests	2 x 2 tiles from sheet cells (56, 15) closed and (29, 19) open; open if the local player's chest list has it	those cells are ids 1784 and 829, the closed and opened chest ids in the 2.22 `arrays.dat` (Seihai `chest-drawing.md`)	**matches**
draw order	players and NPCs together, sorted by pixel y	height layers: `drawunderplayer`, default, players, `drawoverplayer`, `drawaslight` (Seihai `npc-draw-order.md`) **2.05**	invented; y-sort would break NPCs that rely on `drawoverplayer`
image files	only `.png` gets a texture; a `.gif` is registered and draws nothing	the classic packages ship `.gif` sheets (1.21 has `letters.gif`, `sprites.gif`, `pics1.gif`); later builds take `.png` too	limitation

point	reference [2dac]	real clients	verdict
names and chat	nickname centred at (x + 24, y + 44), chat at (x + 24, y − 34), TTF font, black 1-px shadow	chat drawn by the name routine, stacked up from the head, no balloon (Seihai `chat-log.md`) **2.05**	font and offsets are invented; not checked against the real offsets
HUD	an FPS counter and a test window	`state` sheet HUD (`status-display.md`)	missing

GANI playback

point	reference [2dac]	real clients	verdict
step timer	one timer per loaded `.gani`, shared by every object using it, advanced by `3 * frame time` against `0.05`	each object steps on its own; the wait counter starts at 1 and `WAIT` adds to it (`formats/gani.txt`)	invented
`WAIT`, `PLAYSOUND`	skipped	hold and sound cue	missing
`LOOP`, `CONTINUOUS`, `SETBACKTO`	parsed, not used	used	missing
`ATTR1`, `PARAM`	`ATTR1` draws nothing	attribute sprites (`body-colours-and-attributes.md`)	missing
body colours	none	palette per slot **2.05**	missing
`shadow` sprites	drawn at 2/3 alpha	not checked	unknown
placement rows	split on spaces, read as triples with `atoi` (works for `a b c, d e f` only because `atoi("c,")` stops at the comma)	comma-separated placements	accident, don't copy

Seihai's `walk-gani.md` already steps one frame per walk step and lists the real `WAIT` timing as open. The reference does not close that gap.

Chat and commands

Typing in the bottom field sets the chat prop on the local player **[2dac]**. The server's chat commands then run with the local player treated as admin: `warpto` and `.ul` skip the rights checks when the player is local **[2dac]**. That is a convenience of the fork, not a real-client behaviour.

Bottom line

The fork shows GServer-v2 from the inside, which `server-notes/` already covers from the full GServer source. As a client it is an early prototype: every client behaviour it has is simpler than what the 2.05 and 2.22 notes record, and several (y-sorted drawing, shared GANI timers, 15 tiles/s walking) would visibly break the feel if copied. Its two correct details (the tile sheet mapping and the chest cells) Seihai already has.

`client-internals/reference-reimplementations/levelscripteditor.md`

LevelScriptEditor

Source: `xtjoeytx/LevelScriptEditor` at `2a81976` (2021-02-25), read-only, C# WinForms (`src/Levels/*.cs`, `src/Forms/*.cs`, `src/State/GlobalState.cs`). Tags as in `README.md`.

What it is

Not a level editor. It opens a folder, loads every `*.nw` file, lists each level's NPCs in a tree, and lets you edit one NPC's script, image and a description. Its main feature is "replace similar scripts": find every NPC in the folder

whose script text is identical (optionally with the same image), and replace them all in one go, with a "mark complete" flag and a "N / M completed" counter **[lse]**. It reads like a tool for a script port (GS1 to GS2 on a big world), not for building levels.

The `.nw` reader

point	reference [lse]	real clients / existing notes	verdict
formats	`.nw` with first line `GLEVNW01` only	2.05 also opens `Z3-V1.03/1.04`, `GR-V1.00–1.03` (`level-editor-205.md`)	narrower
`BOARD x y w layer data`	skips runs outside 0..64 or with `x + w > 64`; keeps one board per layer index	2.22 does not read the layer field: every row goes on the one board (Seihai `level-editor.md`, from the 2.22 notes)	layers are a later-era idea
tile code	`index(c1) << 6 + index(c2)` on `A–Z a–z 0–9 + / =`	same, without = (`formats/glevnw01.txt`)	matches; an invalid character gives a bad id instead of ending the row
`LINK`	level name may contain spaces (everything before the last six tokens)	GServer and the notes say the name has no spaces	lenient, fine
`NPC image x y`	image may contain spaces; x and y as doubles	same; `-` for no image	matches
`SIGN`, `CHEST`	read	read	matches
`BADDY … BADDYEND`	**not read**	written by 2.05 and 2.22	see the writer
anything else (`HEIGHTS` …)	ignored	2.22 has `HEIGHTS` (Seihai `level-editor.md`)	see the writer

The `.nw` writer: lossy

On save the whole file is rebuilt from what was parsed **[lse]**:

- `GLEVNW01`, then 64 `BOARD 0 y 64 layer` rows per layer, then links, signs, chests, NPCs, each with `\n` line ends.
- **Baddies, heights and every unknown line are dropped.** Saving one NPC change deletes every baddy in that level.
- The order is LINK, SIGN, CHEST, NPC. 2.05 writes LINK, SIGN, CHEST, BADDY, NPC (`level-editor-205.md`).
- NPC x and y are written with .NET's default number format, which follows the machine's culture, so on a German or French Windows `30.5` is saved as `30,5`.
- NPC scripts are trimmed of leading and trailing blank space.

Seihai's level tool already went the other way on purpose: its `save` keeps the file's own lines and writes byte-identical output when nothing changed (Seihai `level-editor.md`). Do not take this writer.

NPC header comments

The tool stores its own metadata as comment lines at the top of a script: `//#[MARKED]`, `//#[DESC:text]` **[lse]**. A line starting `// NPC made by` right after them is kept separately and written back. Real clients treat these as plain `//` comments, so they are harmless in a level, but they are this tool's invention, not a Graal convention. If Seihai ever shows them, treat them as comments.

What is worth taking

The workflow, not the code: an index of identical scripts across a level folder, with one edit applied to every copy and a completion count. The `npc-library` already has 3,172 unique scripts; the same grouping would tell a Seihai level author which scripts repeat across GR-V1 levels and which ones are one-offs.

Version timeline

Precursors and early clients, then 1.x, 2.x, and later clients.

`client-internals/zelda-online-1998.md`

Zelda Online (1997–98) — precursor analysis

Date: 2026-10-14 Author: Stefan Knorr (knorr@informatik.uni-rostock.de) Copyright: GServ Standalone (C) Stefan Knorr 1997; Zelda Client for GServer (C) Stefan Knorr 1998

What this is

The Java Zelda Online local version that predates the Delphi Graal client. The server is a text-line lobby; the client is a full Zelda-style game that loads `.zelda` levels and optionally joins multiplayer on port 14801.

Layout (promoted to shanalysis/zelda-online-1998)

Path	Contents
`server/`	`GServer.java`, `Player.java` — lobby server
`game/`	`Zelda.java`, `ZeldaMain.java`, `ZeldaClient.java`, `MapPanel.java`, `Computer.java`, `FileLink.java`
`programming/`	`makeArrays.java`, `addSprites.java` — generate `arrays.dat`
`readme.txt`, `readme-2.txt`	original notes

Levels (51 `.zelda`) and binaries (class, gif, arrays.dat, audio, bat) moved to `/recycler/trash/`.

Server protocol (text lines, port 14801)

Prefix	Meaning
`N` / `O` / `S`	game#name (normal / observer / super)
`I` + 8-char id	server assigns id
`+` / `-`	enter / leave available list
`P` + id	play against
`C` + id	disconnect from party
`D` + data	game data relay
`V` + id	observe

Player ids are 8-char (`0000` + 4-digit). Game data is relayed between opponents; no binary framing.

Level format (`.zelda`)

Header: `Z3-V1.02` or `Z3-V1.03` (8 bytes).

Board: 64×64 tiles, RLE. Code bits = 11 (v1.02) or 12 (v1.03). High bit set means count; otherwise tile id. Double-repeat flag for alternating pairs.

After board: - text lines of `FileLink` records (`filename x y w h newx newy`) until `#` - binary computer records (4 bytes each: x, y, type, stop) - text sign records (`signx-32`, `signy-32`, coded text)

Tile / sprite tables

`makeArrays.java` writes `arrays.dat` with: - `nowall` (passable tile ids) - `swamp`, `jumpstone`, `bushobj` / replace, `liftobj` / replace, `chestfields` / replace, `movestone` - sprite placement tables (`spritevalues`, `hurtsprites`, `compusprites`, etc.)

`addSprites.java` appends the sprite crop table (x, y, w, h packed into ints).

Client connection

`ZeldaClient` connects to host:14801. On enter it sends `N` + level-prefix + `#def`. Subsequent `D` packets carry position, chat, nickname, sprite colors, computer moves, board modifies, extras, carries, shots. Leader election is by earliest login time.

Relation to later Graal

Zelda Online	Later Graal
port 14801	listserver / world ports evolve
text-line lobby	binary framed protocol with version token
`.zelda` RLE board	`.nw` GLEVNW01 text board
`arrays.dat` tile tables	`tiledefs.txt` / pics1.png
Java applet	Delphi client

This is the last point before the version-token and binary-packet era.

See also

`precursors-and-fan-builds.md` (2026-10-29) covers both Java versions from the uploads in more depth. Zelda Online 1 is a single-player applet with no network code. Zelda Online 2 ships 51 levels. It also covers the 11-bit `Z3-V1.02` code width, the full GServ line set, password saves, the ZeldaEditor and how its dialogs became 1st Graal's editor, and Java Graal.

`client-internals/precursors-and-fan-builds.md`

Precursors and fan builds

Date: 2026-10-29

This note covers a set of uploads that sit around the Delphi client line:

- the Zelda Online Java versions (1998)
- Stefan Knorr's Zelda level editor (1998)
- Java Graal (2000)
- the Graal Game Boy Color demo (1999)

- 39ster's Graal client 1.00 (2014)
- GraalX (2001)

The sources were read where they exist and the class files and binaries were read statically where they don't. Nothing was run. No code is reproduced here. Two of the "source" archives turned out to be decompiler output and are treated that way (see below).

What the uploads are

There are 14 uploads but only 9 distinct items. They were compared by content hash.

Upload(s)	Same as	What it is
`zeldaonline.zip`, `zeldaonline1.rar`	each other, except that the rar adds 36 levels	Zelda Online 1: a single-player Java applet
`zeldaonline1_src.zip`	–	Decompiler output for the Zelda Online 1 classes, made in 2003, not the author's source
`zeldaonline2.zip`, `zeldaonline2.rar`	each other, byte for byte. The zip is also the copy in `zeldaonline2.7z` in the project files	Zelda Online 2 "local version": game, server and the author's commented source
`zeldaonlineeditor.zip`, `zeldaonlineeditor.rar`	each other. The exe is also in the next item	The Zelda level editor (`ZeldaEditor.exe`)
`ZeldaOnline_aka_very_first_graal.zip`	–	The editor, the untouched `zeldaonline2.zip`, and an unpacked Zelda Online 2 that a player edited in February 2002
`javagraal_rip.rar`	–	Java Graal as served by Cyberjoueur: `Graal.jar`, loose classes, art and 261 level and asset files. The archive is damaged: 26 files (10 `.graal`, 15 `.zelda`, 1 more) fail extraction
`javagraal_src.zip`	–	Decompiler output for the five classes in `Graal.jar` (2003). Its class and method set matches the jar exactly
`graalgb.rar`, `Graal v1.10 - Adventure Demo (PD) [C].zip`	each other: the same 256 KB ROM	The Graal Game Boy Color demo
`GraalX.rar`	–	A 2.01-era Delphi client. Already in `build-catalogue.md`
`39ster Graal v1.00.7z`	–	A 2014 fan remake. It is not the 39ster 2.1 GServer covered in `ash/`

How they relate

```
1998-05  Zelda Online 1      Java 1.0 applet, single player, Z3-V1.02/1.03 levels
1998-07  Zelda Online 2      Java 1.1 app, adds multiplayer through GServ on 14801
1998-07  ZeldaEditor         Delphi 3, writes Z3-V1.03; its dialogs become 1st Graal's editor
1998-09  1st Graal           Delphi; reuses the editor forms and the relay lobby
1999-09  Graal GB demo       Game Boy Color homebrew by Knorr ("Graal v1.10 Adventure Demo")
2000-07  Java Graal          Zelda Online 2's client with Graal art, reading GR-V levels, relay on
14802
2001-03  GraalX              2.01 pre-release that keeps the .nw loader
2014-04  39ster client 1.00  offline fan remake on an SDL/GameMonkey engine
```

Zelda Online 1 (May 1998)

- A Java 1.0 applet (`Zelda` class, class file version 45). The page runs it at scale 2 in a 512×448 box. The class files are dated 16 May 1998.

- **Single player only.** There is no socket code, and levels, `arrays.dat`, images and sound are all fetched relative to the applet's codebase.
- The start level is `level1.zelda`. The rar ships 36 levels, all `Z3-V1.03`. The loader also accepts `Z3-V1.02`.
- The story is hard-coded per level name: people placed on `level13`, `level28`, `level22` and `cave2`, and chests on `level21`, `level24`, `village3`, `level16` and `level29`, all gated by progress flags.
- The class already holds the text alphabet that later Graal uses for sign text (the 97-entry string documented in `formats/sign-text.md`), together with the glyph-code list `A B X Y u d l r h x y z 1 2 3 4 ..`. So the Graal sign alphabet goes back to the first Java applet.

Zelda Online 2 "local version" (July 1998)

- A Java 1.1 *application*, not an applet. It opens a start window with "One-Player mode" and "Multiplayer mode" and a server address field that defaults to the loopback address. It refuses to run on Java 1.0.
- Files are read from the working directory, not over HTTP.
- The package has `game/`, `server/` and `programming/` folders, and the author's real source is included, with comments. The files are dated 18–26 July 1998.
- It ships 51 `Z3-V1.03` levels. `level1.zelda` is again the start level.
- There is an extra `ZeldaMain (1).class` at the top level. It differs from `game/ZeldaMain.class` in one constant only: the tile index of the "free stone" (the tile left behind when a stone is lifted) is 1023 instead of 584.
- The options window has a nickname field, an option to show your own name, sprite colours for cap, cap edge, belt and coat, and a **password**. The password is a 10-character save code (see below).
- The readme warns that the local version uses different tile indexes from the online server, so bushes and stones look wrong when the two are mixed.

Password saves

The save code is 10 characters from a 64-character alphabet, so it holds 60 bits. Packed into it are:

Field	Bits
player x (0–63)	6
player y (0–63)	6
level index (from a fixed level list)	9
rupees	10
darts	6
power, in half hearts	5
glove level 2, can shoot, has map, found brother	1 each
the four story chests opened	1 each
checksum: the number of set bits in the payload	6 (lowest)

The 60 bits are then XORed with a fixed mask and their positions transposed (bit *i* moves to `(i mod 10)·6 + i div 10`) before being written out six bits per character. Reading a code reverses those steps, checks the bit count, and fails on a wrong length or an unknown character.

Zelda Online 2 in the "very first graal" zip

The `ZeldaOnline2` folder in `ZeldaOnline_aka_very_first_graal.zip` is the stock release plus a player's edits from 13–15 February 2002:

- 4 new levels (`agrethouse1`, `agretweapons1`, `eggsecretroom1`, `gameover`)

- 4 changed levels (`level1`, `level11`, `level29`, `house15`)
- an `objects.dat` with 4 more stamps (109 instead of 105): "Chest - Red Rupee", "Blackness", "Big Blackness" and "Breakaway Wall Part"
- a `GServProto.txt` log line from one server start

None of this is from 1998.

ZeldaEditor (July 1998)

- Built with Delphi 3 (Borland RTL, "Copyright (c) 1983,97"). Linked around 25 July 1998.
- Main window `TZeldaFrame`, titled "Zelda Editor". It has a toolbar with New, Open, Save, Save As, Undo, Cut, Copy, Paste, Delete, "Turn Selection Into A Warp Field", Edit Links, Edit Baddies and Edit Signs. A stamp ("object") palette sits on the right with add, save and delete buttons.
- Dialogs: `TLinkList`, `TLinkOptions`, `TNewOpponent`, `TEditSigns`, `TObjectName` and `TShowCodes` (the sign glyph codes).
- It saves only `Z3-V1.03`. Window positions are kept in `ZeldaEditor.ini`, one `Name:left,top,width,height` line per window.

1st Graal grew out of this editor. The 1st Graal client (Sept 1998) carries `TEditSigns`, `TLinkList`, `TLinkOptions`, `TObjectName` and `TShowCodes` under the same names. `TEditSigns` differs only by a few pixels plus `KeyPreview` and a key handler. `TNewOpponent` became `TEditBaddy`, and the main form became `TAdventure`. So the Delphi Graal client started as the level editor with a game added, not the other way round. This fits Graal shipping with an editor built in.

`objects.dat` (stamps)

The editor's stamp list is a flat sequence of records:

- the name, ended by `\n`
- one byte for the width and one byte for the height
- width × height 32-bit little-endian tile ids, row by row

Tile id 2047 (`0x7FF`) is a transparent cell. The 1998 file has 105 stamps, from "Bush", "Sign", "Stone" and "PushStone" up to whole houses, bridges and a cathedral. The same format is still used by the `objects.dat` shipped with Graal 1.21.

Shared Java game data

`arrays.dat` is built by the `makeArrays` and `addSprites` tools in `programming/`. It is a list of typed integer arrays. Each array starts with a 32-bit big-endian header: the high 16 bits give the kind (0 for a flat int array, 1 for an array of arrays) and the low 16 bits the count. The values that follow are big-endian 32-bit ints.

The game reads these arrays in this fixed order:

1. non-wall tiles
2. swamp tiles
3. jump stones
4. bush objects and their replacements
5. liftable objects and their replacements
6. chest tiles and their replacements
7. movable stone

8. sprite crop table
9. hurt sprites
10. leap animation
11. baddy sprites
12. bumped sprites
13. explosion frames
14. swamp-soldier frames

Java Graal and 39ster's client also ship an `arrays.dat`. 39ster's is a plain-text tile-type table instead.

Level format as the Java code reads it

This confirms and extends `formats/level-binary.md`:

Magic	Code width	Read by
`Z3-V1.02`	**11 bits**	Zelda Online 1 and 2 only
`Z3-V1.03`	12	Zelda Online 1 and 2, ZeldaEditor (writes it), Java Graal
`Z3-V1.04`	12	Java Graal
`GR-V1.00`, `GR-V1.01`	12	Java Graal, 39ster
`GR-V1.02`, `GR-V1.03`	13	Java Graal, 39ster

- The run-length rules are the ones already in `level-binary.md` (control flag in the top bit, count in the low byte, `0x100` for a double run). A decoder that follows them reads all 271 levels in these uploads with no errors.
- In `Z3-V1.02` and `Z3-V1.03` the Java games read each baddy as a fixed 4-byte record: x, y, type, then a byte that is always `\n`. That is the same thing as "3 bytes plus an empty verse line". The list ends at a record whose first byte is `0xFF`.
- Link lines are `file x y w h newx newy`. A negative `newx` or `newy` means that coordinate is kept when warping.
- Sign lines are x+32, y+32, then text in the sign alphabet.
- Java Graal reads GR-V NPC lines but only shows the image: the script after `#` is dropped, and a missing `.gif` suffix is added. It skips the GR-V1.01+ chest section.

Network: GServ Standalone and the Java clients

Zelda Online 2's `server/` holds "GServ Standalone (C) Stefan Knorr 1997, based on JccVsd (C) Stephan Kulow". It is a game-neutral lobby that Knorr also used for chess (a `ChessSync` thread sends every "Chess" player a one-character timer line every second). It listens on **14801**, gives out ids `0000` plus a 4-digit counter that wraps at 9999, and logs to `GServProto.txt`. This is the lobby that `protocol/relay-1998.md` describes from the Delphi side. The source adds:

Line	Direction	Meaning
`N<game>#<name>`	C→S	Join as a player (state "waits") and get `I<id>`
`O<game>#<name>`	C→S	Join as an observer. You see players in that game who are already in a party
`S<name>`	C→S	Super user: sees every player in every game, listed as `name (game)`
`+`	C→S	Available again (after `-`)
`-`	C→S	Not available. Your id is withdrawn from the other players' lists

Line	Direction	Meaning
`P<id8>`	C→S	Join that player's party. You are linked to them and to everyone already in their party
`C`	C→S	Leave the party
`V<id8>`	C→S	Watch a player: you receive their party's `D` traffic
`D<data>`	C→S	Relay to everyone in your party
`H<score>`	C→S	High score. The client sends it and this server ignores it
`I<id8>`	S→C	Your id
`+<id8><name>`	S→C	A player in your game is available. The name follows the id directly
`-<id8>`	S→C	That player left or became unavailable
`P<id8>`	S→C	That player is now in your party
`C<id8>`	S→C	That player left your party
`D<data>`	S→C	Relayed data, unchanged. The server does not add the sender

Zelda Online 2 client

- Connects to the typed address on 14801.
- Room name: `Z` + level file name without extension + `1.6`, for example `Zlevel11.6`. Every level is its own room. On a level change the client sends `C` and `-`, then joins the new room with the name `def`. The real nickname is sent in `D` later.
- When the first `+` arrives it sends `P` for that player straight away. On `P` it pushes its state: login time, position, chat, nickname, colours and carried object.
- `D` payload types: `P C N S M H B T D E O W A`. Their layouts are the ones in `relay-1998.md`, and the 1st Graal kept them. The level-leader rule (earliest login time, ties broken by the lower id) also comes from here.
- The client clamps baddy knock-back to ±4 before encoding. One of the four clamps writes the wrong axis, so a large positive vertical knock-back overwrites the horizontal value.

Java Graal client

- Connects to **its own codebase host on 14802**, the Cyberjoueur relay port that the Delphi betas also used.
- Room name: `G2` + level file name. A second "fixed room `Graal2`" mode exists in the class but the applet never selects it.
- `D` types: the Zelda Online 2 set without `S` (colours) and with `L` (level name) added.
- Without `S`, Java players appear with default colours. Delphi clients of the same period send types the applet ignores (`F`, `G`, `I`, `Q`, `R`, `U`, `X` to `Z`, `0` to `8`). So mixed sessions would only share movement, chat, board changes, baddies, items and arrows. This is *inferred*, not observed.
- **The `G2` prefix and the `Graal2` fallback are exactly what 4th Graal alpha (October 1998) uses.** The room prefixes and fallback names were read from the string tables of all 18 betas. 1st and 2nd use `Zelda2`, 3rd and 4th use `Graal` with a bare `G` prefix, and 4th alpha alone uses `G2` / `Graal2`. From 5th Graal on (`G3` / `Graal3` up to `G8` / `Graal8`) the rooms no longer match.
- So Java Graal speaks the relay dialect of 4th Graal alpha. Its `D` types are all ones that 4th alpha also sends or handles, and 4th alpha's extra types (`I`, `U`, `X`, `Y`, `Z`, `Q`, `F`, `G`, `R`) are ones the applet ignores. A Java player and a 4th alpha player on the same level file were in the same room. Later betas were not. This is read from the binaries and the Java source, not from a capture. The only open point is whether 4th alpha appends the bare file name for a local level, as the applet does. **Run live on 2026-10-31:** it does (`G2level13.graal`),

and a Java Graal and a 4th alpha player met and chatted on Knorr's GServ. Java Graal and 5th Graal met through a room-renaming proxy (`relay-room-bridge.md`).
- The applet was built in July 2000, 20 months after 4th alpha, so its dialect was already long out of date. It only made sense on a relay that still had 4th alpha clients, or Java players alone.

Java Graal (July 2000)

- `Graal.jar` holds 5 classes (`Graal`, `GraalClient`, `Computer`, `FileLink`, `ServerPlayer`), all dated 2 July 2000 and signed (SHA and MD5 digests in the manifest). The rip's loose copies are identical to the jar's.
- It is a Java 1.1 applet (512×418) embedded in a Cyberjoueur page next to their Bomberman, chess, Reversi and Tetris applets, with the Flycast ad block from 1999. The rip itself was taken on 23 February 2002.
- In code terms it is Zelda Online 2's game and client with Graal art (`pics1.gif`, `sprites.gif`, `letters.gif`) and Graal levels. It starts on `level13.graal` and has a `nickname:` chat command.
- `levels/` holds 180 readable levels (plus 25 damaged ones), NPC images and a few WAV files. Many `.zelda` files actually carry `GR-V` headers. The level names (`ebon*`, `mod*`, `sominside*`, `dungeona*`) look like player-made Cyberjoueur web levels.

Graal Game Boy Color demo (1999)

The full disassembly is written up in `gameboy-demo.md`. In short:

- The ROM header has the title `GRAAL`. It is dual-mode (it runs on both Game Boy and Game Boy Color) on an MBC1 cartridge with 256 KB of ROM. Only banks 0 to 5 are used. The header checksum is valid and the file is dated 29 September 1999.
- It is a C program built with GBDK 2.x and LCC. The whole game runs in the VBlank interrupt.
- "Welcome to Graal / Demo version by Stefan Knorr" is a sign in the first room; there is no title screen. So this is Knorr's own port, not a fan project. The "(PD)" in the dump name marks it as public-domain homebrew.
- There are seven rooms of 32×32 tiles. Their links use the PC client's link record, with a room number in place of the file name.
- Gameplay text includes a captive you free, who shows you how to lift and throw stones and then tells you to press Select and switch to "Fung Si Yun".
- Names in the ROM such as Lizardon, Pachukite, Pach Soldier, Gold Soldier, Water Athena and Red Squall belong to a table of 31 character types, not only to baddies. The Select menu lets the player switch to any of them.
- The text font is ordered space, A–Z, a–z, 0–9, then punctuation, starting at byte `0x20`. This is not the PC sign alphabet.

39ster's Graal client 1.00 (2014)

- All files are dated 23 April 2014. `GameMaker.exe` is a small C++ engine on SDL 1.2 (plus zlib and stb_image), unrelated to YoYo's GameMaker. It embeds the GameMonkey scripting language, and all of the Graal logic is in `main.gm`, about 1,500 lines of script.
- It reads `GLEVNW01 .nw` levels and `GR-V1.00`–`1.03` levels with the same 12- and 13-bit rule, plus `.gani` animations. It converts tile ids into the 2.x `pics1.png` layout and starts on `level13.nw`.
- **Offline only.** It has no socket code, and NPC scripts are not run: the NPC object has an execute hook with the call commented out, so NPCs only show their images.
- It bundles 1,733 `.nw` levels and 1,321 GIFs collected from community worlds.

GraalX

GraalX is already in `build-catalogue.md`: a pre-release of 2.01, linked 17 hours before it, with 2.01's token, list port and DES-hidden list host, which keeps the `GLEVNW01` reader and writer. A form-by-form comparison adds one more check. GraalX has the same 28 forms as 2.0 rev3 and 2.01, and differs from 2.01 only in two control positions on the main form.

Open

- ~~Whether a Delphi beta ever used the `G2` room prefix.~~ Answered above: 4th Graal alpha did, and no other beta.
- ~~The GB demo's full script and map format.~~ Answered in `gameboy-demo.md`.
- Who made the February 2002 Zelda Online 2 edits.

`client-internals/precursor-runs-2026-10-30.md`

Precursor runs and behaviour constants (2026-10-30)

This note follows `precursors-and-fan-builds.md`. That note was a static read. This time the Java precursors and the Game Boy Color demo were **run** in a sandbox, and the game-feel numbers were read out of the Zelda Online 2 and Java Graal sources. The aim is to give Seihai's per-version profiles exact numbers for the oldest builds: tick rate, walking, pushing against walls, lifting, bush drops, respawn, and how chat is drawn.

Tags: **[src]** read from the Java source, **[live]** seen in a running client or on the wire. No code is quoted. Screenshots are in the project's shared folder under `graal-decompile/precursors/runs-2026-10-30/`, not in this repo.

How it was run

- **Zelda Online 2.** The bundled GServ Standalone server ran on port 14801, with two copies of the applet in an applet harness under Xvfb. Both clients joined, saw each other, walked, cut bushes, pushed against walls and chatted. The traffic was captured with tcpdump. **[live]**
- **Java Graal (July 2000).** Two copies of the applet ran in the same harness. The applet only connects to its own codebase host on 14802, so a plain TCP forwarder from 14802 to the GServ server on 14801 stood in for the old relay. Both players saw each other. **[live]**
- **Game Boy Color demo.** Run in PyBoy with scripted input. **[live]**

Behaviour constants (Zelda Online 2 and Java Graal)

Every number below is the same in both games. The Java Graal applet is the Zelda Online 2 engine, re-skinned with Graal art and given new levels.

What	Value	Tag
Main loop	50 ms per tick (20 Hz)	[src]
Walk step	half a tile per tick while a key is held	[src]
Corner slide (walking into the edge of a block)	a quarter of the walk step, sideways around the corner	[src]
Sword	5 ticks; hits are tested on ticks 1 to 3 only	[src]
Being hurt	40 ticks of hurt state, half a heart of damage	[src]
Shield bump	4 ticks	[src]

What	Value	Tag
Lifting	5 ticks	[src]
Baddy moves	sent by the level leader every 2 ticks (100 ms)	[src]

Walking into a wall

The player keeps a push counter that counts ticks spent walking into something solid. **[src]**, and the pose was seen in the running client. **[live]**

- **Ticks 0 to 17 (0.9 s):** nothing visible. The player stands in the walk pose against the wall.
- **Tick 18:** the push pose starts. The push frames advance one step every 4 ticks. The ledge-jump check also runs from tick 18, so a jump down a ledge needs 0.9 s of pushing.
- **Tick 28 (1.4 s):** a push stone in front of the player moves.
- **Tick 38:** the counter goes back to 18, so the push animation loops without restarting the 0.9 s wait.
- Letting go of the key, or turning, resets the counter to 0.

Compare `ledges-push-pull.md` for the Delphi clients.

Bushes, stones and pots

- **Cutting a bush** rolls one number from 0 to 99. **[src]**
  - 0 to 24: one green rupee (25%).
  - 25 to 29: a heart (5%), worth one heart of power.
  - 30 and 31: a five-rupee item (2%).
  - 32 to 99: nothing (68%).
- **Lifting.** There are four liftable object kinds. Kinds 0 to 2 can always be lifted. Kind 3 (the heavy stone) needs the stronger glove. A lifted bush (kind 0) rolls the same drop as a cut one. Kind 2 always leaves a heart. **[src]**
- **While carrying**, the player is drawn with a separate set of four carry sprites, one per direction, with the object drawn above the head. The player is not hidden under the object in these games. Hiding under a carried bush came later; see `carry-hide-and-glove.md`. **[src]**
- **Respawn: there is no timer.** Cut bushes, lifted stones and broken pots stay gone only for the visit. Every board change is cleared when a level loads, so everything is back when the player leaves and comes back. Nothing grows back while the player stays on the level. **[src]** Re-entry was not tested live.
- **Sharing board changes.** Each change is relayed as a `B` line. The level leader (earliest login time) sends all of the current changes to every newcomer, so a player who joins sees the bushes already cut. **[src]** The captured sessions had no board changes on the wire, because nothing was cut while both clients were in the same room. **[live]**

Chat

There is no chat bubble. [src], [live]

- Chat text is drawn in TimesRoman bold, 20 points, in black, over a white copy of the same text shifted one pixel right and one pixel down. The result reads as black text with a thin white edge.
- The text is centred on the player (the player's x plus 12 pixels, times the zoom), just above the head.
- The nickname is drawn in the same style three tiles lower, under the feet.
- There is no box, no background and no tail. The bubble seen in later Graal belongs to the Delphi clients, not to these.

What the wire showed

- **Room names.** Zelda Online 2 joined as `NZlevel11.6#def`, and Java Graal joined as `NG2level13.graal#def`. These match what the sources say. **[live]**
- **Join order.** The server answers `I<id8>`, then sends `+<id8><name>` for each available player. The client answers the first `+` with `P<id8>` and then pushes its whole state: login time, position, chat, nickname, colours and carried object. **[live]**
- **Doubled relay.** When two clients see each other at the same moment, they both send `P` for the other. GServ then links the pair twice, and every `D` line after that is delivered twice. One capture had 1144 relayed lines for 572 sent. Nothing visibly broke, because most `D` lines are state updates such as position (550 of 578 sent lines in the Zelda Online 2 capture were `M` moves). A doubled one-off event, such as a board change, would simply be applied twice. It happened with both Zelda Online 2 and Java Graal. A server that emulates GServ for these clients does not need to reproduce it. **[live]**

Game Boy Color demo in an emulator

This adds a run to the static read in `gameboy-demo.md`. Everything it says about the screens held up. **[live]**

- There is no title screen. The ROM goes straight from the Nintendo logo to room 0, the outdoor start with palm trees, bushes and flowers.
- **Select** opens the character carousel in the window. It shows three heads with the middle one's name in the font, and the item slots labelled "Button B" and "Button A" under it. Turning it left and right went through Vincent, Fung Si Yun, Pach Soldier, Ninji, Lunar and Squall in that order, and each step changed the player's look straight away. This confirms the static finding that the carousel is a free picker over all character types.
- The item icons under the slots change with the character. For example Fung Si Yun shows a green item in slot A, and Pach Soldier shows a blue sword in slot B.
- Walking around room 0 and the next rooms showed the NPCs and log walls where the room table in `gameboy-demo.md` puts them.
- Start (music) was not checked by ear here. A later, longer run (see "Seen in an emulator" in `gameboy-demo.md`) found that Start mutes the game.

What this means for Seihai profiles

For a "Zelda Online 2" or "Java Graal" profile:

- Run game logic at 20 Hz and move half a tile per tick.
- Show the push pose after 0.9 s against a wall, and move push stones at 1.4 s.
- Use the bush drop table above.
- Never regrow anything during a visit, and restore everything on level load.
- Draw chat as bold serif black text with a white one-pixel offset and no bubble.
- Do not hide the player under a carried object.

For the Delphi Graal builds these numbers are a starting point only. Their wall push, hiding and respawn timers have their own notes and are not covered here.

Open

- Whether the 1st to 4th Delphi betas still used the 0.9 s / 1.4 s push timings. The Delphi code has not been compared yet.

- The GB demo's music format, which depends on cartridge RAM that the ROM never writes. In the emulator, Start only muted the game (see `gameboy-demo.md`).

`client-internals/early-clients.md`

Early Graal clients — extracted details

Date: 2026-10-14

Derived from the decompile corpus. Raw exports stay in `/shdecompiling/`. This note is cleaned.

Lineage

Era	Binary	Port	Levels	Token
Zelda Online (Java, 1997–98)	applet	14801	`.zelda` (Z3-V1 RLE)	none
1st Graal (Delphi)	1st Graal.exe	14801	`.zelda` (24 refs) + `.graal` (4 refs)	none
1.39	Graal 139.exe	14900	transitional	`GNW07080`
2.05	Graal.exe v2.05	—	`.nw` (GLEVNW01)	`GNW03081`

1st Graal

- 505 functions. No version token. No `GNW` string.
- Default host: the loopback address, port `14801` (same as the Java precursor).
- Save format magic: `Graalsav1`.
- Level names hard-coded: `level1.zelda`, `cave2.zelda`, `house1.graal`, `village1.zelda`, and others. Both extensions coexist.
- Sign text present: `"Sign at ("`.
- String table includes `NZelda2#def`.
- **Room name is not always the constant `Zelda2`** (2026-10-24 decompile). One branch of the join routine uses `Zelda2`; the other, chosen by an argument given when the network client object is created, builds the room from the current level file's base name as `Z` + base + `2` (a level `level1.zelda` would give `Zlevel12`) and then runs a follow-up routine (not traced). Which branch normal play uses was not traced; the two-client local mode is a likely user of the per-level form. The statement in `protocol/relay-1998.md` that 1st and 2nd Graal always join `Zelda2` holds for one branch only. The client always sends the name `def` in the join line and the real nickname later in a `D` payload.

1.39

- Token built as eight char immediates in the login/token builder: `G N W 0 7 0 8 0`.
- Connects to port `14900`.
- **Send helper drops payloads that contain a newline** (2026-10-24 decompile; the same helper exists in 2.05 rev2). A message is `id+32`, payload, newline, so a newline inside the payload would break the framing and the helper silently discards the whole message instead of sending it. The queue is flushed when it would pass 4095 bytes. In 1.39 the client closes the connection if more than about 256 KiB of compressed output is pending (the 2.05 figure was not checked). A client built on this framing should reject or escape payloads containing a newline rather than expect an error.
- Dhrake recovered ~390 names. Preferred corpus is `graal-139-dhrake/`.

Recovered classes

- `TAdventureClient_Create` / `InitTable`
- `TAdventureLevel_Create` / `InitTable`
- `TAdventureCommand_Create` / `InitTable`
- `TAdventureOperation_Create` / `InitTable`
- `TAdventureVariable_Create` / `InitTable`
- `TAdventureWeapon_InitTable`
- `TAdventureServer_InitTable`
- `TAdventureBoolOperation_InitTable`
- `TAdventure_DXDrawFinalize` / `DXDrawRestoreSurface` / `DXTimer`
- UI: `TAdventure_PlayButtonClick`, `SpeedButton`, `StatusBar`, `TabControl`, `HorzScrollBar`, `PaintBox`, `FormCanResize`

Script events present

`playerenters`, `playertouchsme`, `playerendsreading`, `playerlaysitem`, `playerisfemale`, `playerismale`, `playeronhorse`, `playerswimming`, `wasshot`, `shotbybaddy`, `shotbyplayer`, `compusdied`, `timeout`, `created`, `gotheart_`, `drawoverplayer`, `drawunderplayer`, `followsplayer`.

Properties

`hearts`, `rupees`, `shield`, `shieldpower`, `hasweapon`, `isweapon`, `weaponsenabled`, `playerx`, `playery`, `carriesnpc`.

Sounds

`arrow.wav`, `arrowon.wav`, `axe.wav`, `bomb.wav`, `chest.wav`, `compudead.wav`, `crush.wav`, `dead.wav`, `extra.wav`, `fire.wav`, `frog.wav`, `goal.wav`, `item.wav`, `item2.wav`, `jump.wav`, `nextpage.wav`, `put.wav`, `sign.wav`, `steps.wav`, `stonemove.wav`, `sword.wav`, `swordon.wav`, `thunder.wav`, `water.wav`.

Config keys (`graalconfig.txt`)

`serveraddr=`, `fixedport=`, `fixedudpport=`, `dontconnectlevels=`, `dontloadlistheads=`, `loadcolorsfromserver=`, `sortplayersbyname=`, `limitnicknames=`, `nicknamelimit=`, `gamemode=`, `language=`, `font=`, `fontsize=`, `musicon=`, `joystickon=`, `nodirectinput=`, `noudp=`, `nodecompressionerror=`, `nohtmlimages=`, `nohtmlinpms=`, `nomassmessages=`, `notoalls=`, `reversestereo=`, `saveimages=`, `compabilitydraw=`.

Files

`graal.bmp`, `graal.png`, `graalmap.bmp`, `levels\recordpos.txt`, `weblevels\`. Extensions: `.code`, `.gif`, `.graal`.

Rendering

DirectDraw and DirectSound. `TAdventure_DXDraw*` handles surface restore and the timer. `compabilitydraw=` is a config toggle.

2.05

- Token built as char immediates in `TAdventureClient_Create`: `GNW03081`.
- `DispatchGs1.txt` is the GS1 interpreter, not the packet dispatcher.
- Levels are GLEVNW01 `.nw`. Framing is length + zlib. Post-login uses the Borland LCG.

Protocol transition summary

1. Java precursor: text lines on 14801, `.zelda` RLE.
2. 1st Graal: same port, Delphi, `.zelda` dominant, no token, `Graalsav1` saves.
3. 1.39: port 14900, token `GNW07080`, full event set, DirectX, config file.
4. 2.05+: length + zlib, `.nw`, token `GNW03081`, post-login LCG.

Correction (2026-10-21, from `build-catalogue.md`): zlib packet compression starts with 1.2 (May 1999), and the post-login LCG insertion (gen 3) is already in 1.41 rev1 and every 2.0x build, not only 2.05+. 1.39 has no LCG insertion. `.nw` is in NewMain 1.42 and 2.05 but not in 2.00–2.04.

1998 betas to 1.32 (added 2026-10-21)

All 25 client builds from 1st Graal to 1.32 have now been analysed, together with the bundled GServer 1.1.9, 1.21, 1.28 and 1.3 and RemoteControl 1.21, 1.28 and 1.3. See:

- `protocol/relay-1998.md`: the gen-1 text relay (lobby lines, `D` payload types per build, game names `Zelda2` and `Graal` to `Graal8`).
- `protocol/gserv-1999.md`: the GSERV007 to GSERV018 login, zlib framing, which tokens each GServer accepts, and the id ranges.
- `formats/graalsav.md`: Graalsav1 to Graalsv11 and the DES-wrapped `.code` and `.gsave` files.

Two corrections to earlier assumptions:

- `TAdventureServer` (from 15th Graal) is a hostname-to-IP cache record, not a server or a peer-to-peer mode. The 1998 clients only ever connect out to a relay.
- Port 14801 is the compiled-in local default, and 14802 is the Cyberjoueur relay that the connect dialog points at. Neither one is a listening port in the client.

In the 1998 clients the network classes are `TAdventureClient` (the relay connection), `TOtherClient` (one per relay peer) and, from 3rd Graal, `TWebClient` (the HTTP level fetcher). The script classes listed above for 1.39 (`TAdventureLevel`, `TAdventureCommand` …) are not in 1st or 3rd Graal.

`version-profiles/matrix/rules-1st-graal.md`

1st Graal death, hurt, knockback, sword reach and wall test

Date: 2026-11-01. Closes matrix unknown #1 for **1st Graal** (era A). **1.32 stays open** (no decompile; see the end).

Tags: **[bin]** from a static read of the 1st Graal client export (plain behaviour only). The export is the VCL-lossy one already noted in `../../client-internals/early-clients.md`: a few hundred functions, most of them the Delphi library, with the player rules recovered from the routines at the end. Counters are in the 50 ms tick already surveyed for every Delphi build. Where the export's comparisons disagree with themselves, that is called out and not tagged as a number.

Compared with `rules-139.md`. 1st Graal has no AP bar, no ganis, and no later STATUS property.

Summary

Rule	1st Graal	1.39 (for contrast)	Tag
Hurt lock-out	action mode 3 for 40 ticks; a new hit is refused while that mode lasts	same 40	**[bin]**
Knockback steps	one step per tick while the shared counter reads 39 down to 35 (five steps); twelve probes; a blocked step is undone	same five steps	**[bin]**
Knockback direction	baddy and projectile hits store an axis unit step from the attacker's facing; a sword hit stores the centre-to-centre delta	unit vector; a zero vector becomes one tile east	**[bin]**
Hurt drawing	full hurt body while the counter is above 30; odd counters blink to a single frame	pose ~10 ticks, then blink **[lab]**	**[bin]**
Sword reach	swing frames 6 to 10; ahead by up to 4 tiles; sideways strictly under 2	0–4 ahead, sideways under 2	**[bin]**
Sword power	half a heart, fixed	not a fixed half heart in the later profile	**[bin]**
Death	hearts at or below 0: clamp to 0, clear the counter and the action mode, life-state becomes dead	same clamp; then a 30-tick timer	**[bin]**
Restart after death	a key, once the shared counter is above 27, reloads `house1.graal` at x 24.5, y 30 with hearts and the heart cap both set to 3	saved entry, or (30.5, 30) online; hearts to max	**[bin]**
Wall test	six probes; a blocked step uses the push counter, which wraps from above 37 back to 18; at 18 a ledge test starts an 8-tick hop	six probes; same 18 / wrap-at-38 push counters	**[bin]**

Hurt and knockback [bin]

A hit is refused while the action mode is already 3. Sword hits, baddy touches and the projectile hits below all use one entry:

1. If the player is carrying something, that carry is dropped.
2. Action mode becomes 3 and the shared counter becomes 40.
3. Hearts change by the hit's power (negative hurts). Hearts are a fractional count. They do not rise above the integer heart cap. At or below 0, the death path below runs, and it clears the mode and the counter that this entry just set.
4. A knockback step is stored.

The hurt tick then drops the counter by one. At 0 the action mode returns to normal. While the counter is still above 34 after that drop (readings 39 down to 35), the player moves by the stored step. Twelve points, four groups of three, are tested against the block test; if any is blocked, that tick's step is undone.

Walking is also refused during hurt while the counter is still at least 30. Below that, and until the counter hits 0, walking is allowed again but a new hit is still refused.

Drawing, while the counter is above 30, uses the hurt body. An even counter, or any counter still above 34, draws the full hurt frame list; an odd counter at or below 34 draws a single frame. Once the counter is 30 or below and the mode is still hurt, the normal body is drawn, and odd counters again draw only one frame.

Baddy touch and the common projectile hit store an axis-aligned step of length 1 from the attacker's facing: facing 0 decreases Y, facing 1 decreases X, facing 2 increases Y, facing 3 increases X. A sword hit instead stores the delta between centres (victim minus attacker). The export then scales that delta by its length, but the length adjustment is applied twice to the horizontal component and the vertical component is left at the half-delta, which is the same class of bad assignment this export shows elsewhere. Direction "away from the attacker" is tagged. Unit length for a sword knockback is not.

There is no "zero step becomes one tile east" in these stores.

Powers that reach this entry:

- Sword, and a baddy touch: half a heart.
- A baddy touch counts when both axes are within 1 tile of the body centres (player centre at x+1.5, y+2; baddy centre at x+1, y+1).
- Two projectile types (sprites 65 and 261) hurt by half a heart; the other types in that list hurt by one heart. The same 40-tick mode is used.
- An arrow whose sprite is 107 to 110 is blocked, instead of hurting, when its direction is the opposite of the player's facing and the player's sprite index is below 6. Otherwise an arrow hurts by half a heart.

Sword reach [bin]

Another player's swing is tested only while the local life-state is the normal value. The other player's packed sprite keeps the facing in the low two bits and the frame above that. A hit is possible only on frames 6, 7, 8, 9 and 10.

The box is facing-relative, measured between centres (each centre is x+1.5, y+2). The victim must sit ahead of the attacker, by a span of 4 tiles, and the sideways distance must be strictly under 2. On two facings the export includes both ends of the 4-tile span; on the other two the ends are exclusive. That split is treated as decompiler noise. The tagged reach is "up to 4 tiles ahead, sideways strictly under 2", the same shape as 1.39, and the exact 0 and 4 edges are not tagged.

Power is half a heart, not a sword-power scale. 1st Graal's sword does not grow.

Death and the restart [bin]

When hearts fall to 0 or below, the heart write:

1. Clamps hearts to 0.
2. Clears the shared counter and the action mode.
3. Sets the life-state to dead.
4. Notifies the relay object when one exists. The payload was not read. There is no later-style STATUS property on this client, and this path does not spill the inventory.

The dead life-state changes drawing: the tile buffer is replaced by a filled rectangle, and a text panel is drawn. The body uses one fixed sprite (index 157) only once the shared counter is above 27. Before that, with the action mode already cleared, the normal body is drawn.

A keypress while dead restarts the player once that counter is above 27. The same restart also runs for one other life-state (value 4), with no counter test. The key index handled before this test does not restart; the other indexes do. The restart is the new-game setup, not a saved `Graalsav1` entry:

- Level `house1.graal`.
- Position x 24.5, y 30. The second local player is shifted 12 tiles toward increasing X (x 36.5, same y).
- Facing 2 (increasing Y).
- Hearts set to 3, and the heart cap set to 3, whatever the cap was before.
- Carry, counter and action mode cleared.

The loop that advances the shared counter while dead was not found in this export. Death itself sets the counter to 0, and both the dead sprite and the restart key wait for a reading above 27. A 30-tick cap, of the kind 1.39 has, is therefore not tagged. No death sound is on this path; this build has no sound engine.

Wall and walk test [bin]

The step added for a facing comes from a per-facing table. The length is not an immediate in the routine. The offline run already measured 0.5 tile per tick (`live-runs-2026-10-30.md`); that measurement is not re-done here.

The candidate point is tested with six probes. The block test fails closed: a point is blocked when an object's box contains it, when it lies outside the 64 by 64 board, when either coordinate is below half a tile, or when the tile is not on the passable-tile list. The list itself was not copied out.

If any of the first four probes is blocked, the action mode becomes the push mode and the stepped axis is undone. A fractional part on the free axis is snapped (facings 0 and 2 snap Y down; facings 1 and 3 snap X onto a half-tile). A particular pattern of the six probes instead adds a nudge from a second per-facing table; the nudge offsets were not read. A nudged step clears the push counter. A plain blocked step increments it.

The push counter wraps: a reading above 37 is set back to 18. When the counter is 18 and a ledge-tile test passes, the action mode becomes the hop and the counter becomes 8. The hop then follows a facing table for those 8 ticks and returns to the normal mode. While pushing, once the counter is above 17 and the player is not carrying, the body sprite steps through the push frames.

What Seihai should change

See `seihai-gaps.md`. Seihai has no era A profile. When one is added, the 1.39 / 2.05 combat row is the wrong default: sword power is a fixed half heart, hurt is 40 ticks with five knockback steps, a killing blow goes straight to the dead life-state, and the death key restarts `house1.graal` at (24.5, 30) with 3 hearts rather than restoring a saved cap. The walk test is already the six-probe, push-counter family.

1.32

Not tagged: a listing of the Decompile folder on 2026-11-01, client exports and the Accessories folder, still has no 1.32 export, so no death, hurt or wall number is added.

Still open

- The routine that counts the shared counter up while dead, and whether it stops at 30.
- Exact inclusion of the 0-tile and 4-tile edges of the sword box, and unit length of a sword knockback.

- The six walk-probe offsets, the corner-nudge offsets, and the numeric walk-step table (the 0.5 tile step is the live measurement only).
- 1.32 for all five rules, and the rest of eras A and B.

`client-internals/gameboy-demo.md`

Graal Game Boy Color demo (1999)

Date: 2026-10-29, emulator run added 2026-10-30

This note covers the full disassembly of `Graal V1.1 Adventure Demo (PD) [C].gbc` (262,144 bytes, MD5 2b2fef91…, the same bytes as `graal.gb` in `graalgb.rar`). The first look at this ROM is in `precursors-and-fan-builds.md`.

The disassembly itself is in the project's shared folder under `graal-decompile/gameboy-demo/`, not in this repo. That folder holds RGBDS-syntax sources for all 16 banks, the graphics as `.2bpp` files, PNG renders of every room, the scripts that produce them, and a README. The sources reassemble to a byte-identical ROM. That was checked with a small assembler written for this job; RGBDS itself was not run. This note is cleaned: it has no code and no code addresses. ROM locations are given only where they describe a data format. The first sections come from reading the code. The section "Seen in an emulator" adds what the running ROM did, and corrects the static read where they differ.

What it is built with

- **GBDK 2.x and its LCC C compiler.** The start-up code is GBDK's crt0: interrupt handler chains, the `.mode` table, `add_VBL` and the rest. The game code has LCC's fingerprints: arguments on the stack, `switch` statements as jump tables, and `int` constants read from a per-bank constant pool. Of 161 functions, 41 are GBDK library or crt0 routines (`set_bkg_data`, `set_win_tiles`, `set_sprite_palette`, `joypad`, `memcpy`, `arand`, the multiply and divide helpers and so on). The other 120 are the game.
- **Everything runs in the VBlank interrupt.** `main` sets up the screen, loads room 0, installs one VBlank handler with `add_VBL`, turns on sound, switches the CGB to double speed and returns. From then on crt0 just idles. The handler does one game tick: OAM DMA, scrolling, the window slide and music on every VBlank, and the game logic on every second one (about 30 Hz).
- **Banking.** One helper writes the MBC1 bank register and remembers the bank. Code in banks 1 and 3 is reached through small bank-0 wrappers that switch, call and switch back.

Header and bank map

The cartridge type is `$02` (MBC1+RAM), but the RAM size byte is 0. The CGB flag is `$80`, so the ROM runs on both models. Only banks 0 to 5 hold anything. Banks 6 to 15 are all `$FF`.

Bank	Contents
0	crt0, the GBDK library, and the core game: room loader, input, text box, Select menu, collision, the VBlank tick
1	Entity update and AI, entity drawing, lifting and throwing, camera, the music player and its note table
2	Graphics: two tilesets of 256 tiles, the 82-tile font, walk bitmaps, five BG palette sets, animated tiles, 172 sprite tiles
3	Objects (lifted and thrown things), sound effects, entity set-up, sprite sorting, two AI routines, the character table with heads, palettes and names

Bank	Contents
4	Room pointer table and rooms 0 to 4
5	Rooms 5 and 6

Screen and coordinates

- A room is exactly one 32×32-tile BG map, which is 256×256 pixels. Nothing is streamed: the loader writes the whole map to VRAM, and the camera scrolls over it.
- Positions are 16-bit values in half pixels. A map tile is 16 units (8 pixels).
- Directions are numbered 0 up, 1 left, 2 down, 3 right. That is the same order as the PC client.
- The window layer holds the text box and the Select menu, and slides up from the bottom of the screen.
- Sprites are 8×16. The game builds them in a WRAM buffer, sorts them by Y, and copies them to the DMA page every tick.

Room format

Bank 4 starts with a table of seven 16-bit header pointers. Rooms 0 to 4 are in bank 4 and rooms 5 and 6 in bank 5; the loader picks the bank as room / 5 + 4.

Header (13 bytes):

Offset	Size	Field
0	2	map pointer
2	1	tileset (0 outdoor, 1 indoor)
3	1	BG palette set (0 to 4)
4	1 + 2	link count, link list pointer
7	1 + 2	text count, text list pointer
10	1 + 2	NPC count, NPC list pointer

- **Map:** 1,024 tile ids, then 1,024 CGB attribute bytes (palette, VRAM bank, flips), both 32×32. The loader keeps two WRAM copies of the tile ids: one as loaded and one that changes when things are lifted. Collision reads the second one.
- **Link (8 bytes):** x, y, width, height, new x, new y, then the target room as a 16-bit number. A new x or y of −1 keeps the player's coordinate. This is the PC client's link record (`file x y w h newx newy`), with a room number in place of the file name. The player's tile is tested against the rectangle on every step.
- **Text (4 bytes):** x, y and a pointer to a zero-terminated string. A text at (0, 0) is never read from the map; NPCs and events show it by index. Signs are read only while the player faces up and the tile above is solid.
- **NPC (16 bytes):** x and y in tiles, direction, character type, two unknown bytes, a text index (−1 for none), five movement bytes (a mode, then patrol bounds), a 16-bit flag word whose eleven low bits are copied into separate entity fields, and a 16-bit event number. Flag bit 0 makes the NPC turn to face the player and chase once lined up. The other flags are still unknown.
- **Event 1** is the only event in the demo, on the captive in room 4. It is a timed script that counts down a timer in the entity. The captive walks a few steps, stops, lifts the stone in front of it, turns and throws it, and then text 1 opens ("Now press Select…"). While it runs, the player is held in the lower part of the room.
- **Walk bits:** each tileset has a 256-bit walkable bitmap in bank 2, plus a second 256-bit set that the object code reads when something is put down. What that second set means is not settled.

The seven rooms

The six outdoor rooms form a 2×3 grid. Room 1 is the inside of the house in room 6.

```
room 4 (captive)   room 6 (house, "That's your house!")
      |                   |
room 3 (two chasers) room 5 ("Hello!")
      |                   |
room 0 (start) ---- room 2
```

Room	Tileset	Links to	NPCs (character type)	Texts
0	outdoor	2 (east), 3 (north)	–	a sign: "Welcome to Graal! Demo version by Stefan Knorr"
1	indoor	6 (the door)	–	–
2	outdoor	0 (west), 5 (north)	Ryu	–
3	outdoor	4 (north), 0 (south)	Pach Soldier, Squall	–
4	outdoor	3 (south)	Fung Si Yun (the captive), event 1	the captive's two texts
5	outdoor	2 (south), 6 (north)	Mandi, Lunar, Mandi	"Hello!" (all three NPCs)
6	outdoor	5 (south), 1 (the house door)	Vincent, Squall	a sign: "That's your house!"

The "Welcome to Graal" text is an ordinary sign in the first room, not a title screen. There is no title screen: `main` goes straight to room 0.

Room 4's texts, as the 18-character lines lay them out:

> Thanks for freeing me! I will follow you and help you completing your task. But first let me show how to lift stones! Press A to lift, A again to throw!
>
> Now press "Select" and switch to "Fung Si Yun"

PNG renders of all seven rooms with their real palettes, and of the whole grid, are in the shared folder under `png/`.

Text

- The text box is three lines of 18 characters at the bottom of the screen. The strings are padded with spaces so that words break at the right places.
- A new press on the d-pad while the box is open shows the next three lines. After the last page the box closes. In play, only a direction other than the one the player faces turns the page; pressing the facing direction does nothing.
- Character codes: `0x20` space, `0x21`–`0x3A` A–Z, `0x3B`–`0x54` a–z, `0x55`–`0x5E` 0–9, then punctuation (`0x5F` "!", `0x62` ".", `0x63` ",", `0x67` used both as an apostrophe and as a quotation mark). The tile drawn is the code minus `0x20`; the font sits at tile 0 of CGB VRAM bank 1.

Characters

Bank 3 holds a table of 31 character types. Each type has a name, three CGB OBJ palettes and a 16×16 head with two frames. Several types reuse another type's head with new colours.

#	Name	#	Name	#	Name	#	Name
0	Fung Si Yun	8	Zoraas	16	Oldman	24	Green Pach
1	Pach Soldier	9	Mad Max	17	Gold Soldier	25	Blue Lizard
2	Ninji	10	Pachukite	18	Gold Squall	26	Fire

#	Name	#	Name	#	Name	#	Name
3	Lunar	11	Matt	19	Gold Zoraas	27	Silver
4	Squall	12	Brandon	20	Gold Max	28	Theif (sic)
5	Vincent	13	Bob	21	Gold Matt	29	Red Squall
6	Mandi	14	Athena	22	Gold Silver	30	Water Athena
7	Ryu	15	Lizardon	23	Red Max		

Recoloured heads: Gold Soldier and Green Pach use Pach Soldier's head; Gold Squall and Red Squall use Squall's; Gold Zoraas uses Zoraas's; Gold Max and Red Max use Mad Max's; Gold Matt uses Matt's; Blue Lizard uses Lizardon's; Water Athena uses Athena's; Gold Silver uses Silver's.

These names are the character types of the demo, not only baddies. NPCs in rooms pick one by number, and so does the player.

- The player starts as type 0. Fung Si Yun, the captive in room 4, is type 0 too.
- **Select** opens a carousel in the window that shows three heads and the name of the middle one. Left and right turn it, and each step changes the player's own character type by one, wrapping through all 31. So the Select menu, as coded, is a free character picker over every type. It is not limited to the companion that the room 4 text mentions.

Entities

- There are at most eight entities. Entity 0 is the player and the rest come from the room's NPC list. Each one is 50 bytes.
- Fields that are understood: position, character type, direction, text index, event number, state, animation frame, OBJ palette, movement mode and bounds, a timer, a knock-back velocity, and the id of a carried object (−1 for none).
- States: 0 idle, 1 walking, 2 reading a text, 3 swinging the sword, 4 knocked back, 5 chasing the player, 8 carrying, 9 lifting. The update routine dispatches on the state. Movement mode 1 is a random walk; modes 2 to 4 patrol inside bounds kept in the record, and their exact rules were not worked out.

Player controls

Button	Effect
D-pad	Walk; turn the page in a text box (see below); turn the Select carousel
A	The character's A item: lift and throw for Fung Si Yun, a bomb for Ninji, a boomerang for Ryu, nothing for the rest (see below)
B	Sword, unless carrying. Every character has one; only its colour changes
Select	Open or close the character carousel
Start	Toggle music

Lifting works on 2×2 map objects. The game looks two tiles ahead and compares the object's top two tile ids with a list of four liftable kinds: three kinds of bush and stone, and a dropped object. For the first three it writes that kind's "lifted" 2×2 tiles into the map; for a dropped object it puts back the room's original tiles. The player goes into the lifting state, then carries the object as a sprite. A thrown object is tracked as an object of its own. In play it always broke where it came down, and nothing was written back into the map (see below). The "dropped object" kind in the list was never seen.

Sound

- There are eight one-shot sound effects on the two pulse channels and the noise channel. They are skipped while music plays.
- A music player is in bank 1, with a 72-entry table of note periods. **Start** turns it on and off.
- The player enables cartridge RAM before it reads, and its one song pointer is `$A000`, the start of cartridge RAM. The header declares no RAM, and nothing in the ROM writes to that area. So the demo carries no song of its own, and what Start plays depends on whatever answers at `$A000`. This is a static reading. It may be a leftover from a development cartridge that had a song loaded into its RAM.

Seen in an emulator

The ROM was run in PyBoy 2.8.1 in CGB mode, headless, with scripted input, screenshots every one or two frames, and the entity records read from WRAM while it ran. Contact sheets of every test, and the scripts that made them, are in the shared folder under `graal-decompile/gameboy-demo/emulator/`. Times are in 60 Hz frames. The game logic runs every second frame, so one logic tick is 2 frames.

Start and screen

- The Nintendo logo is followed straight away by room 0. There is no title, no HUD and no health display at any time.
- The CGB really is put into double speed.
- The camera keeps the player in the middle of the screen and stops at the room edges.

Walking

- The player moves 4 pixels per logic tick, which is 2 pixels a frame or 120 pixels a second. A room (256 pixels) takes about 2 seconds to cross.
- **Against a wall** the player keeps facing the wall and the walking animation keeps running in place. There is no push pose, however long the button is held.

Text

- Walking up into a sign (the stone post in room 0) or into an NPC with a text opens the box. It slides up from the bottom over a few frames.
- Capital letters are drawn in red and everything else in black.
- Select still opens the character menu while a text box is up.

Sword

- B swings the sword in an arc from one side to the front of the facing direction. One swing takes about 24 frames. Holding B swings again and again.
- A hit on an NPC knocks it a few pixels away, and it blinks for a moment. Nothing dies.
- The soldiers in room 3 chase the player and swing back. Their hits knock the player back the same way. Pinned against the top of the room, the player was knocked back over and over and could not get away, but nothing else happens: there is no damage and no game over.

Lifting, carrying and throwing (Fung Si Yun only)

- A in front of a bush, the stone post or another liftable 2×2 object swaps it for its "lifted" tiles at once (a dark stump for a bush). The object is drawn as a sprite where it stood for 10 ticks (20 frames), and then appears over the player's head.

- **While carrying**, the player is drawn facing the screen with both arms up, whatever the walking direction, and the object covers the player's head completely. So the carried object hides the head; it does not hide the whole player.
- A throws the object in the facing direction. It flies for about 6 ticks, then breaks into four leaf pieces that hang for about 8 ticks and vanish. This happened both when it hit a tree and on open grass. Nothing was put back into the map.
- Only character type 0 can lift. With any other type, A next to a bush did nothing.

Respawn

- Lifted bushes do not grow back while you stay in the room. One was still a stump after two minutes.
- Leaving the room and coming back restores every bush and stone, because the room is reloaded from ROM.

The character menu and A items

- The menu fills the whole window. It shows three heads with arrows either side, the middle character's name with capitals in red, and "Button B" and "Button A" with an item icon under each.
- Every character has a sword under B, in one of three colours (red for most, blue for Pach Soldier and Gold Soldier, purple for Mandi).
- Only three characters have anything under A, and the icon is chosen in code by the character number:
  - **Fung Si Yun (0):** a bush, meaning lift and throw.
  - **Ninji (2):** a bomb. A puts it on the ground in front. It sits for about 110 frames, flashes for about 70 more, and then blows up as a large plus-shaped blast about three tiles each way. The screen jumped as the blast went off, which looks like the blast knocking the Ninji player back; that was not checked in the entity record.
  - **Ryu (7):** a boomerang. It flies about four tiles in the facing direction and comes back to the player, about one second in all.
- This is more than the static read found. The A items are a second action set, picked by character, and the room 4 text ("switch to Fung Si Yun") is about getting the lift action.

Rooms and NPCs

- All seven rooms loaded and looked like the PNG renders made from the ROM.
- Room 2: Ryu stands still. Room 5: Mandi, Lunar and Mandi stand still; by the room data, each says "Hello!".
- Room 3: the first soldier patrols left and right along the top of the room at 1 pixel a frame, half the player's speed. The second wanders. Once lined up with the player, both chase at a little over 1 pixel a frame and swing their swords.
- Room 6: Vincent and Squall wander around the house.
- **Room 4, the captive.** Walking into Fung Si Yun opens the first text. When it closes, he walks two tiles left, lifts the bush there, turns and throws it, and the second text opens ("Now press Select and switch to Fung Si Yun"). This matches the event script in the code. The player and the captive look the same, because the player also starts as Fung Si Yun. The text reads as if the player was meant to start as someone else.

Sound

- The sound effects play (a noise burst for the sword).
- **Start silences the game.** With nothing at the song address, reading it gives `$FF` (as on a cartridge without RAM). The music player then writes 0 to the master volume and the channel routing, and Start again does

not undo it. From then on the sword and every other effect are silent until the game is reset. How `$FF` is read here depends on the emulator, but it is also what a real cartridge without RAM usually returns.

Compared with the PC client

- The same link record: a rectangle plus a new x and y, with −1 to keep a coordinate.
- The same direction numbers, and the same lift, carry and throw of 2×2 bushes and stones with A. Here the liftable objects are a fixed list of tile pairs.
- Rooms are fixed 32×32-tile maps, not 64×64 levels. Positions are in half pixels, and there is no NPC scripting: NPC behaviour is a few built-in movement modes, flags and event numbers.
- The text alphabet is a straight ASCII-like run, not the PC sign alphabet.

What a Seihai "Game Boy demo" profile would need

This is the look and feel the run showed, as a list for a profile that copies the demo.

- One 256×256-pixel room at a time, with no scrolling between rooms: a link loads the next room with the screen off.
- Movement at 2 pixels a frame (logic at 30 Hz, 4 pixels a tick). No push pose against walls; the walk cycle keeps running.
- A text box of three lines of 18 characters in the window, sliding up, capitals in red. Paging by a d-pad direction other than the one faced.
- No HUD, no health, no death. Hits only knock back.
- Sword on B for everyone. A depends on the character: lift and throw, bomb, boomerang, or nothing.
- Carried objects cover the head, and the carrier is drawn facing the screen.
- Thrown objects always break. Nothing respawns until the room is loaded again.
- A Select menu that picks any of the 31 characters, with their head, name and B/A icons.
- No chat and no network. There is no chat bubble in the demo.

Open

- The music data format, since no song is in the ROM. Pressing Start mutes the game rather than playing anything.
- Several entity fields and most NPC flag bits.
- The exact patrol rules of movement modes 2 to 4. The run showed a left-right patrol and a wander, not the rules behind them.
- Whether the Select carousel was meant to stay a free picker in a later version.
- The "dropped object" kind in the liftable list, which was never seen in play.

`client-internals/relay-room-bridge.md`

Java Graal meets the Delphi betas: a relay room bridge (2026-10-31)

Question: can Java Graal (July 2000) and 5th Graal (October 1998) play in one room? Both talk to Stefan Knorr's GServ relay lobby (`protocol/relay-1998.md`), but they join different rooms. This note runs them, and 4th Graal alpha, live against Knorr's own GServ, and tries a proxy that renames rooms.

Tags: **[live]** seen in running clients or on the wire, **[bin]** read from the binaries' strings. No decompiler output is reproduced.

Result

Pair	Relay	Meet?	What went across
Java Graal + 4th Graal alpha	stock GServ	**yes** [live]	both drawn, nicknames, chat both ways
Java Graal + 5th Graal	stock GServ	no [bin, live rooms]	different rooms (`G2<level>` vs `G3<level>`)
Java Graal + 5th Graal	GServ behind the bridge, `G2 → G3`	**yes** [live]	both drawn, nicknames, walking both ways (5th Graal's from run 4), chat both ways, a level change and back

So the "Java Graal speaks 4th alpha's dialect" reading in `precursors-and-fan-builds.md` holds on the wire, and its last open point is closed: 4th alpha joins `G2` + the bare level file name (`G2level13.graal`), the same string the applet builds. For every later beta a one-line rename in the join is enough to put them together; the `D` payloads already line up.

Rooms each build joins

Every Delphi beta opens **two** connections to the relay: one to a room per level, one to a fixed game-wide room. The Java builds open one.

Build	Level room	Game-wide room	Basis
1st Graal	`Zhouse12` (`Z` + level, no extension)	`Zelda2`	[live]
2nd Graal	as 1st	`Zelda2`	[bin]
3rd Graal	`Ghouse1.graal` (`G` + level file)	`Graal`	[live]
4th Graal	`G<level>`	`Graal`	[bin]
4th Graal alpha	`G2level13.graal`	`Graal2`	[live]
5th Graal	`G3level13.graal`	`Graal3`	[live]
5th alpha, 6th to 1.1.9	`G3` … `G8` + level	`Graal3` … `Graal8`	[bin] (1.1.9 also in `live-trace-early.md`)
Java Graal	`G2level13.graal`	none (a `Graal2` mode exists but is never selected)	[live]
Zelda Online 2	`Zlevel11.6` (`Z` + base + `1.6`)	none	[live], 2026-10-30

1st Graal's `Zhouse12` is either `Z` + `house12` or `Z` + `house1` + `2` (the `Z` + base + `2` branch in `early-clients.md`); one run does not tell them apart. Either way it never equals Zelda Online 2's `Z<base>1.6`.

How it was run

- **Server:** Zelda Online 2's `server/GServer.java` (Knorr's GServ Standalone), compiled with a current JDK and run unchanged on 14801.
- **Bridge:** `tools/relay-bridge/room_bridge.py` on 14802, in front of GServ. It passes every line unchanged except `N<room>#<name>` / `O<room>#<name>`, where a rule `OLD=NEW` swaps the room prefix (`G2=G3` turns `G2level13.graal` into `G3level13.graal`). It logs every line both ways. With no rule it is a plain logging proxy.
- **Java Graal:** the 2002 rip's `Graal.jar` (the loose class files in the rip are truncated), served over HTTP as the applet's codebase and started with `tools/relay-bridge/AppletRun.java`, a small `AppletStub` frame. The applet dials its codebase host on 14802, so it always goes through the bridge.
- **Delphi betas:** the shipped `Graal.exe` under Wine 9 (Xvfb, openbox, the Wine virtual desktop), started with `WINEDLLOVERRIDES="dinput=d"` so the arrow keys work (see run 4). `tools/relay-bridge/`

`run_delphi_beta.sh` sets this up. From 3rd Graal on the server box is greyed out at Cyberjoueur's dialog address on `<loopback-outside>:14802` (the historical fixed IPv4, remapped onto the loopback interface as a fake outside address so the beta reached the bridge). 4th alpha and 1st Graal keep an editable the loopback address on 14801 and went straight to GServ (captured with tcpdump, decoded with `tools/relay-bridge/pcap_lines.py`).

Captures and screenshots: `live-runs/relay-bridge-2026-10-31/`.

Run 1: Java Graal and 5th Graal through the bridge

```
room_bridge.py --listen 14802 --server <loopback>:14801 --map G2=G3
```

- Java Graal joined `G2level13.graal`; the bridge sent `NG3level13.graal#def`. 5th Graal joined `G3level13.graal` and `Graal3`. GServ announced each to the other (`+`), both sent `P`, and the state burst followed.
- 5th Graal drew the Java player, its walking (a run of `P` lines) and its chat (`hi from java`). Java Graal drew the 5th Graal player with its nickname (`Fifth`) and chat (`hello from 5th`). (`run1-java-to-5th.png`, `run1-5th-to-java.png`.)
- Java Graal walked into `house10.graal` and back: it sent `C` and `-`, then joined `G2house10.graal` (bridged to `G3house10.graal`), and on return `G3level13.graal` again, and the 5th Graal player was there again.
- `D` types sent in the run: Java Graal `T P C N D`; 5th Graal `T P Y C N S`. Java Graal ignores `Y` and `S` (colours), so the 5th Graal player shows in default colours there. Nothing broke on either side.
- 5th Graal's own walking was not seen in this run: its arrow keys did not reach the game under Wine, so its player stood still. Run 4 fixes that and shows the walking.

Run 4: 5th Graal walks in the shared room

Same setup as run 1 (`--map G2=G3`), with 5th Graal started under `WINEDLLOVERRIDES="dinput=d"`.

- **Why the keys were lost.** 5th Graal reads the keyboard through DelphiX `TDXInput`, which loads DirectInput. Wine's `+dinput` trace shows the keyboard data format it sets: `dwDataSize 256, dwNumObjs 1`, one object at offset 1 (`run4-dinput-format.txt`). Windows fills the whole 256-byte key buffer anyway. Wine fills only the objects listed, so the arrow-key bytes stay zero. Chat typing and mouse clicks arrive as window messages, which is why they worked. This is the same fault as in the 1.x clients (`live-trace-early.md`, `live-gserver13.md`).
- **The fix.** With `dinput=d` the DirectInput load fails, DelphiX falls back to `GetKeyboardState`, and `xdotool keydown` reaches the game. This is the setting the 2026-10-30 six-era runs used (`version-profiles/matrix/live-runs-2026-10-30.md`). No change to the exe and no memory patch is needed. The game window must have focus (`xdotool windowactivate`/`windowfocus`), which the openbox + virtual desktop setup allows.
- **Before:** without the override, holding Right for 1.5 s sent nothing. **After:** holding Right, Down and Left sent a stream of `D #P…` lines, one every 100 ms while a key was held, each relayed to Java Graal (`run4-5th-java-walk-bridge.log`). [live]
- Java Graal drew the `Fifth` player walking right across the level13 yard (`run4-5th-walks-java-view.png`, before and after). [live]
- 5th Graal's own screen often stops repainting the board under Wine (as noted for 2.05 in `board-respawn.md`), so judge its movement from the wire or from the other client, not from its own window.

Run 2: Java Graal and 4th Graal alpha, no rename

Bridge with no rules (Java side), 4th alpha straight to GServ.

- 4th alpha joined `G2level13.graal` and `Graal2`; Java Graal joined `G2level13.graal`. Same room, so GServ paired them with no help.
- Chat went both ways (`java meets fourth`, `fourth says hi`), and each drew the other with its nickname (`run2-4thalpha-java-chat.png`).
- 4th alpha sent `T P Y C N S`, Java Graal `T P C N D`, as in run 1.

Run 3: rooms of 3rd and 1st Graal

Only the joins were needed: 3rd Graal sent `NGhouse1.graal#def` and `NGraal#def`; 1st Graal sent `NZhouse12#def` and `NZelda2#def`.

What this means

- A relay that serves these builds together needs only the join rename. The `D` lines are already the same dialect from Zelda Online 2 through 1.1.9, and each side ignores the types it does not know.
- The rename must keep the level part: mapping a level room onto a game-wide room (for example `G2<level>` onto `Graal3`) would put players on different levels in one room.
- Bridging other pairs is the same rule with other prefixes (`G2=G5` for Java Graal with 7th to 9th Graal, and so on). Only 4th alpha and 5th Graal were run.
- Seihai lists all of these builds as offline-only (it has no relay client); its rows carry the rooms above (`documentation/offline-only-builds.md` in Seihai).

`client-internals/live-trace-early.md`

Live runs of the pre-1.33 clients (1.1.9 to 1.32)

Date: 2026-10-30

These are captures of real clients and their bundled servers running under Wine, inside a network namespace that has loopback only. They check the static notes in `protocol/relay-1998.md`, `protocol/gserv-1999.md` and `client-internals/version-profiles/v121-v132.toml`. Tags: **[cap]** seen on the wire in these runs. The raw pcaps and decoded dumps are in the project files under `graal-decompile/live-trace/early/`.

How the runs were made

- Wine 9 (win32 prefix), Xvfb with openbox, Direct3D renderer set to `gdi`. Every pre-1.33 client opens its window under Wine.
- Servers: the 1.1.9 `Gserver.exe` (relay), GServer 1.28 (`GServerv.exe`, from the 1.28 beta 2 zip) and GServer13 (from the 1.3 test zip). Accounts were added in the server's Edit accounts window. The 15th Graal `.graal` levels were copied in, because the later zips don't ship `onlinestartlocal.graal`.
- Clients are pointed at the lab with `graalconfig.txt` (`gamemode=One Player Online`, `serveraddr` set to the loopback address). For 1.1.9 the connect dialog's default host `<list-host-classic>` was mapped to the loopback address.

- **1.21** ships only as an InstallShield 3 package. Its setup was run under Wine, and the installed folder (218 files) is kept in the project files under `graal-decompile/unpacked-installers/graal121/`. The installed `Graal.exe` and `GServer.exe` have the same MD5 as the copies analysed statically.
- **Keyboard under Wine.** These clients give DirectInput a keyboard data format that lists one object over a 256-byte buffer. Wine only fills listed objects, so arrow keys never reach the game, while chat and Alt keys still work because they arrive as window messages. A live patch that widens the format to 256 key objects in memory fixes this in 1.3 test and in 1.21. The exe checks its own checksum, so the file itself can't be edited. With the patch, Wine's dinput trace shows `dwNumObjs 256`. In 1.21 the player then walks. 1.28 beta 2 walks with the same patch, plus `joystickon=false` in `graalconfig.txt`. 1.1.9 walks with the same patch, and both 1.1.9 clients in a two-player run could move.
- **1.28 UDP receive under Wine.** At first only the 1.28 instance started first could move. The second stopped polling its keyboard right after login. The cause is in the game's UDP receive routine. It calls `recvfrom(sock, buf, 0x400, 0, &from, &fromlen)` without ever setting `fromlen`, a local variable that held a stack address (about 3.2 million) in the traced runs. When the first datagram arrives (an arrow from the other client), Wine fills the address buffer out to that length and writes past the top of the main thread's stack. The exception chain on the stack is then overwritten, and the thread loops through nested access violations until the stack overflows. The window stays open and Wine keeps passing it key events, but the game loop is dead. The first instance never got a datagram (the second died before sending one), so it kept running. Windows checks only that `fromlen` is at least 16 and writes 16 bytes, so the bug would not show there (*inferred*). An in-memory patch that sets `fromlen` to 16 before the call fixes it. With the patch on both instances, both players walk and UDP flows both ways. The patch script is `udpfix.py` in the project files.

GServer 1.21 with 1.21 (GSERV007)

[cap]

- The login frame has the same shape, with `GSERV007`. The installed server ships `accounts.txt` with two accounts, `guest`/`guest` and `super`/`super`. Each line holds `chr(len+32)` + name, `chr(len+32)` + password, then the starting props in the same encoding as id 9.
- The reply is **one zlib frame** holding 9 (own props), 14 (warp to `level1.graal`), 100, 101 with the board, 6 (level name), 1 (links) and 10. 1.28 and 1.3 test put 101 in a frame of its own. There is no 39, as in 1.28.
- Walking sends one id 2 message every 100 ms with X/Y and SPRITE, as in 1.3 test.
- The installer pops up "No MCI device open." under Wine (no MIDI device). Pressing OK continues.

GServer13 with 1.3 test (GSERV014)

Run by the live-trace lab. **[cap]**

- The first client frame is a u16 big-endian length followed by a zlib stream holding `" GSERV014" + chr(5+32) "guest" + chr(5+32) "guest" "\n"`, as in `gserv-1999.md`.
- A bad account gets id 16 with "An account with the name and password you typed in doesn't exist."
- Reply order after a good login: 9 (own props), [28 (flags), only when the account has flags], 14 (level warp: x, y, level), 100 (raw size) and then a separate zlib frame holding 101 and the board, 6 (level name), 1 (links), 5 (signs), **39 (level modification time)**, 3 (NPC props, scripts included), 10.
- The client then asks for NPC images with 23, sets flags with 18, and adds the level's horses with 7. The server answers 30 (file send failed) for the missing images, and deletes the horses with 18 when the horse lifetime runs out (10 s).
- Walking sends one id 2 (props) message per 100 ms with X and SPRITE. Chat is id 2 with prop 12.

GServer 1.28 with 1.28 beta 2 (GSERV013)

[cap]

- The login frame has the same shape, with `GSERV013`.
- The reply order is the same as 1.3 test, except that **there is no id 39**. The level-timestamp cache is new in 1.3 (the 1.3 notes: "levels will be saved on the local hard disk … they contain a timestamp").
- The own-props string (id 9) is one prop shorter than in 1.3 test, which ends with an extra prop (`@R`). That fits a parser that stops at prop 31 in 1.28 beta 2 and 32 in 1.3 test (`v121-v132.toml`).
- A second login with an account that is already online gets id 16 "The account is already in use." The account's first session keeps playing.
- **Second player.** The newcomer gets its own 9 and then, in the same frame, an 8 (other player's props) for the player already in the level, before the 14 warp. The player already there gets 8 messages about the newcomer, starting with the nickname and level. The ids in those 8 messages are the server's player numbers (1 and 2 here). Where the UDP address and port are carried has not been decoded. The other-player props are the obvious place (*inferred*).
- **UDP.** Once both clients are in the same level they send UDP datagrams straight to each other's port, without going through the server. Each datagram is one uncompressed message, `chr(id+32) + data`.
  - **Walking:** the moving client sends **UDP id 0** to the other player every 100 ms. The five bytes are the player props X, Y, SPRITE, STATUS and CARRYSPRITE (15 to 19), each +32. That is the same field list as UDP id 0 in 2.05 (`protocol/udp-p2p.md`).
    - X and Y are in half tiles; Y went up by 2 per step while walking down.
    - SPRITE is animation frame·4 + direction (0 up, 1 left, 2 down, 3 right). Walking down cycled 22, 14, 6, 14 and stopped at 2.
    - STATUS was 16 (`0x10`) and turned 20 (`0x14`) while the player crossed the lake south of the start, then back to 16 past it. At the same moment the client sent prop 18 = 20 over TCP. So `0x04` is the swimming state in this build (*inferred* from where it flipped; later servers name `0x04` "male").
    - The trailing `0x1F` is CARRYSPRITE −1 (nothing carried).
  - At the same time the walking client sends TCP id 2 to the server with only the changed props (16, 17 and, when it flips, 18). **The server forwards nothing to the other player.** The other client sees the walk only through UDP. That is the "player positions & sprite numbers … sent directly to all players that are in the same room" of the 1.28 notes.
  - **Arrows:** id 12 twice every 0.5 s is **arrow add**. The level `onlinestartlocal.graal` has two NPCs whose scripts run `shootarrowleft` and `shootarrowup` on a 0.5 s timeout. The payloads are x, y (half tiles, at each NPC), direction (1 left, 0 up), a type byte, and a last byte sent as `v+33`, the same layout as the later `ARROWADD` (`protocol/packet-layouts.md`). Only one client sends them, to every other player in the level. In a three-client run the first player in the level sent them to the other two. When it quit, the player who had joined second took over within a second and sent them to the third. So 1.28 has one player per level that runs the NPC timeouts for everyone, and the role passes on when that player leaves. **The server picks that player** with id 10 (`ISLEADER`, no payload). The first player's login reply ended with 10, and the second and third players' replies had none. When the first player quit, the server sent 10 to the player who had joined second, and then sent 8 with prop 51 (player gone) about player 1 to both remaining players. The arrows from the new leader started straight after. 1.21 and 1.3 test also end the first player's login reply with 10, so the leader role is older than UDP.
  - A newcomer sends one UDP id 22 with no payload to each player already in the level (seen in the two- and three-client runs). 2.05 uses 22 for chat, so this is most likely the newcomer's empty chat text (*inferred*).

GServer13 with 1.31 (GSERV016) and 1.32 (GSERV018)

[cap]

- Both clients send the same login frame with their own token.
- GServer13 refuses both with id 16 "You need at least Graal version 1.3 to play on this server." A token newer than the server's list is treated as too old. It does not cause a disconnect without a message, or an "unknown version" error.
- **Saved password.** 1.31 and 1.32 (and, by string search, 1.3 test to 1.39) save the login under `HKCU\Software\Graal\Client`: `accountname`, `nickname` and `password`. The password is the 48-character DES string form from `saved-passwords.md`, with the same key that 2.05 uses. `tools/file-formats/graaldes.py` turned the value from this run back into `guest`. So the warning in `saved-passwords.md` applies from 1.3 test on, not only to 2.x. 1.28 beta 2 still reads `accountname=` and `password=` as plain text from `graalconfig.txt`.

1.1.9 Gserver with two 1.1.9 clients (relay)

Server started with "Use local levels". **[cap]**

- **Each client opens two TCP connections.** One joins the game `Graal8` and stays in it. The other first joins `G8` with no level, then leaves (`-`, `C`) and re-joins as `G8onlinestart.graal`, then `G8onlinestartlocal.graal`, as the client moves through levels. All lines go to the same server port.
- After every `N` the server answered `NR`, then `NL`, then `I<id8>`. Ids are `0000000n`, numbered across all connections in the order they connect (the three-client run below gave 1 to 6). A connection keeps its id when it changes room.
- **Both connections pair up.** When the second client joined, each connection got `+<id>def` for the other client's connection in the same game. The client answered `P<id>`, and the server confirmed with `P` to both sides. That happened first on `Graal8` and then on `G8onlinestartlocal.graal` once both clients were on that level. The `Graal8` pair carried only the first state push (6 `D` lines each way). The level pair carried the rest (about 120 `D` lines each way in a minute). So the per-level room is where play traffic goes, as `relay-1998.md` says, and the fixed `Graal8` room is an extra channel. What the client uses that channel for beyond the first push was not seen in this short run.
- The state push after `P` is, in order: `L` (level, sent twice as the client moves on), `P` (position), `N` (nickname), `S` (colours), `7` (pose flag), and later `T` (10-character login time), `Y` (sword/shield power) and `C`.
- `D` lines carry the sender id as two base-128 characters (`"  "` = 0, `"!"` = 1 …), as in `relay-1998.md`.
- **Type `A` is an arrow shot.** Each client sent `A` about four times a second, alternating two 5-byte payloads. They are the arrows of the two shooting NPCs in `onlinestartlocal.graal` (see the 1.28 UDP id 12 notes above). Unlike 1.28, both 1.1.9 clients sent them, because each client runs the level's NPC scripts itself.
- **Walking.** With the keyboard patch both 1.1.9 clients walk. The moving client sends `D` type `P` on the level connection every 100 ms: x and y in half tiles, then frame·4 + direction, the same encoding as SPRITE in 1.28. Walking right gave `^\3`, `` `\3 ``, `b\+` … (x up by 2 per step, y fixed); stopping sent the idle frame (`#` = 3, facing right).
- **"Use levels from cyberjoueur".** With that option the server sent `NR` alone after each `N`, with no `NL`. The client then joined `G8onlinestart.graal` and stayed there, instead of moving on to `onlinestartlocal.graal`. It opened no other connection (no HTTP).

1.1.9 Gserver with three 1.1.9 clients (relay)

Server started with "Use local levels". **[cap]**

- Every connection pairs with every other connection in the same room. When the third client joined `Graal8`, the server sent it `+` for both existing `Graal8` connections, and sent each of them `+` for the newcomer. The same happened in the level room. Each pair then exchanged `P`. So a relay game is a full mesh of pairs, and every `D` line goes to all the sender's partners.
- Each time the newcomer's level connection joined a room, the server first sent it `C<id>` for every other connection id, including ones it was about to pair with, then the `+` lines. This looks like a reset of old partnerships before the new ones (*inferred*).
- When a client quit, the server sent `C<id>` for its level connection to the partners in that room, and `-<id>` for both of its connections to every connection, in all rooms. The other clients carried on, and the remaining ones could still walk.
- **No leader.** All three clients sent the NPC arrows (`A`) all the time, and the two remaining clients went on doing so after the first quit. In the relay each client runs the level's NPC scripts for itself. The leader in 1.21 and later comes from the GSERV server (id 10, above).

What this changes

- `gserv-1999.md`: the login reply order is now the captured one, plus the 1.28 difference (no 39), the refusal of newer tokens, the second-player 8 messages and the UDP shape.
- `relay-1998.md`: the two connections, `NR` then `NL` (or `NR` alone with Cyberjoueur levels), `A` = arrow shot, and the `P` sprite encoding.
- `gserv-1999.md`: UDP id 0 = X, Y, SPRITE, STATUS, CARRYSPRITE, id 12 = arrow add, and id 10 = level leader.
- `v121-v132.toml`: new `level_leader` feature.
- `v121-v132.toml`: `credentials_in_registry` is now confirmed and names the DES string form.

Open

Nothing from these runs is left open.